

Automatically Generating ML Compiler Backends from Tensor Accelerator ISA Descriptions

DEVANSH JAIN, University of Illinois Urbana-Champaign, USA

AKASH PARDESHI, University of Illinois Urbana-Champaign, USA

MARCO FRIGO, University of Illinois Urbana-Champaign, USA

KAUSTUBH KHULBE, University of Illinois Urbana-Champaign, USA

KRUT PATEL*, NVIDIA, USA

SAATVIK LOCHAN, University of Illinois Urbana-Champaign, USA

JAI ARORA, University of Illinois Urbana-Champaign, USA

CHARITH MENDIS, University of Illinois Urbana-Champaign, USA

Machine learning (ML) compilers play a key role in enabling high-performance implementations of ML workloads. These compilers use existing CPU and GPU backends to generate device-specific code. In recent years, many tensor accelerators (or AI accelerators) have been designed to further accelerate these workloads, with commercial products like AWS Trainium publicly available. However, compared to commodity hardware, a majority of tensor accelerators do not have mature ML compiler backends with robust code generation support. Moreover, tensor accelerator designs are subject to fast iteration cycles, making it difficult to manually develop and maintain ML compiler backends. Therefore, to enable faster integration of novel tensor accelerator designs in ML infrastructure, we need to make the compiler backend construction process more agile.

In this paper, we introduce ACT, a compiler backend generator that automatically generates compiler backends for tensor accelerators, given just the instruction set architecture (ISA) descriptions. These backends are integrated with XLA, a production ML compiler. ACT uses a novel ISA-parameterized compilation algorithm to generate a compiler backend with an equality-saturation-based instruction selection phase and a constraint-programming-based memory allocation phase. We generated compiler backends for 6 accelerator platforms from industry (e.g., AWS Trainium, Intel AMX) and academia (e.g., Gemmini). We showed that these generated backends match or outperform commercial compiler backends and expert-written kernel libraries, while maintaining low compilation overheads. Notably, ACT-generated backend for AWS NKI ISA improved the code generation coverage for AWS Trainium by 2.3x compared with AWS's production compiler, `neuronx-cc`.

ACT is part of a larger open-source ecosystem, built around our ISA description language TAIDL, that automatically generates essential software tools, such as test oracles and compiler backends, from ISA descriptions of tensor accelerators. Our tooling has been adopted by multiple academic and industry teams designing novel tensor accelerators. The ecosystem is available at <https://github.com/act-compiler/act>.

CCS Concepts: • **Software and its engineering** → **Compilers; Retargetable compilers.**

Additional Key Words and Phrases: ML Compilers, AI Accelerators, Automated Compiler Construction

*Work done while at University of Illinois Urbana-Champaign.

Authors' Contact Information: Devansh Jain, University of Illinois Urbana-Champaign, USA, devansh9@illinois.edu; Akash Pardeshi, University of Illinois Urbana-Champaign, USA, pardesh2@illinois.edu; Marco Frigo, University of Illinois Urbana-Champaign, USA, mfrigo3@illinois.edu; Kaustubh Khulbe, University of Illinois Urbana-Champaign, USA, kkhulbe2@illinois.edu; Krut Patel, NVIDIA, USA, krutp@nvidia.com; Saatvik Lochan, University of Illinois Urbana-Champaign, USA, slochan2@illinois.edu; Jai Arora, University of Illinois Urbana-Champaign, USA, jaia3@illinois.edu; Charith Mendis, University of Illinois Urbana-Champaign, USA, charithm@illinois.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART325

<https://doi.org/10.1145/3839457>

1 Introduction

In recent years, machine learning (ML) workloads have gained popularity, specifically deep learning models. ML practitioners use ML frameworks such as Tensorflow [1], PyTorch [4], and JAX [35] to express their computations, and ML compilers such as XLA [20], TorchInductor [4], and TVM [17] to lower these computations into highly performant executables targeting CPUs and GPUs.

Akin to general-purpose compilers, ML compilers also use a hierarchical compilation pipeline (Fig. 1). The frontend lowers programs written using ML frameworks to a tensor-operator-based intermediate representation (IR) known as tensor computation graphs. These graphs are then optimized by several graph-level optimization passes [77] such as algebraic simplification and layout selection. The scale of these optimized graphs [76] can range up to 1000s of nodes. This is followed by operator fusion and tiling passes, which emit 100s of *tilted kernels* (sub-graphs with typically 10–50 nodes). These tiled kernels are compiled to device-specific assembly code individually using *existing* compiler backends. For example, XLA [20], a popular ML compiler, uses the existing lowering pipeline in MLIR [60] and LLVM [59] to compile these tiled kernels in XLA-HLO IR [22], XLA’s tensor IR, into assembly code for commodity hardware (CPUs and GPUs).

To get the best performance out of ML workloads, many domain-specific tensor accelerators (a.k.a. AI accelerators) have been designed. They have shown superior runtime performance, and the fast proliferation of such accelerator designs in both academia and industry is a testament to their importance and popularity. For example, consider commercial tensor accelerators such as Google TPUs [55, 56], AWS Trainium [81, 82] and Intel AMX [46], and a large body of academic designs such as Gemmini [36], MAERI [58], FEATHER [93], EVA [104]. These accelerators are diverse, with different software-managed memory hierarchies and instruction sets (compute capabilities).

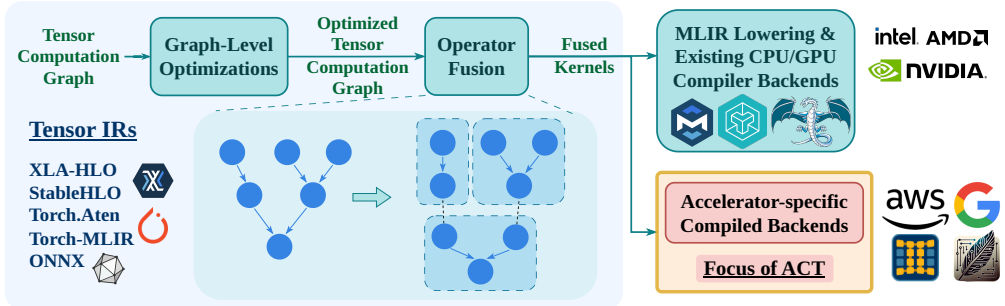


Fig. 1. Typical hierarchical compilation pipeline present in ML compilers. Our solution, ACT, focuses on compiler backend construction (shaded red) for tensor accelerators like AWS Trainium [82] and Gemmini [36]. We use XLA compiler [20] as our reference ML compiler, which uses XLA-HLO IR [22] and StableHLO IR [23].

1.1 Need for Generation of Accelerator-specific Compiler Backends

Widespread use of a hardware architecture requires compiler backends that generate device-specific code, as evident for CPUs and GPUs — companies like Google and Amazon [89] have invested heavily in compiler toolchains for their tensor accelerators [54, 55, 81, 82]. In contrast, a majority of tensor accelerators proposed in the literature do not have mature backends and are often evaluated only on synthetic workloads [43], which *inhibits their integration* with popular ML compilers.

Tensor accelerators without mature compiler backends instead rely on custom hand-written kernel libraries, which support only a limited set of kernels and miss out on fusion opportunities outside this set [25, 41, 70, 71]. For the Gemmini [36] accelerator, we observe that operator fusion can lead to over 1.7x better performance and a 50% reduction in data movement (§8.6). Therefore, compiler backends that support code generation for *arbitrary tiled kernels* are needed.

Manually constructing such backends for every accelerator is too tedious and too slow to adapt to the fast pace of accelerator research (§2.3). Therefore, we propose an *agile* methodology — *automated generation* of compiler backends for the multitude of tensor accelerator designs.

1.2 Challenges & Goals

Automatically generating compiler backends that support code generation for arbitrary tiled kernels for many diverse tensor accelerators comes with new challenges not present in commodity hardware (e.g., prior works like vectorizer generators for vector ISAs [18, 91, 94]).

Challenge 1: Code generation and optimizations for software-managed scratchpads.

Most tensor accelerators have software-managed scratchpads with different memory hierarchies. The data access patterns of tensor accelerators vary significantly in shape and size with multi-dimensional addressing (e.g., AWS Trainium [84]) and multi-dimensional base elements (e.g., 16×64 matrices in Intel AMX [46]). An ideal compiler backend needs to *automatically* manage data allocation and reuse for these scratchpads. Works such as Exo [45] model scratchpads themselves, but leave the data allocation to end-users, whereas 3LA [43] does not model scratchpad reuse.

Challenge 2: Handling parameterized ISA instructions. Tensor accelerators have instructions with parameters that control execution counts. For example, the Google TPU [56] and AWS Trainium [84] instructions are parameterized by input shapes (e.g., $B \times 128$). Methodologies adopted by existing vectorizer generators [18, 91, 94] can only handle fixed-size vector instructions.

Challenge 3: Handling complex ISA descriptions. Tensor accelerator ISA descriptions often include complex data layout transformations [46, 82, 93], that require accelerator-specific legalization passes to support such instructions. Existing kernel libraries [45, 46] often rely on long and tedious pattern-matching rules manually written by experts for each new accelerator.

Moreover, the compiler backend generator should ideally achieve three goals.

Goal 1: Increased compilation coverage. Tensor accelerator designs often require expensive host fallback to execute certain operators (or sub-graphs) due to their limited compute capabilities. The compiler backend should have a large compilation coverage, i.e., correctly compile for all tiled kernels supported by the accelerator — in other words, a *sound and complete compiler backend*.

Goal 2: Full automation. To minimize the engineering effort, the compiler backend generator should *fully automate* the backend generation process with just the ISA descriptions as input.

Goal 3: Interoperability with existing ML compilers. The generated compiler backends should be *compatible with existing ML compilers* to facilitate seamless integration and deployment.

1.3 Our Solution

In this paper, we introduce **ACT (Accelerator Compiler Toolkit)**, the first compiler backend generator that *automatically* generates *sound and complete* compiler backends for tensor accelerators from just their ISA descriptions. ACT consumes ISA descriptions written in TAIDL [51], a popular ISA description language that supports diverse accelerator designs [36, 46, 56, 84, 93, 104]. ACT achieves automatic generation (goal 2) by *parameterizing* the compilation algorithm with TAIDL [51] constructs, which are concretized for each accelerator given their ISA descriptions. ACT-generated backends are compatible with XLA [20], a popular ML compiler (goal 3). These backends consume tiled kernels in XLA-HLO IR as input and produce accelerator-specific assembly code.

We introduce a novel intermediary, *partially instantiated instructions (pii)*, to handle instructions parameterized by input matrix shapes (challenge 2), prevalent in tensor accelerator ISAs. We term such instruction parameters/attributes as *computational* (α), with the rest termed as *addressing* (β).

ACT's *ISA-parameterized* compilation algorithm consists of two main phases—one for solving each attribute set. The first phase introduces *parameterized* instruction selection based on equality

saturation [90] and uses multiple types of rewrites (e.g., IR-to-IR, IR-to-ISA). ACT generates the IR-to-ISA rewrites from the TAIDL descriptions. These generated rewrites, combined with the XLA's IR-to-IR rewrites, perform IR legalization (challenge 3) and resolve the computational attributes (α). The second phase introduces *parameterized* memory allocation for multi-dimensional buffer accesses (challenge 1) based on constraint programming on the addressing attributes (β). We introduce intra- and inter-phase fallback paths to ensure completeness of the algorithm (goal 1).

In summary, the paper makes the following contributions.

- We introduce ACT, the first compiler backend generator that *automatically generates* compiler backends for tensor accelerators, given just their ISA descriptions in TAIDL. As a core component of ACT, we present a novel *ISA-parameterized* compilation algorithm. (§4–§6)
- We augment ACT-generated backends with cost models to generate performant code. (§7)
- We use ACT to generate compiler backends integrated with the XLA compiler for **six** popular accelerator platforms from industry (AWS Trainium, Intel AMX, Google TPUv1) and academia (Gemmini, FEATHER, EVA), demonstrating the generality of ACT. (§8.2)
- We show that the ACT-generated compiler backends match or outperform existing accelerator-specific backends and expert-written kernel libraries (up to **6.88x** speedup). We also improve compilation coverage for AWS NKI by **2.3x** over AWS's compiler `neuronx-cc`. (§8.3–§8.6)
- We show that compilation times of the ACT-generated compiler backends are less than a second for a wide range of kernel sizes, even for large kernels (**529 ms** for 390 nodes). (§8.7)

ACT is part of a larger open-source ecosystem built around our ISA description language TAIDL [51]. The ecosystem and its associated tutorials are accessible at <https://github.com/act-compiler/act>.

2 Background

We first briefly discuss XLA-HLO IR, tensor accelerators, and equality saturation workflow, providing the background needed for the problem formulation (§3) and our solution (§4–§7).

2.1 XLA-HLO IR: XLA Compiler's Tensor IR

XLA-HLO (High Level Operations) IR [22] is a tensor IR used in the XLA compiler [20], the default ML compiler for TensorFlow [1] and JAX [35]. Here, we describe the key concepts of XLA-HLO IR relevant to this paper. These concepts are also applicable to other tensor IRs such as Torch.Attn [4].

Tensors are n -dimensional objects (generalization of 0-D scalars, 1-D vectors, 2-D matrices) that are usually represented as multi-dimensional arrays. A tensor of *tensor-type* $\mathbb{E}[S]$ has a shape S , a tuple of integers denoting the dimension sizes, with elements of scalar basetype $\mathbb{E}$ such as `uint8` (`u8`), `int32` (`i32`), `fp32` (`f32`), `bfloat16` (`bf16`). A *tensor value* has a fixed tensor-type with fixed data for each of its elements. A *tensor variable* has a fixed tensor-type but can hold variable data at runtime.

Tensor Operators. The XLA compiler manipulates tensors as first-class objects using tensor operators, which are usually tensor-type agnostic and parameterized by various parameters, collectively called *operator attributes*. For example, the operator `slice( $x, s, e, p$ )` extracts a sub-tensor from an input tensor x of any tensor-type, from indices s to e with a stride of p . Here, s, e, p and the tensor-type of x are operator attributes of `slice`, with validity conditions such as $0 \leq s < e \leq \text{size}(x)$ and $p > 0$. We call tensor operators parameterized by these attributes *abstract tensor operators*. Table 1 briefly describes the XLA-HLO tensor operators used in this paper. Appendix D visualizes these, with their operational and denotational semantics documented in [22, 23] and [6], respectively.

During the compilation of a given model, all operator attributes are instantiated. We call these instantiated tensor operators *concrete tensor operators*, which take tensor variables as input and produce a tensor variable as output. Consider an abstract tensor operator f instantiated with attributes A , producing a concrete tensor operator f_A . It takes n tensor variables $X = (x_i)_{i=1}^n$ as input and computes a tensor variable y as output, i.e., $y = f_A(x_1, \dots, x_n)$, or in short, $f_A(X)$.

Tensor Operator	Operator Attributes	Description
$y = \text{slice}_{[s:e]}(x)$	$\text{type}(x), s, e$	Read sub-tensor $x[s:e]$
$y = \text{upslice}_{[s:e]}(x_1, x_2)$	$\text{type}(x_1), s, e$	Update sub-tensor $x_1[s:e] = x_2$
$y = \text{concat}_{dim}(x_1, x_2)$	$\text{type}(x_1), \text{type}(x_2), dim$	Concatenate x_1, x_2 across dimension dim
$y = \text{reshape}(x)$	$\text{type}(x), \text{type}(y)$	Reshape from $\text{type}(x)$ to $\text{type}(y)$
$y = \text{bitcvt}(x)$	$\text{type}(x), \text{type}(y)$	Bitcast conversion between basetypes
$y = \text{broadcast}_{dim}(x)$	$\text{type}(x), dims$	Broadcast over dimension dim
$y = \text{reduce}_{dim}(x)$	$\text{type}(x), dims$	Reduce-sum over dimension dim
$y = \text{transpose}_{[dims]}(x)$	$\text{type}(x), dims$	Permute the dimension order to $dims$
$y = \text{dot}(x_1, x_2)$	$\text{type}(x_1), \text{type}(x_2)$	Matrix multiplication of x_1, x_2
$y = \text{exp}(x)$	$\text{type}(x)$	Element-wise exponential
$y = \text{div}(x_1, x_2)$	$\text{type}(x_1)$	Element-wise divide
$y = \text{copy}(x)$	$\text{type}(x)$	Identical copy

Table 1. Brief descriptions of tensor operators based on XLA-HLO IR [22] discussed in the paper. $\text{type}(x)$ refers to the tensor-type of tensor variable x . dim represents the dimension number (starting from 1). We use NumPy-like slice notation $x[s_1:e_1, s_2:e_2, \dots]$ and ignore type-based attributes in visualizations.

Tensor Computation Graphs. XLA’s internal IR, called XLA-HLO IR, represents computations as a *tensor computation graph* — a directed acyclic graph with tensor operators as nodes and operands as edges. The edges establish the data dependencies between different tensor operators. Unless specified otherwise, a tensor computation graph consists of concrete tensor operators.

IR Properties. XLA-HLO IR is a *data flow graph* IR, i.e., it does not have explicit imperative control flow. Control flow is represented using higher-order functional operators such as `while`. XLA-HLO IR only supports *static tensor shapes*, i.e., the shapes of all tensor variables are known at compile time. Shape dynamism is handled by StableHLO passes [21] prior to XLA-HLO IR.

2.2 Tensor Rewrites in XLA Compiler

The XLA compiler performs semantic-preserving transformations on tensor computation graphs using a set of algebraic rewrite rules, termed *tensor rewrites*. A tensor rewrite $P_l \rightarrow P_r$ substitutes subgraph P_l with subgraph P_r , under certain preconditions on the operator attributes.

XLA uses 175+ tensor rewrites, with 115 of these verified to full generality by TensorRight [6]. In our work, we adopt this rewrite set, denoted as $\mathcal{R}$, as foundational axioms of the XLA-HLO IR. One such rewrite is $\text{slice}_{[s_1:e_1]}(\text{slice}_{[s_2:e_2]}(x)) \rightarrow \text{slice}_{[s_1+s_2:e_1+s_2]}(x)$. Similar to prior works [52, 97, 101, 105], we define semantic equivalence modulo these tensor rewrites $\mathcal{R}$.

2.3 Tensor Accelerators

Tensor accelerators (a.k.a. AI accelerators) are a class of hardware accelerators optimized for tensor computations using different microarchitectural innovations such as systolic array-based executions. Some have programmable dataflows [36, 58, 78], while others are fixed [16, 19, 29, 56, 103]. They have diverse functional units such as matrix transpositions [55], matrix reshaping units, etc., with novel innovations such as intelligent on-chip routing [93]. TAIDL [51] standardizes how ISA semantics are described for a variety of diverse tensor accelerator designs.

2.4 Term Rewriting using Equality Saturation

Term rewriting [7] is classically destructive, making optimizations sensitive to the order in which rewrites are applied — the well-known “phase-ordering” problem [96]. Equality saturation [90] overcomes it using *e-graphs*, which apply rewrites simultaneously and compactly represent equivalence relations over terms as equivalence classes (*e-classes*) of equivalent nodes (*e-nodes*).

The equality saturation workflow consists of three stages: initialization, exploration, and extraction. First, the initial e-graph is created from the input term. Next, the exploration stage applies rewrites until saturation is reached (i.e., applying rewrites does not add any new information) or a e-node limit is reached. Finally, the optimal term is extracted from the e-graph.

3 Problem Statement

In this section, we formulate the problem of a compiler backend generator that generates sound and complete accelerator-specific backends from just the accelerator ISA description. First (§3.1), we describe the input to these backends by illustrating the code generation pipeline of the XLA compiler [20]. Next (§3.2), we provide a high-level summary of TAIDL [51], a popular ISA description language for tensor accelerators that serves as the input to our generator. Finally (§3.3), we define soundness and completeness, and state the problem of a compiler backend generator.

3.1 Tiled Kernels: Input to Accelerator-specific Compiler Backends

XLA's code generation pipeline starts after the middle-end passes of operator fusion and tiling [76], which partition the optimized tensor computation graph in XLA-HLO IR (§2.1) into several sub-graphs and determine the tile factors for each of these sub-graphs. This pipeline is summarized in Fig. 2, and snippets from different stages of this pipeline are shown in Fig. 3.

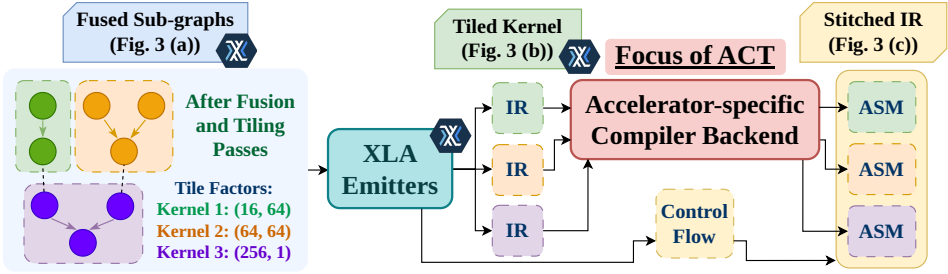


Fig. 2. Summary of code generation pipeline of XLA compiler [20] after operator fusion and tiling passes. Snippets from different stages of this pipeline are shown in Fig. 3. In this paper, we focus on accelerator-specific compiler backends (shaded red) and borrow XLA's existing implementation for the rest of the pipeline.

```

1 ...
2 %fused_subgraph.1 {
3   %Q = bf16[2048,1024]{:T(16,64)}
4     parameter(0)
5 }
6 ...
7 ENTRY %main {
8   ...
9   %qkv.out = bf16[2048,1024]{:T(16,64)}
10    fusion(...), calls=%fused_subgraph.1
11 }

```

(a) XLA-HLO IR after operator fusion and tiling passes. The tile factors are annotated by `{:T(...)}` in purple.

```

1 ENTRY %tiled_kernel.1 {
2   %Q = bf16[16,64] parameter(0)
3   ...
4 }

```

(b) Tiled kernel emitted for %fused_subgraph.1

```

1 ...
2 scf.for %i = 0 to 128 {
3   ... {
4     %x0 = llvm.inline_asm ...
5     ...
6   }
7 } ...

```

(c) Stitched IR with accelerator-specific assembly code inlined in the control flow for tiling.

Fig. 3. Snippet from different stages of XLA's code generation pipeline. The accelerator-specific compiler backend consumes a tiled kernel (b) and emits assembly code (lines 4–5 in (c)) inlined in the stitched IR.

After XLA middle-end passes, the XLA-HLO IR includes multiple sub-graphs (e.g., Fig. 3 (a) lines 2–5) with tile factors annotated in purple (e.g., `{:T(16, 64)}`). First, XLA's `Emit` modules [24] emit (i) control flow for tiling (e.g., `scf.for` loop in Fig. 3 (c) line 2) and (ii) single-tile versions of all sub-graphs (e.g., Fig. 3 (b)). We name these single-tile versions of sub-graphs as *tiled kernels*.

A **tiled kernel** is syntactically similar to the original sub-graph, except that the tensor shapes are based on the tile factors (e.g., $\text{bf16}[16, 64]$ in Fig. 3 (b) vs. $\text{bf16}[2048, 1024]$ in Fig. 3 (a)). It is a tensor computation graph in XLA-HLO IR (§2.1), which includes static tensor-types, and computes a single tile of the original sub-graph. In this paper, we will visualize tiled kernels as directed acyclic graphs with nodes as tensor operators (from Table 1), for example, in Fig. 4.

The **accelerator-specific compiler backend** compiles these tiled kernels individually into accelerator-specific assembly code. Finally, the generated assembly code for different kernels is inlined and stitched together with control flow for tiling into a single *stitched IR* (e.g., Fig. 3 (c)). We defer the specifics of execution of the stitched IR for various accelerators to evaluations (§8).

The input to an accelerator-specific compiler backend is a tiled kernel in XLA-HLO IR and the output is a sequential accelerator-specific assembly code.

Running Example. Fig. 4 visualizes a tiled kernel G_{QKV} for QKV computation, which is prevalent in many transformer topologies [27, 95]. For illustration purposes, we assume a single-batch single-head QKV computation over $\text{bf16}[64, 64]$ tiles. We use this as a running example for the rest of the paper.

Challenges

Key challenges in designing accelerator-specific compiler backends are the diversity of tensor accelerator designs and the complexity of the ISA descriptions. Different accelerators have different memory hierarchies and instruction granularities, which require different compilation strategies. Furthermore, ISA descriptions of tensor accelerators are often parameterized by input shapes [36, 56, 104] and include complex data layout transformations [46, 82, 93], which require accelerator-specific legalization passes to map tiled kernels to the accelerator ISA.

3.2 TAIDL: Tensor Accelerator ISA Description

We choose TAIDL [51], Tensor Accelerator ISA Definition Language, to describe accelerator ISAs since it supports several existing tensor accelerators [36, 46, 56, 84, 93, 104] and can precisely model the semantics of new tensor accelerator ISAs. Here, we provide a high-level summary of TAIDL [51] necessary for later sections. Detailed definitions and explanations are deferred to Appendix A.

TAIDL standardizes the way ISAs and their semantics are developed for tensor accelerators. It builds on top of the existing formal models [6, 52, 61, 98] of tensors and tensor operators. The key principle behind TAIDL is that tensor accelerators support coarse-grained instructions that work on multi-dimensional data (commonly represented as tensors). Therefore, TAIDL descriptions use compositions of tensor operators in XLA-HLO IR to model the instruction semantics.

An *ISA description* comprises (1) the software-managed storage units (collectively termed *data model*) like scratchpads, and (2) the semantics of instructions that perform computations like data movement and compute instructions on these storage units. This is analogous to the x86 ISA description comprising a data model (registers, memory) and an instruction set (add, mov, etc.).

Running Example. Consider a hypothetical accelerator H_{QKV} (Fig. 5 (a)) with two on-chip storage units—a 16 KB scratchpad and an 8 KB intermediate buffer—and access to global memory via a DMA engine. Its two compute units are a general matrix multiply (gemm) unit and a softmax activation unit. It supports five data movement and two compute instructions listed in Fig. 5 (b). We use this hypothetical accelerator H_{QKV} as our running example for the rest of the paper.

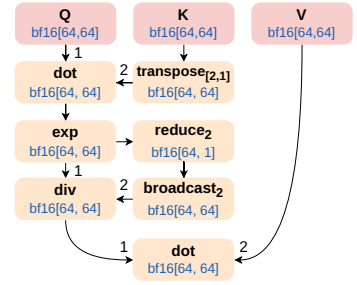


Fig. 4. Self-attention kernels in Transformer models like BERT [27] use QKV computation $\text{softmax}(Q \cdot K^T) \cdot V$ over 4-dimensional tensors. For illustration purposes, we assume a single-batch single-head QKV computation G_{QKV} over $\text{bf16}[64, 64]$ tiles.

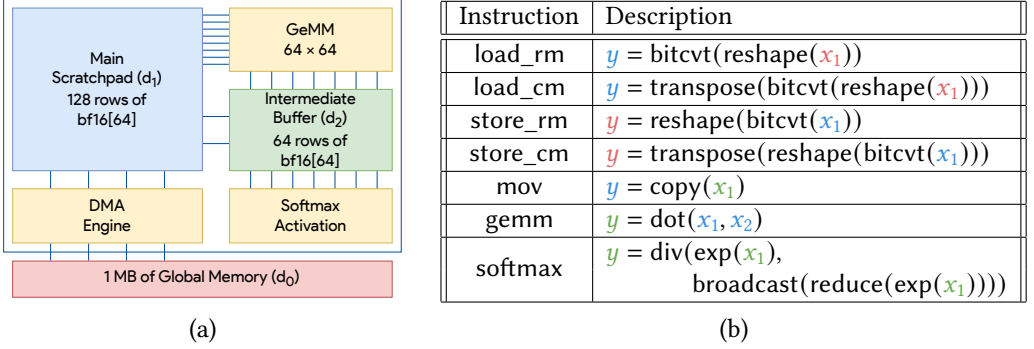


Fig. 5. Running example. (a) High-level accelerator design for a hypothetical accelerator H_{QKV} .

(b) Brief description of its instruction set. Tensor variables are colored based on their storage unit (d_0 , d_1 , d_2).

(i) TAIDL constructs: Data Model

The software-managed storage units, like scratchpads on an accelerator, are abstractly represented as *tensor buffers*. TAIDL represents a tensor buffer d as a tensor (of type $\mathbb{E}[S_0, S_1]$) with a subset of dimensions representing *access dimensions* (S_0) and the remaining dimensions representing the granularity of data access (S_1). TAIDL assumes that an accelerator has a byte-addressable 1-dimensional global memory (HBM) that is accessible by the host processor and is denoted d_0 .

Running Example. Tensor buffer representations for software-managed storage units of H_{QKV} are represented in lines 2–4 of Fig. 6. The main scratchpad (d_1) consists of 128 rows ($S_0=[128]$) of 64-length bf16 vectors ($S_1=[64]$). Instructions read and/or modify the scratchpad at a row granularity.

(ii) TAIDL constructs: Instruction Set

The instruction set is a set of *abstract instructions* that modify the tensor buffers and are parameterized by instruction attributes. A *concrete instruction* is an instantiation of an abstract instruction with specific values for the attributes. For example, x86 instructions, such as mov and add, modify registers or memory and are parameterized by register names and memory addresses.

An *assembly code* for a tensor accelerator consists of a stream of concrete instructions and a constant tensor. This is analogous to x86 assembly code with code and data sections, respectively.

TAIDL models the semantics of abstract instructions of tensor accelerators as a composition of tensor operators in XLA-HLO IR [22]. Fig. 5 (b) provides a brief description of the instruction set of H_{QKV} using tensor operators from Table 1.

In this work, we extend TAIDL descriptions by classifying the instruction attributes into two sets: (i) *computational* (α) attributes, like input matrix shapes (challenge 2), that determine the core tensor computation; and rest of the attributes as (ii) *addressing* (β) attributes, like input addresses. Therefore, an abstract instruction θ which reads n_θ tensors located in buffers $d_\theta^{(i)}|_{i=1}^{n_\theta}$ and modifies a tensor located in buffer $d_\theta^{(0)}$, is represented using an *abstract tensor computation* g_θ concretized by the set of computational attributes α , $(n_\theta + 1)$ *data slice address maps* $h_\theta^{(i)}|_{i=0}^{n_\theta}$ over the set of addressing attributes β , and a *validity constraint* e_θ over the set of attributes α and β .

Running Example. The abstract instruction load_rm in H_{QKV} is represented in lines 7–16 of Fig. 6. Lines 8–9 describe the tensor buffers $d_{\text{load_rm}}$ and data slice address maps $h_{\text{load_rm}}$ of the input and output tensors. It is a data movement instruction that loads n rows of data ($n \times 128$ bytes) from the HBM (d_0) starting at address addr_{in} , to the main scratchpad (d_1) starting at row addr_{out} . The concrete instruction load_rm($n=4$, $\text{addr}_{\text{in}}=0$, $\text{addr}_{\text{out}}=2$) loads 512 bytes from $d_0[0:512]$ to $d_1[2:6]$. Line 10 describes the validity constraint $e_{\text{load_rm}} = (n \leq 128)$. Finally, lines 12–16 represent the abstract tensor computation $g_{\text{load_rm}}$ using tensor operators (Table 1) and attribute $n \in \alpha_{\text{load_rm}}$.

```

1 # Data Model
2 qkv.set_hbm("d0", size="1MB") # u8[2^20]
3 qkv.add_data_model("d1", S0=[128], S1=[64], E="bf16") # bf16[128,64]
4 qkv.add_data_model("d2", S0=[64], S1=[64], E="bf16") # bf16[64,64]
5
6 # Instruction Semantics
7 instr = qkv.add_instr(name="load_rm", alpha=["n"], beta=["addr_in", "addr_out"])
8 instr.set_inputs(["d0", start=["addr_in", len=["n * 128"]]]) # u8[n*128]
9 instr.set_output(["d1", start=["addr_out", len=["n"]]]) # bf16[n,64]
10 instr.set_constraints(["n <= 128"])
11 # y = bitcvt(reshape(x1)) represented using XLA-HLO syntax
12 instr.set_abstract_computation("""ENTRY load_rm {
13   x1 = u8[~n * 128] parameter(0);
14   a = u8[~n, 64, 2] reshape(x1);
15   ROOT y = bf16[~n, 64] bitcast_convert(a);
16 }""")

```

Fig. 6. Snippet of ISA description of H_{QKV} in TAIDL with instruction attributes annotated as α and β .

3.3 Problem Formulation: Compiler Backend Generator

Finally, we formulate the problem statement of a compiler backend generator that generates sound and complete compiler backends from just the accelerator ISA descriptions in TAIDL.

A compiler backend for a tensor accelerator consumes a tiled kernel (§3.1) as the input and either (i) successfully compiles to emit an *accelerator-specific assembly code* (§3.2), or (ii) does not compile, i.e., either a compilation failure or does not terminate. We define the semantic equivalence between the tiled kernel and the compiled assembly code using bisimulation transformations, modulo the set of known tensor rewrites of XLA-HLO IR (as discussed in §2.2).

Soundness: A compiler backend is said to be *sound* if, for all successfully compiled tiled kernels, the emitted assembly code is semantically equivalent to the input tiled kernel.

Completeness: A compiler backend is said to be *complete* if it successfully compiles for all input tiled kernels that have a semantically equivalent assembly code.

The detailed definitions and formalisms of these concepts are deferred to Appendix A.

We design a compiler backend generator such that for any given ISA description in TAIDL, it generates an accelerator-specific compiler backend that is *sound* and *complete*.

4 Overview of Accelerator Compiler Toolkit (ACT)

We present a compiler backend generator — ACT, Accelerator Compiler Toolkit — that generates a sound and complete compiler backend from a given tensor accelerator ISA description in TAIDL. A key contribution of our work is a novel compilation algorithm parameterized by TAIDL constructs ($d_i, \theta, \alpha, \beta, g_\theta, h_\theta, e_\theta$, discussed in §3.2). In this section, we provide a high-level overview of the ISA-parameterized algorithm (in §4.1–§4.2) and outline the compiler backend generation process based on this algorithm (in §4.3). We elaborate the details in §5 and §6.

4.1 Overview of the ISA-parameterized Compilation Algorithm

ACT's parameterized compilation algorithm is designed as a modular pipeline, with each module focusing on a specific aspect of the compilation. The algorithms for each module are *parameterized* by the TAIDL constructs, which are instantiated given an ISA description.

Fig. 7 visualizes the high-level structure of ACT's ISA-parameterized compilation algorithm. The algorithm consumes a tiled kernel and emits an assembly code. The assembly code consists of concrete instructions $\theta_i(\alpha_i, \beta_i)_{i=1}^N$, i.e., abstract instructions θ_i with initialized computational (α_i) and addressing (β_i) attributes. The algorithm consists of two phases: (1) initializing α_i s, (i.e., *instruction selection*), and (2) initializing β_i s, (i.e., *memory allocation*).

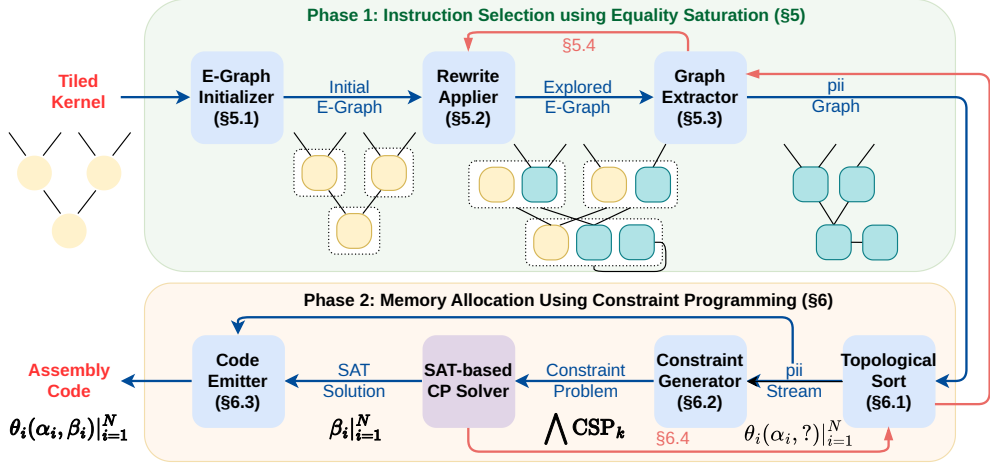


Fig. 7. Overview of ACT's parameterized compilation algorithm. Blue modules are parameterized by the TAIDL constructs (§3.2). The purple module is an external tool. The red edges represent the fallback mechanisms designed for completeness (proof in Appendix C) and the enumeration pipeline (discussed later in §7).

Phase 1: Tensor Instruction Selection

The first phase consumes the tiled kernel and emits a directed acyclic graph of *partially* instantiated instructions (pii), called a pii graph. A *partially* instantiated instruction is an abstract instruction θ with α initialized and β uninitialized, denoted as $\theta_{\alpha,?}$. This phase initializes α , i.e., explores semantically equivalent instruction selection opportunities for the tiled kernel. This is analogous to the instruction selection pass in CPU backends. It is formulated as an equality saturation problem and utilizes an existing equality saturation engine [100] to perform graph-level semantic-preserving substitutions. The three modules required for equality saturation—e-graph initializer, rewrite applier, and graph extractor—are parameterized by the TAIDL constructs.

Phase 2: Tensor Memory Allocation

The second phase consumes the pii graph and emits the final assembly code. This phase initializes β , i.e., assigns addresses to the unmapped slices. This is analogous to the register allocation pass in CPU backends. First, the pii graph is topologically sorted to respect instruction dependencies. Then, these slices are mapped to the address space of the accelerator's tensor buffers. This mapping must (1) maintain instruction dependencies, (2) assign non-overlapping address ranges to slices with overlapping live ranges, and (3) yield valid instruction attributes. These are formulated as a constraint satisfaction problem over integers and passed to an existing SAT-based CP solver [39]. Finally, the assembly code is emitted based on the solver's output. The three modules—topological sort, constraint generator, and code emitter—are also parameterized by the TAIDL constructs.

Inter-phase fallback mechanism

Unlike commodity hardware, accelerator designs have constraints not captured during instruction selection, such as non-trivial addressing constraints and lack of spill-reload instructions for some tensor buffers (e.g., AWS Trainium [82]). Therefore, the second phase may fail to find a valid solution for a pii graph. The compilation algorithm then falls back to the first phase, which extracts a different pii graph or performs another iteration of rewrites. This inter-phase mechanism, along with the intra-phase fallbacks, is essential for completeness.

Soundness. ACT's parameterized compilation algorithm is sound by construction (proof in Appendix C). Note that the soundness is defined modulo the foundational axioms of XLA-HLO IR (as discussed in §2.2). This axiomatic approach is similar to prior works [52, 97, 101, 105].

Completeness. ACT's parameterized compilation algorithm is also complete. Intuitively, this is shown by proving that for any input tiled kernel with an equivalent assembly code, there exists a sequence of rewrites and a valid ordering of the pii graph that leads to a successful compilation, and the fallback mechanisms ensure that all possible sequences of rewrites and orderings are explored and enumerable. The detailed proof of completeness is provided in Appendix C.

Termination. For kernels with no equivalent assembly code, the compilation algorithm may not terminate, since completeness and guaranteed termination cannot both hold (proven by reduction from the halting problem in Appendix A). In our context of ML compiler backends, completeness is crucial to ensure that the middle-end does not fall back to host (CPU) code generation, avoiding the associated performance degradation (§8.6). In practice, we bound non-termination with a timeout.

4.2 Novelty of the ISA-parameterized Compilation Algorithm

Code generation for tensor accelerators brings unique challenges (§1.2) that are not articulated nor solved by existing techniques. Off-the-shelf rewrite synthesizers, e-graph extractors, and memory/register allocators do not handle parameterized tensor instructions, multi-dimensional scratchpad allocation, or complex layout-transforming ISA semantics. Table 2 summarizes the novel features of ACT's parameterized compilation algorithm that address these challenges.

Limitations of existing techniques	Novelty in ACT
Vector-ISA instruction selectors [18, 91, 94] assume fixed-size instructions, unlike shape-parameterized tensor instructions.	<i>Reduction to a two-phase algorithm (§4.1)</i> via a new pii intermediary, splitting attributes into computational (α , Phase 1) and addressing (β , Phase 2).
Typical rewrite synthesizers [72, 73] generate rewrites <i>without preconditions</i> , and cannot model parameterized instructions with computational attributes.	<i>Preconditioned rewrite synthesis (§5.2)</i> that generates one IR-to-ISA rewrite per instruction, with <i>preconditions</i> on computational attributes for parameterized instructions.
Existing accelerator backends and libraries do not ensure <i>completeness</i> and may fail to compile kernels even when a valid assembly code exists, forcing host (CPU) fallback.	<i>No-ops modeled as identity pii nodes (§5.2)</i> for inner-loop tiling and padding, enabling provable discovery of <i>all</i> equivalent instruction sequences and ensuring completeness.
One-shot extractors [11, 99–101] extract a <i>single</i> graph, freely choosing at most one e-node per e-class, unable to handle fallback scenarios or e-node selection constraints.	<i>Enumerative e-graph extraction (§5.3)</i> of <i>all</i> equivalent pii graphs for fallback scenarios, restricting e-nodes by previously selected ones and allowing <i>multiple</i> e-nodes per e-class.
Register allocators (graph coloring [3], puzzles [80]) and scratchpad allocators [65, 67] model only interference over 1-D sites, without accelerator addressing constraints.	<i>CSP-based memory allocation (§6.2)</i> combining accelerator addressing constraints with live-range interference over multi-dimensional tensor buffers and related pruning strategies (§6.4).

Table 2. Novel features of ACT's parameterized compilation algorithm.

4.3 Overview of the Compiler Backend Generation Process

ACT's ISA-parameterized compilation algorithm is implemented as a generic compiler backend with pure virtual functions and symbolic templates based on TAIDL constructs. A user specifies an accelerator ISA in TAIDL (as shown in Fig. 6). ACT processes this ISA description to instantiate a specialized compiler backend by filling in the symbolic templates with ISA-specific details. This generation process is visualized in Fig. 8. This is similar to the design principle of classic compiler component generators such as Flex [30] (for lexical analysis) and Yacc [53] / Bison [79] (for parsing).

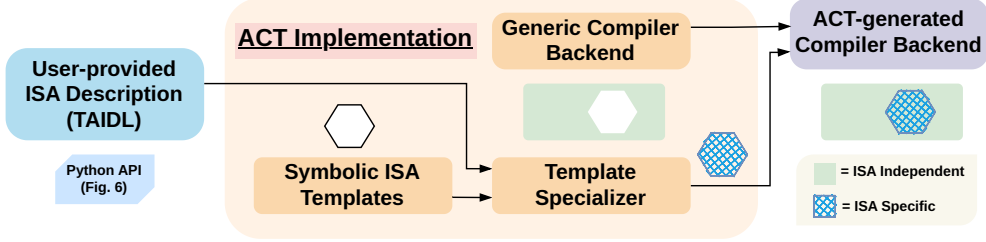


Fig. 8. Overview of ACT's compiler backend generation process. The orange region shows the implementation of the parameterized algorithm as a generic compiler backend (with missing holes) and symbolic ISA templates. At generation time, these symbolic templates are specialized using the user-provided ISA description.

The generation process runs once per accelerator and produces a standalone accelerator-specific compiler backend executable. At compile-time, this generated backend executable is called once per tiled kernel. Detailed correspondence with Flex and Bison is tabulated in Appendix B.

More concretely, ACT converts the TAIDL description into rewrite rules and preconditions for equality saturation (Phase 1). It also generates ISA-specific implementations of abstract methods such as `get_h0()` and `get_e()`, corresponding to the TAIDL constructs $h_{\theta}^{(0)}$ and e_{θ} . The constraint problem (Phase 2) is formulated abstractly on the TAIDL constructs, without any hardcoded ISA assumptions. This separation ensures that compiler backends are derived systematically from just the ISA description, without embedding ISA-specific logic in the compiler.

Engineering Effort per Accelerator. Existing TAIDL descriptions take around 10-12 lines per simple instruction (like Intel AMX [46] load/store, Intel AVX-512 [48] vector compute, Gemini [36]) and 12-15 lines per complex one (like Intel AMX [46] matrix compute, FEATHER [93] layout reconfigurations), so generating a new accelerator-specific backend takes little effort. These backends are also compatible with XLA, a production ML compiler, amplifying this reduction.

5 Phase 1: Tensor Instruction Selection Using Equality Saturation

We formulate the tensor instruction selection problem as a search problem over the space of equivalent tensor computation graphs. We use e-graphs [90] to compactly represent this space of equivalent tensor computation graphs and enable efficient exploration for instruction selection.

In this section, we focus on the novel features of Phase 1 highlighted in Table 2 — preconditioned IR-to-ISA rewrites and identity instructions (§5.2), as well as an enumerative e-graph extraction algorithm (§5.3-§5.4). We use the step-wise lowering of the tiled kernel G_{QKV} (Fig. 4) for the accelerator H_{QKV} (Fig. 5) as an illustrative example.

5.1 Module 1: E-Graph_INITIALIZER

In our work, an e-graph is a directed graph where each e-node is a concrete tensor operator or a *partially instantiated instruction* (pii $\theta_{\alpha,?}$, defined in §4.1). An e-class is a set of e-nodes representing semantically equivalent computations rooted at the e-nodes. By design, an e-node is optionally associated with a tensor buffer. Specifically, a pii e-node $\theta_{\alpha,?}$ is associated with the output tensor buffer of its instruction ($d_{\theta}^{(0)}$ notation from §3.2). The e-graph is initialized with the input tiled kernel G with a 1-D byte representation of the input and output tensors. The 1-D byte representation for the bf16 tensors requires

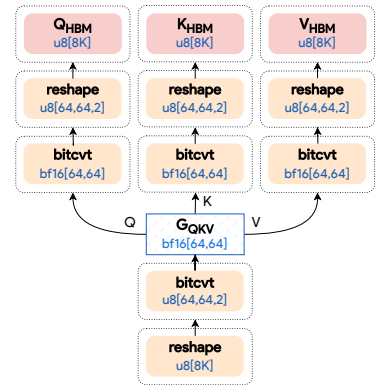


Fig. 9. The initial e-graph for G_{QKV} [after Module 1]. G_{QKV} (Fig. 4) is not expanded here for compactness. Note that the edges are now flipped with the output tensor as the root e-node.

bitcasting a bf16 to two u8s followed by a reshape operation. The leaf e-nodes are assigned to the tensor buffer d_0 , i.e., allocated on HBM, while the root is the output tensor, where extraction begins (§5.3). Fig. 9 shows the initial e-graph for G_{QKV} (Fig. 4), without expanding the sub-graph of G_{QKV} .

5.2 Module 2: Rewrite Applier

The second module specifies the rewrites used in the exploration stage of equality saturation. These include three types of rewrites: (1) existing set of IR-to-IR tensor rewrites from the XLA compiler, (2) new IR-to-ISA rewrites generated from the accelerator ISA description, and (3) rewrites based on *no-ops*, termed as *identity instructions*, i.e., instructions that do not modify the memory state.

IR-to-IR rewrites from XLA compiler. ACT-generated backends use the curated set of existing IR-to-IR rewrites from the XLA compiler (discussed in §2.2) to explore the space of equivalent tensor computation graphs. Note that these 175+ rewrites are maintained by the XLA compiler developers and are not specific to any accelerator. Furthermore, e-graphs can simultaneously explore different sequences of rewrites, avoiding the need to implement accelerator-specific IR legalization passes.

IR-to-ISA rewrites generated from ISA description.

At generation time, ACT generates new rewrites with preconditions using a pre-order traversal from the ROOT of the abstract tensor computation g_θ (line 15 of Fig. 6). These IR-to-ISA rewrites create new pii nodes in the e-graph. Combined with IR-to-IR rewrites, they replace manually-written accelerator-specific pattern-matching rules.

For instruction θ , ACT generates the rewrite $g_\theta \rightarrow \theta$ that substitutes a concrete tensor computation $g_\theta(\alpha)$ with pii node $\theta_{\alpha,?}$ under the precondition $\exists \beta, e_\theta(\alpha, \beta) = \text{True}$. For example, rewrite $\text{bitcvt}(\text{reshape}(x)) \rightarrow \text{load_rm}(x)$ is generated from Fig. 6. Fig. 10 shows a snippet of the explored e-graph of Fig. 9 with new pii nodes colored as blue (d_1) and green (d_2). Fig. 11 shows the precondition generated for load_rm , which determines the computational attribute n at compile-time and checks the validity constraint $e_{\text{load_rm}}$. Most existing rewrite synthesizers [72, 73] generate rewrites without such preconditions, requiring multiple rewrites to cover a parameterized instruction, whereas ACT generates only *one* IR-to-ISA rewrite per instruction.

Identity instructions (no-ops). We model *no-ops* as identity instructions slice^H and concat^H , i.e., instructions that do not modify the tensor buffers. These are pii nodes analogous to tensor operators slice and concat but with the constraint that the inputs and the output are on the same buffer. slice^H applies only under a precondition (the slice dimensionality must match a tensor buffer, §3.2), and concat^H constrains its two operands to be allocated contiguously in the same tensor buffer so the next instruction reads them as one operand. These identity instructions ensure exploration of all equivalent pii graphs (proof in Appendix C).

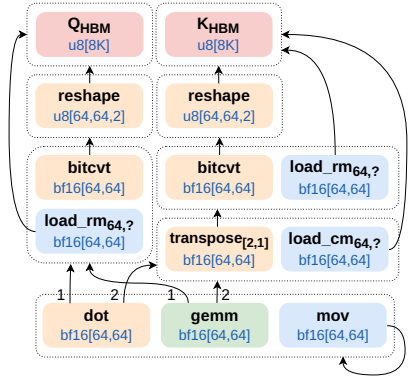


Fig. 10. Snippet of explored e-graph [after Module 2] after applying IR-to-ISA rewrites on the initial e-graph (Fig. 9). Dotted boxes are e-classes, solid boxes are e-nodes, and edge labels 1/2 are operand indices. The new pii nodes result from the ACT-generated IR-to-ISA rewrites for load_rm , load_cm , gemm , and mov .

```
1 pub fn precond_load_rm(..) -> bool {
2   // y: bf16[alpha_n, 64]
3   if y.shape.len() != 2
4     || y.shape[1] != 64
5     || y.dtype != Dtype::BF16 {
6     return false;
7   }
8   let alpha_n = y.shape[0];
9   if alpha_n > 128 {
10    return false;
11  }
12  ... // Repeat for a, x1
13 }
```

Fig. 11. Snippet of the rewrite precondition (Rust) generated from the semantics of load_rm in Fig. 6. Line 8 determines the computational attribute n .

5.3 Module 3: Graph Extractor

The third module performs the extraction stage of the equality saturation workflow. It extracts a directed acyclic graph of pii nodes (i.e., a pii graph) from the explored e-graph (after Module 2).

First, a bottom-up traversal (from leaf e-classes) is performed to detect e-classes representing compile-time constant tensors. The e-classes with a leaf e-node, such as tensor operators *eye*, *iota*, are constants. An e-class is a constant if any one of its e-nodes has all its operands as constants. Such e-classes become base cases of the extraction below (line 6 of Fig. 12).

Then, a top-down traversal (from root e-class) is performed to extract the pii graphs. It is guided by the input and output tensor buffers ($d_\theta^{(i)}|_{i=0}^{n_\theta}$ notation from §3.2) associated with each pii e-node. Fig. 12 presents the pseudocode — `extract(EC, d, N)` returns *all* pii graphs computing e-class *EC* into tensor buffer *d* within *N* nodes, invoked at the root with $d = d_0$ (line 24). Only the e-nodes whose output tensor buffer $d_\theta^{(0)}$ matches the requested *d* are admissible (line 12), and each of them in turn requests $d_\theta^{(i)}$ for its *i*-th operand (lines 15–16). For example, `load_cm64` is admissible for the buffer that `gemm` requests for its second operand in Fig. 10, since $d_{\text{load_cm}}^{(0)} = d_1 = d_{\text{gemm}}^{(2)}$. The graphs of an admissible e-node are the Cartesian product of its operands' graph sets (line 19), and those of an e-class are the union over its admissible e-nodes. The traversal terminates upon reaching an input variable leaf e-node or a constant e-class, or hitting a pii node limit N_{max} (line 3).

Classical one-shot extractors [11, 99–101] freely select at most one e-node per e-class without any e-node constraints. Our extraction differs on both counts — selection is constrained by the buffer requested by the parent e-node, and an e-class expanded under several requested buffers contributes *multiple* of its e-nodes to one pii graph. Enumerating *all* of them is necessary rather than merely thorough, since Phase 2 can fail for a given pii graph (§4.1) and its cost is known only after code generation (§7). Without our novel extraction algorithm, most kernels (> 95%) fail to compile for popular tensor accelerator ISAs such as AWS NKI [84] and Gemini [36].

Fig. 13 (a) shows the first extracted pii graph from the explored e-graph of G_{QKV} (Fig. 10).

5.4 Intra-phase Fallback Mechanism: Enumerating All Equivalent pii Graphs

The exploration and extraction stages of equality saturation may not terminate for all tiled kernels. The exploration stage iteratively applies the rewrites but may not reach saturation for all tiled kernels. The explored e-graph may have loops, and infinite equivalent pii graphs may exist.

To ensure that all equivalent pii graphs are enumerable without getting stuck within an infinite loop of either of the two stages, we use a bounded approach of exploration and extraction. We iteratively increase the pii node limit N_{max} for the extraction stage with the iteration count *k* of the exploration stage, i.e., $N_{\text{max}} = \Gamma(k)$ where Γ is a strictly monotonically increasing function of *k* (e.g., $\Gamma(k) = 2^k$). For every value of *k*, we extract a finite number of pii graphs, and thus, this set is enumerable. This is analogous to enumerating $(a, b) \in \mathbb{N}^2$ using the Cantor pairing function [13].

```

1 def extract(ec, buf, limit):
2     # node budget N_max exhausted
3     if limit == 0: return {}
4
5     # base cases: input or constant
6     if ec.is_input or ec.is_const:
7         return {leaf(ec)}
8
9     graphs = {}
10    for e in ec.enodes:
11        # buffer-guided selection
12        if e.out_buf != buf: continue
13        children = []
14        for i, c in enumerate(e.args):
15            cbuf = e.in_buf[i]
16            sub = extract(c, cbuf, limit-1)
17            children.append(sub)
18        # enumerate all combinations
19        for combo in product(*children):
20            graphs.add(pii(e, combo))
21    return graphs
22
23 # Module 3 entry point
24 extract(egraph.root, HBM, N_max)

```

Fig. 12. Top-down traversal to extract all pii graphs following buffer constraints within the N_{max} node limit.

6 Phase 2: Tensor Memory Allocation using Integer Constraint Programming

The second phase of the compilation pipeline is responsible for compiling the generated pii graph down to accelerator-specific assembly code. This phase involves scheduling the pii graph and allocating the intermediate tensors on the tensor buffers. We formulate the memory allocation as an integer constraint satisfaction problem (CSP) and solve it using a SAT-based CP solver.

In this section, we focus on the novel features of Phase 2 highlighted in Table 2 — the parameterized multi-dimensional CSP formulation (§6.2) and interference-based pruning (§6.4).

6.1 Module 4: Topological Sort

Instruction scheduling needs to adhere to *def-use* edges of the pii graph and perform a topological ordering of the nodes. Our priority is to minimize the live ranges of the intermediate tensors and reduce the possibility of memory allocation failing. Sethi-Ullman algorithm [87] generates optimal code for arithmetic binary trees with minimal registers. We extend the Sethi-Ullman numbering to multiple tensor buffers storing varying sizes of tensor variables, and order the operands of each pii node by decreasing tensor buffer requirement. We defer the extended numbering to Appendix B. Fig. 13 (a) shows the topological ordering of the extracted pii graph annotated with circled numbers.

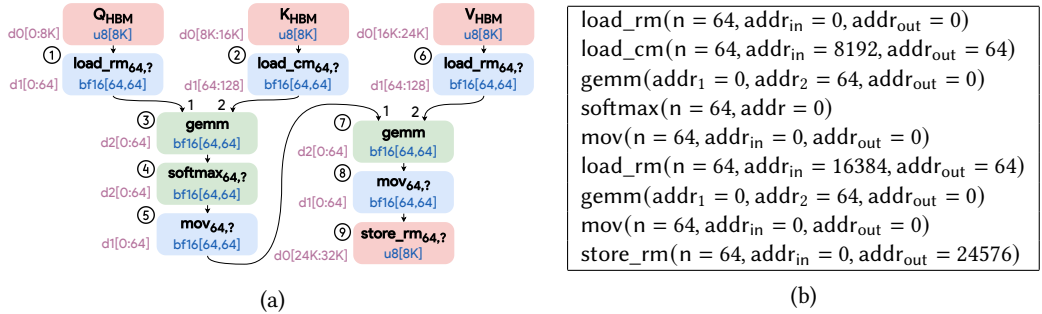


Fig. 13. (a) Extracted pii graph annotated with topological ordering (circled) [after Module 4] and assigned tensor data slices (purple) [after Module 5]. (b) The compiled assembly code for G_{QKV} [after Module 6]. Input tensors Q, K, V occupy relative HBM addresses 0, 8K, 16K (as determined by the XLA middle-end).

6.2 Module 5: Constraint Satisfaction Problem Generator

By this stage, we have a pii graph and a topological ordering of pii nodes $\theta_{\alpha^{(i)},?}^{(i)}|_{i=1}^N$. Next step is to instantiate the addressing attributes $\beta^{(i)}$ of the pii nodes, i.e., allocate the intermediate and constant tensors on the tensor buffers. We formulate a constraint satisfaction problem (CSP) by computing the interference graph and incorporating addressing constraints of the tensor buffers.

The CSP is parameterized by TAIDL constructs (from §3.2) and consists of four components: (1) instruction validity constraints (e_θ), (2) def-use edge constraints over the data slice addresses from the address map h_θ , (3) HBM addressing constraints for the input and output tensors in the input tiled kernel G , whose addresses the XLA middle-end pre-allocates, and (4) live range interference constraints. The final CSP formulation is summarized in Fig. 14.

The final CSP is passed to an existing SAT-based CP solver library [39] to find a satisfying solution over the addressing attributes $\beta^{(i)}$. Fig. 13 (a) shows the data slice addresses assigned to pii nodes as determined by solving the constraint problem.

Implementation of Module 5. Let's look at a concrete example of template-driven implementation (discussed in §4.3) of the *parameterized* algorithm of Module 5. It consists of a generic skeleton (Fig. 15) with symbolic templates (Fig. 16) specialized at generation-time (Fig. 17). At compile time, the CSP builder calls these specialized functions at each pii node.

$$\begin{aligned}
\text{CSP}_1 &= \bigwedge_{i=1}^N \left(e_{\theta(i)}(\alpha^{(i)}, \beta^{(i)}) \wedge (I_s^{(i)}, I_e^{(i)}) = h_{\theta(i)}^{(0)}(\beta^{(i)}) \right) \\
\text{CSP}_2 &= \bigwedge_{(V^{(y)}, V^{(x_i)})} (I_s^{(x_i)}, I_e^{(x_i)}) = h_{\theta(y)}^{(i)}(\beta^{(y)}) \\
\text{CSP}_3 &= \left(\bigwedge_{i=1}^{N_V} (I_s^{(N+i)}, I_e^{(N+i)}) = (I_s^{(G_{x_i})}, I_e^{(G_{x_i})}) \right) \wedge (I_s^{(N)}, I_e^{(N)}) = (I_s^{(G_y)}, I_e^{(G_y)}) \\
\text{live_range}(i) &= \begin{cases} [0, N], & V^{(i)} \text{ is an input variable node,} \\ [0, b_i - 1], & V^{(i)} \text{ is a constant leaf node,} \\ [a_i, b_i - 1], & V^{(i)} \text{ is an intermediate node} \end{cases} \\
\phi &= \{(i, j) \mid d_{\theta(i)}^{(0)} = d_{\theta(j)}^{(0)} \wedge \text{live_range}(i) \cap \text{live_range}(j) \neq \emptyset\} \\
\text{CSP}_4 &= \bigwedge_{(i,j) \in \phi} \text{disjoint}(i, j) \\
\text{CSP} &= \text{CSP}_1 \wedge \text{CSP}_2 \wedge \text{CSP}_3 \wedge \text{CSP}_4
\end{aligned}$$

Fig. 14. Summary of the CSP formulation used in Module 5. Notations used: N is the number of pii nodes, N_V is the number of input variables in the tiled kernel G , $V^{(i)}$ denotes node i , $e_{\theta(i)}$ is the instruction validity constraint, $h_{\theta(i)}$ denotes the data slice address map, $(I_s^{(i)}, I_e^{(i)})$ are data slice addresses, a_i is the topological number of node i , b_i is the maximum topological number among uses of node i , $d_{\theta(i)}^{(0)}$ is the output tensor buffer of node i , and $\text{disjoint}(i, j)$ enforces non-overlapping slice addresses for nodes i and j .

```

1 void CSP1(const std::vector<INSTR*>& instructions) {
2   for (const auto& instr : instructions) {
3     solver.AddConstraint(instr->get_e());
4     solver.MakeEquality(instr->output->start_addr, instr->get_h0().first);
5     solver.MakeEquality(instr->output->end_addr, instr->get_h0().second);
6   }
7 }

```

Fig. 15. Snippet of Generic implementation of Module 5 using abstract C++ class `INSTR` for ISA construct θ with pure virtual functions `get_h0()` and `get_e()` representing ISA constructs $h_{\theta}^{(0)}$ and e_{θ} respectively,

```

1 class INSTR_<<str instr.name>> : protected INSTR { ... }
2 std::pair<IntExpr*, IntExpr*> INSTR_<<str instr.name>>::get_h0() override {
3   return {<<ortools::IntExpr* instr.output.start>>, MAKE_SUM(<<ortools::IntExpr*
4     instr.output.start>>, <<int64 instr.output.len>>)};

```

Fig. 16. Symbolic Template for ISA construct $h_{\theta}^{(0)}$.

```

1 class INSTR_load_rm : protected INSTR { ... }
2 std::pair<IntExpr*, IntExpr*> INSTR_load_rm::get_h0() override {
3   return {addr_out, MAKE_SUM(addr_out, n)};
4 }

```

Fig. 17. Specialized Template for $h_{\text{load_rm}}^{(0)}$ generated by processing line 9 of Fig. 6.

6.3 Module 6: Code Emitter

The final module sequentially emits the sorted pii nodes with their solved addressing attributes and discarding identity instructions (no-ops). Fig. 13 (b) shows the emitted assembly code for G_{QKV} .

6.4 Intra-phase Fallback Mechanism

The depth-first topological ordering generated by the heuristic-based algorithm (§6.1) is not sufficient for the completeness of Phase 2. We prove this using a counterexample in Appendix B. The tensor accelerator instructions and the identity instructions (slice^H & concat^H) often have stricter constraints (e_θ) on the data slice addresses than what classical register allocation algorithms support. Theoretically, we need to search through all topological orderings for completeness.

Pruning technique. We observe that CSP_4 is monotonic w.r.t. the interference graph, i.e., $\phi_1 \subseteq \phi_2 \implies \neg \text{CSP}_4(\phi_1) \rightarrow \neg \text{CSP}_4(\phi_2) \implies \neg \text{CSP}(\phi_1) \rightarrow \neg \text{CSP}(\phi_2)$. Therefore, we discard any topological ordering with a sub-interference graph of a previously failed topological ordering. This reduces the exponential search space of topological orderings to a small number of unique interference graphs, which is tractable to search through if the heuristic-based ordering fails. Exhaustively searching these topological orderings guarantees the completeness of Phase 2.

7 Code Generation with Cost Models

In the previous sections, we have discussed the core algorithm of ACT with fallback mechanisms that guarantee completeness, i.e., find an equivalent assembly code if one exists. We observe that the first equivalent assembly code may not be performant enough. Therefore, we designed a code generator with a simple enumeration pipeline using cost models to generate performant code.

<pre> 1 def codegen(G, n=2): 2 # (asm, cost, enumeration time) 3 final = (None, inf, 0) 4 for asm in act_enum(G): 5 # Compare candidate asm 6 time = now() 7 cost = cost_model(asm) 8 if cost < final.cost: 9 # Select candidate asm 10 final = (asm, cost, time) 11 elif time > n * final.time: 12 # Termination heuristic 13 return final </pre> (a) Candidate selection using performance cost model	<pre> 1 def act_enum(G): 2 egraph = init(G); 3 while not egraph.saturated: 4 egraph = egraph.applier(k) 5 piis = egraph.extract($\Gamma(k)$) 6 for pii in piis: 7 schedules = topo(pii); 8 for schd in schedules: 9 csp = csp_gen(schd); 10 sol = solve(csp) 11 if sol.SAT: 12 asm = emit(pii, sol) 13 yield asm </pre> (b) Candidate enumeration function using ACT-generated compiler backend
--	---

Fig. 18. Pseudocode for code generator using cost models to generate performant code

The code generator (Fig. 18) uses ACT-generated compiler backend as an enumerator to generate multiple equivalent assembly candidates. A simple candidate selection loop iterates through these candidates and selects the final assembly code with the least cost using a performance cost model.

Candidate enumeration. Fig. 18 (b) shows the pseudocode of the enumerator function based on the ACT algorithm (described in Fig. 7) with `yield` statement¹ at line 13. The fallback mechanisms (lines 3, 6, 8) give control back to the previous module. This allows the compiler backend to continue after a successful compilation and repeatedly `yield` equivalent assembly code.

Candidate selection. Fig. 18 (a) shows the code generator that iterates through the candidate assembly codes (line 4) enumerated by the ACT-generated compiler backend (Fig. 18 (b)). Performance statistics of these candidates are collected (line 7), and the candidate with the best performance (least execution cost) is selected (line 8) as the final assembly code.

Termination heuristic. We design a termination heuristic that stops the enumeration if the performance of the enumerated candidates doesn't improve for a considerable time (Fig. 18 (a) line 11). Such a heuristic is required since there are, theoretically, infinite equivalent assembly code candidates with redundant load-store pairs. However, redundant data movement makes such

¹<https://docs.python.org/3/reference/expressions.html#yield-expressions>

candidates longer than ideal and less performant. The first phase prioritizes shorter assembly codes (§5.4), and thus, such candidates are likely to be enumerated after their ideal counterparts. Therefore, we expect the performance to saturate after the ideal candidates are enumerated. In §8.5, we show that the code generators match the performance of state-of-the-art kernel libraries like the oneDNN library [50], with the observed compilation time less than 100 ms.

Choice of Cost Model. Performance statistics can be collected using different fidelity cost models, including cycle-accurate simulators [88], analytical models [74], and learned cost models [66]. Note that the enumeration pipeline and the ACT algorithm are *agnostic to the choice of cost model*.

8 Evaluation

We performed three sets of evaluations to demonstrate the effectiveness of ACT.

- **Supported Accelerators (§8.2):** We generated seven compiler backends for diverse tensor accelerator designs from industry and academia, demonstrating generality of our approach.
- **Comparative Evaluations (§8.3–§8.6):** We evaluated (a) the compilation coverage of ACT-generated compiler backends and (b) performance of their compiled code, against existing accelerator-specific backends and expert-written kernel libraries for tensor accelerators.
- **Compilation Time Analysis (§8.7):** We performed breakdown analyses of the compilation time for varying tiled kernel sizes from synthetic and real-world workloads.

8.1 Implementation

We used egg [100] for equality saturation (§5) and Google OR-Tools [39] for solving CSPs (§6). ACT is implemented as a generic compiler backend (1.9 KLOC of C++ & 6.7 KLOC of Rust), symbolic ISA templates (64 LOC of C++ & 68 LOC of Rust), and a template specialist (1.7 KLOC of Python). The ISA descriptions are parsed by the ANTLR-based TAIDL front-end [51], which we reuse.

8.2 Supported Accelerators

TAIDL [51] provides ISA descriptions for diverse accelerator designs from industry [46, 56, 82] and academia [36, 56, 93]. We extend these TAIDL descriptions by annotating the instruction attributes as computational (α) or addressing (β) — a one-line change to the 10–15 lines of the original TAIDL descriptions for each instruction (e.g., line 7 of Fig. 6). Including the running example H_{QKV} implemented using Allo [14], we have a total of **seven** accelerators tabulated in Table 3.

We use ACT to generate compiler backends for these seven accelerators, showing *generality* of our approach. Since ACT’s compilation algorithm is parameterized by TAIDL constructs, it handles diverse design features of these accelerators with no changes to the core compilation algorithm.

Tensor Accelerator	Existing Backend	Existing Library	ACT Backend
AWS Trainium [82]	✓	✓	✓
Intel AMX [46]	✓	✓	✓
Google TPUv1 [56]	✗	✗	✓
Gemmini [36]	✗	✓	✓
FEATHER [93]	✗	✗	✓
EVA [104]	✗	✗	✓
H_{QKV}	✗	✗	✓

Table 3. The seven tensor accelerators supported by ACT. ✗ refers to the absence of publicly available accelerator-specific compiler backends or expert-written kernel libraries.

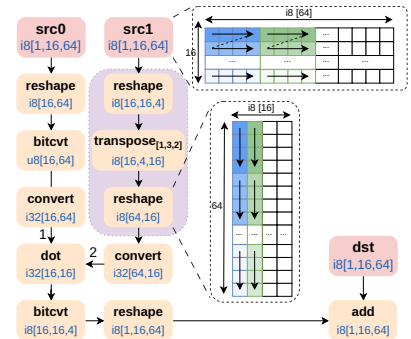


Fig. 19. Highlighted region shows the layout transformation applied to tile register src1 in Intel AMX instruction tdpbusd.

We list the notable design features below.

- (1) **AWS Trainium** [82] includes two on-chip scratchpads with 2-dimensional memory [86], organized in 128 partitions. AWS provides a virtual ISA, AWS NKI [84], to program Trainium.
- (2) **Intel AMX** [46] includes 2-dimensional registers (tiles) and supports instructions with complex layout transformations like `tdpbust` (visualized in Fig. 19), which are semantically modeled in TAIDL as a series of reshape and transpose operators (purple region).
- (3) **Google TPUv1** [56] supports instructions with variable-sized inputs ($B \times 256$).
- (4) **Gemmini** [36] supports instructions to reconfigure dataflow (output vs weight-stationary).
- (5) **FEATHER** [93] supports instructions to reconfigure input data layouts.
- (6) **EVA** [104] is a CGRA-based accelerator with evolvable ISA and 3-dimensional addressing.
- (7) **H_{QKV}** is implemented using an accelerator design language (ADL), Allo [14].

Execution of Compiled Assembly Code

Revisiting §3.1, we discussed that ACT-generated compiler backends emit code in the form of a sequence of TAIDL instructions, which are then stitched together with control flow for tiling into a single *stitched* IR. This stitched IR includes MLIR dialects [60] such as `scf` and `arith`.

Depending on the host-accelerator interface, the stitched IR is compiled and deployed differently.

(i) For built-in accelerators such as Intel AMX [46] and Gemmini [36], the stitched IR is compiled into an executable using the host CPU’s code generation (e.g., x86 backend). In this case, the `scf` for construct is compiled to the host’s control flow instructions (e.g., branch and jump instructions).

(ii) For dedicated accelerators such as AWS Trainium [82] and FEATHER [93], the stitched IR is translated to the accelerator’s kernel programming interface (e.g., AWS Neuron Kernel Interface (NKI) [84] for AWS Trainium [82]). In this case, the `scf` for construct is directly mapped to the kernel programming interface’s control flow constructs (e.g., for loop construct in AWS NKI [85]).

This mapping is automatically inferred from the TAIDL description of the accelerator’s loop construct. The stitching implementation is otherwise generic across accelerators.

Note that regardless of the host-accelerator interface, ACT-generated compiler backends focus on tiled kernels in XLA-HLO IR and emit assembly code as per the ISA description in TAIDL.

8.3 Comparative Evaluations

Out of the seven tensor accelerators supported by ACT, only three have a publicly available accelerator-specific compiler backend or expert-written kernel library for comparative evaluations.

- **AWS Trainium (§8.4):** ACT-generated backend (ACT-NKI) vs. `neuronx-cc` [83] (backend)
- **Intel AMX (§8.5):** ACT-generated backend (ACT-AMX) vs. `oneDNN` library [50]
- **Gemmini (§8.6):** ACT-generated backend (ACT-Gemmini) vs. `Exo` [45] and `SW` library [37]

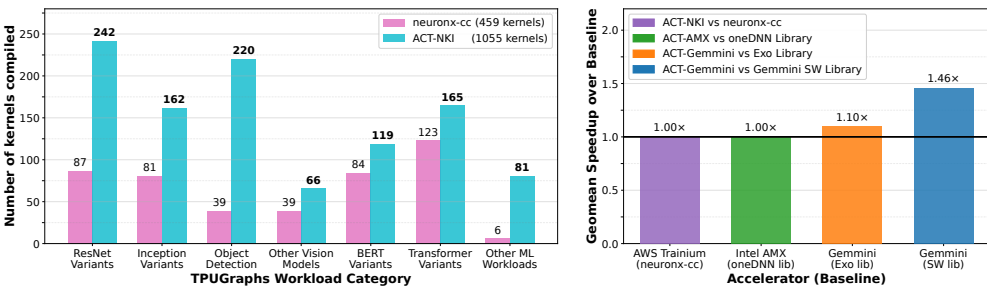


Fig. 20. Summary of comparative evaluations. (a) Number of TPUGraphs kernels correctly compiled by ACT-NKI and `neuronx-cc`, grouped by workload category. (b) Geomean speedup of the compiled code by ACT-generated compiler backends against their corresponding baselines.

Here, we present the notable results with detailed analyses for each accelerator in §8.4–§8.6.

(a) Compilation Coverage

Fig. 20 (a) shows the number of tiled kernels in TPUGraphs [76] Tile dataset that can be compiled to AWS NKI by ACT-NKI and AWS’s compiler neuronx-cc. More details are in §8.4.

$$\text{Compilation Coverage} = \frac{\text{Number of kernels that (i) are successfully compiled to AWS NKI and (ii) the compiled NKI code is functionally correct (randomized testing)}}{\text{Total number of tiled kernels in the benchmark suite (= 1055)}}$$

TPUGraphs kernels are only compatible with AWS Trainium since Google TPUv3 (the original target for TPUGraphs) and AWS Trainium have similar compute capabilities and the same tiling factor (128). Such an extensive dataset of tiled kernels doesn’t exist for Intel AMX and Gemmini. Thus, the compilation coverage study is limited to AWS Trainium.

ACT-NKI achieves **2.3x** better compilation coverage for AWS NKI than neuronx-cc.

(b) Performance of Compiled Code

Fig. 20 (b) summarizes the geomean speedup of the assembly code compiled by ACT-generated compiler backends against their respective baselines [37, 45, 50, 83]. More details are in §8.4–§8.6.

ACT-generated compiler backends generate performant assembly code that matches or outperforms commercial compiler backends and expert-written kernel libraries.

8.4 Comparative Evaluations: AWS Trainium [82]

We evaluate ACT-NKI against AWS’s production compiler neuronx-cc.

Benchmarks

The TPUGraphs [76] Tile dataset is a benchmark suite of tiled kernels generated by Google’s TPUv3 compiler from real-world ML workloads.

Evaluation Methodology

We use AWS’s compiler neuronx-cc [83] with `-print-nki` flag to emit AWS NKI code for the benchmark kernels. We evaluate correctness and performance on an AWS EC2 instance of `trn1`.

Results

Fig. 20 (a) shows that ACT-NKI achieves **2.3x** better compilation coverage than neuronx-cc. Since ACT-NKI has the completeness property, it correctly compiles all 1055 benchmark kernels.

For kernels compiled by both, we observe identical performance, i.e., speedup of **1x**. Note that achieving identical performance is a positive result in itself since neuronx-cc is a production compiler, whereas ACT-NKI is automatically generated by ACT from just the TAIDL description.

Observations & Analysis

The three main reasons for the difference in compilation coverage are:

- (i) ACT-NKI transforms operators not directly supported in AWS NKI like `shift-left` using IR-to-IR rewrites ($\mathcal{R}$ in §2.1) during equality-saturation-based instruction selection (§5.2).
- (ii) neuronx-cc doesn’t support certain single-node tiled kernels, leading to empty NKI code.
- (iii) ACT-NKI detects and allocates compile-time constant tensors (§5.3), unlike neuronx-cc, which fails to initialize constants, leading to unsound assembly code (fails correctness tests).

We reported these results to the AWS Trainium team, who have acknowledged these issues with neuronx-cc. Furthermore, AWS has adopted our tooling, showing the practical impact of our work.

8.5 Comparative Evaluations: Intel AMX [46]

We evaluate ACT-AMX against the Intel oneDNN library, the state-of-the-art kernel library used by production tensor compilers like XLA [20] and TorchInductor [4] to compile to Intel AMX.

Benchmarks

The oneDNN library [50] consists of common DNN kernels heavily optimized for Intel x86 ISA, including AMX. We selected five tiled kernels commonly observed in oneDNN examples [49], also used in AMX-related evaluations in [51]. These kernels are broadly categorized into three types: (i) only int8 matrix operations (`cnn_inf_amx` and `rnn_inf_amx`), (ii) only fp32 matrix operations (`mem_format_avx` and `sgemm_avx`), and (iii) a mixture of int8 and fp32 matrix operations (`cnn_inf_mix`). Detailed statistics and visualization for these kernels are present in Appendix H.

ISA Description

The TAIDL description we used to generate ACT-AMX included 4 AMX-INT8 instructions and 24 AVX512-FP32 instructions. Thus, it supports compilation to both AMX and AVX512.

Evaluation Methodology

The enumeration pipeline (§7) of ACT-AMX uses a simple analytical cost model based on individual instruction costs. For fair comparison, ACT-AMX uses the same fixed tile size as the oneDNN library. We evaluate correctness and performance on an Intel Xeon Gold 5415+ CPU.

Results

Final assembly codes by ACT-AMX *match* the performance of oneDNN library, i.e., speedup of **1x**. The int8 computations are mapped to AMX instructions by both ACT-AMX and oneDNN.

Observations & Analysis

The hand-optimized oneDNN assembly code is one of the candidates enumerated by ACT-AMX, by virtue of ACT’s completeness guarantee. This candidate has the best performance, as per the cost model, and thus selected as the final assembly code by ACT-AMX’s enumeration pipeline.

Similar to AWS NKI results, achieving identical performance is a positive result since oneDNN is an expert-written kernel library with hand-written pattern-matching rules that perform complicated IR legalization due to the complex layout transformations in the instruction semantics (see Fig. 19), whereas ACT-AMX is automatically generated by ACT from just the TAIDL description.

ACT-AMX performs the same IR legalization using a sequence of IR-to-IR rewrites: 22 rewrites over 6 iterations in Phase 1 for `cnn_inf_amx`, and 14 rewrites over 4 iterations for `rnn_inf_amx`. This shows the importance of our design choice to integrate IR-to-IR rewrites with the instruction selection phase of ACT (§5.2), with no hand-written accelerator-specific legalization rules.

8.6 Comparative Evaluations: Gemmini [36]

We evaluate ACT-Gemmini against the Exo library [45] and Gemmini SW library [37]

Baselines

Exo Library. Exo [45] is a kernel programming DSL based on an orthogonal paradigm of exocompilation, i.e., the user writes both the input program and its schedule. This includes specifying tile sizes (line 106 of ²), loop transformations (line 158 of ²), instruction selection (line 149 of ²), and buffer selection (line 207 of ²). Exo developers have manually written hand-tuned schedules for six shapes of GEMM targeting Gemmini. This is the state-of-the-art kernel library for these shapes.

Gemmini SW Library. The Gemmini architecture ships with a custom kernel library [37] hand-optimized by Gemmini’s designers. The Gemmini SW library supports three common DNN kernels—MAC: $A \times B + C$, GEMM: $A \times B$, and ADD: $A + B$, where A , B , and D are i8 matrices.

Benchmarks

Exo Library. We selected the six shapes of GEMM (Fig. 21, left) with hand-tuned Exo schedules.

Gemmini SW Library. We selected 15 tiled kernels (Fig. 21, right) categorized into three classes—(i) “supported”: three kernels directly supported by the Gemmini SW library (MAC, GEMM, ADD), (ii) “composite”: nine compositions of GEMM ($\times$) and ADD ($+$) (fusion of upto 3 library calls), and (iii) “not supported”: three tensor operators not supported by the library (broadcast, reverse, reduce).

²<https://github.com/exo-lang/exo/blob/v1.0.0/src/exo/platforms/gemmini.py>

ISA Description

The TAIDL description we used to generate ACT-Gemmini included 11 RISC-type instructions for Gemmini. Gemmini also includes experimental CISC-type instructions that use an on-chip FSM. Here, we focus on the RISC-type instructions, with the rest discussed in Appendix G.

Evaluation Methodology

The enumeration pipeline (§7) of ACT-Gemmini uses a simple analytical cost model based on individual instruction costs. The reported performance is measured using RTL simulations. Since ACT focuses on backend passes, we delegate middle-end optimizations to XLA’s autotuner [77]. Against the Exo library, we let ACT-Gemmini search across tiling and loop fission factors. Against the Gemmini SW library, we use a fixed tiling factor, same as the SW library.

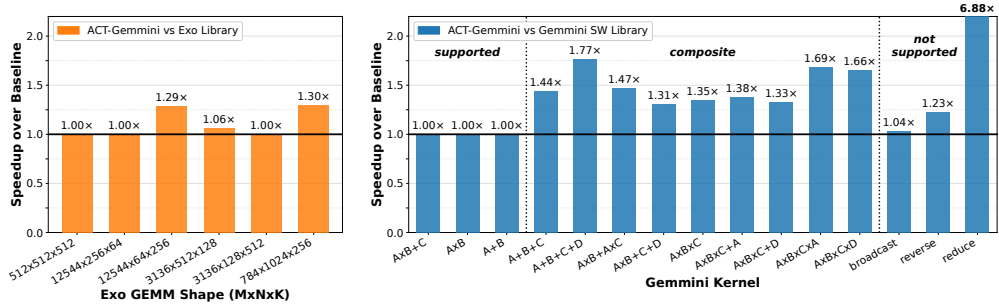


Fig. 21. Speedup of compiled code by ACT-Gemmini over Exo library (left) and Gemmini SW library (right). Note that the “not supported” evaluation (right) is a *cross-architecture* comparison: these operators have no Gemmini implementation in the SW library, so the baseline numbers were measured against the host CPU.

Results

Fig. 21 shows the speedup of compiled code by ACT-Gemmini over the expert-written kernel libraries. ACT-Gemmini with XLA autotuning *matches or outperforms* the hand-tuned Exo library with a speedup of up to **1.30x**. ACT-Gemmini achieves speedups up to **1x**, **1.77x**, and **6.88x** over the Gemmini SW library for “supported”, “composite”, and “not supported” kernels, respectively. For “not supported” kernels, the SW library has no Gemmini implementation, so this speedup is measured against host (Gemmini’s Rocket CPU) execution — the practical cost of host fallback.

Observations & Analysis

Here, we summarize the key observations with individual kernels discussed in Appendix F & G.

Exo Library. ACT-Gemmini enumerates candidates with different tuning factors, including Exo’s hand-tuned factors. Similar to §8.5, the hand-tuned baseline code is one of the enumerated candidates, by virtue of ACT’s completeness guarantee. For three shapes, this candidate has the best performance, as per the cost model, and thus selected as the final assembly code by ACT-Gemmini. For other three shapes, candidates with better loop fission factors were selected by ACT-Gemmini.

Gemmini SW Library (“supported”). Similarly, we observed that the final assembly codes compiled by ACT-Gemmini were the same as the Gemmini SW Library for the three “supported” kernels.

Gemmini SW Library (“composite”). ACT-Gemmini eliminates the redundant load-store instructions between library calls for “composite” kernels by storing the intermediates in the accelerator scratchpads. This increases data reuse and reduces data movement, thus better performance. We observed that speedup increases with the kernel size (# nodes). For GEMM ($\times$), speedup goes from 1x ($A \times B$) to 1.35x ($A \times B \times C$) to 1.66x ($A \times B \times C \times D$). Likewise for ADD (+), 1x to 1.44x to 1.77x. We limit our evaluations to a maximum of 3 library calls to avoid exaggerated performance results.

Gemmini SW Library (“not supported”). ACT-Gemmini is able to compile previously unsupported tensor operators like reduce to Gemmini using IR-to-IR rewrites (§2.1) and compile-time constant detection (§5.3). This leads to large speedups (up to 6.88x) over host execution fallback.

8.7 Compilation Time Analysis

Next, we perform breakdown analyses of the compilation time of ACT-generated compiler backends for varying tiled kernel sizes from synthetic and real-world workloads.

Tiled kernels typically contain 10–50 nodes, as seen in TPUGraphs dataset [76]. However, the Gemini kernels used in §8.6 were limited to at most 10 nodes since the baselines were kernel libraries. Therefore, to analyze the compilation time of ACT-Gemmini on representative tiled kernels, we generated 100 synthetic kernels (size 7–89 nodes) using NNSmith [62], a random DNN generator (more details in Appendix E). We also analyze the compilation time of ACT-NKI for the three largest TPUGraphs kernels, labeled as T1 (150 nodes), T2 (270 nodes), and T3 (390 nodes).

Evaluation methodology

The compilation time reported is final.time in Fig. 18 (a) and was measured on a 20-core Intel i7-13700H CPU, averaged over 20 trials. This is the wall clock time of the *entire* compilation, including all pii candidates, fallback re-attempts (§4.1), and the cost model loop (§7). ACT-Gemmini and ACT-NKI were set to an overall timeout after 5 seconds, which was never reached (max ~500 ms).

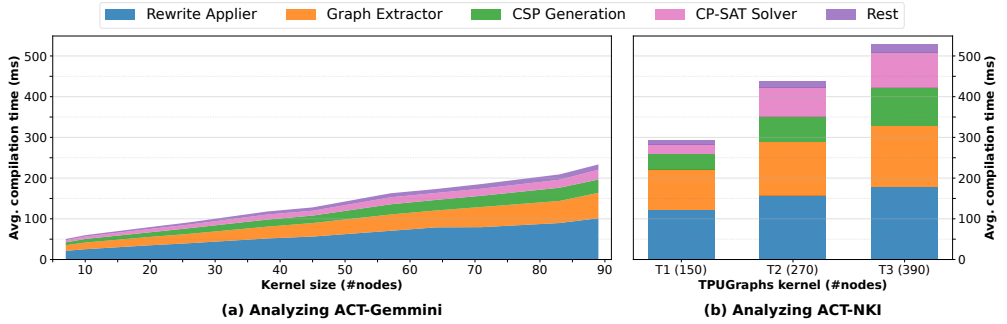


Fig. 22. Breakdown of compilation time. (a) ACT-Gemmini on synthetic kernels generated using NNSmith [62] and (b) ACT-NKI on the three largest kernels in TPUGraphs [76] Tile dataset.

Results & Observations

Fig. 22 shows the breakdown of compilation time of ACT-Gemmini and ACT-NKI. We observed small compilation times (< 233 ms) for a wide range of kernel sizes and even for large kernel sizes (529 ms for 390 nodes). The main reasons for the small compilation time are:

- Compact representation of infinite equivalent assembly code by modeling instruction selection as an equality saturation problem. Fig. 23 shows a simple snippet of an e-graph with cycles representing infinite equivalent assembly code.
- Bounded approach of graph extraction (in §5.4) prioritizes shorter assembly code, thereby enumerating candidates with no redundant data movement first. This leads to faster saturation of performance cost and shorter compilation times.
- Instruction scheduling heuristic (in §6.1) aims to minimize the interference graph. This results in fewer constraints in the generated CSP and, thereby, results in short solver time, even for large pii graphs (86 ms for 390 nodes).

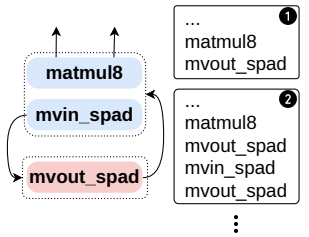


Fig. 23. Snippet of e-graph (left) with cycles between mvin and mvout e-classes. This represents infinite equivalent candidates (right), and the candidate with no redundant data movement is enumerated first.

ACT-generated compiler backends maintain low compilation overhead for a wide range of tiled kernel sizes.

9 Related Works

ISA description languages. VADL [33] and ILA [44] describe ISAs at the *scalar* level, using constructs such as `forall`, `fold`, and explicit array indexing, and focus primarily on CPU ISAs. TAIDL [51] instead describes instruction semantics at the *tensor* level (§3.2), and its tensor operators and types can themselves be parameterized by instruction attributes. Together, these features enable TAIDL to model the complex, parameterized instruction sets of tensor accelerators. We adopt TAIDL since it already supports several existing accelerator platforms [14, 36, 46, 56, 82, 93, 104].

Vectorizer generators and synthesis-based compilation. Vectorizer generators such as Vegen [18], Diospyros [94], and Isaria [91] target CPUs like x86 and DSPs like Hexagon HVX with fixed-length vector instructions. Synthesis-based instruction selectors such as Hydride [57] and Rake [2] also target fixed-width instructions, which are amenable to SMT-based reasoning. Compared to these techniques that only focus on instruction selection, ACT generates the entire compiler backend, including memory allocation for tensor accelerators. Furthermore, prior works cannot directly handle the complex parameterized instructions present in tensor accelerator ISAs.

Automated compiler backend generation. Prior work on automated CPU backend generation includes declarative machine-description approaches [28], CEGIS-based instruction-selector synthesis [10], processor-description-based generation [32], and LLM-based approaches [102]. OpenVADL [34, 75] generates TableGen-style LLVM backends from VADL [33], assuming little semantic gap between VADL and LLVM IR and thus requiring no complex legalization. Another related work in this area is a backend generator for the xADL processor-description language [8, 9], which targets only scalar register-based CPU ISAs and leaves vector ISAs to future work. Tensor accelerators instead have parameterized instructions, software-managed scratchpads, and layout-transforming semantics (e.g., Intel AMX [46], Fig. 19) that require accelerator-specific legalization. ACT therefore interleaves legalization with instruction selection in an e-graph rather than using a pattern-based selector, which also avoids phase ordering (§2.4).

Compiler support for novel accelerators. 3LA [43] is a recent work that makes commendable progress in developing basic compiler support by enabling simulation testing for end-to-end ML workloads on novel hardware. However, it requires the user to provide tensor IR-to-accelerator IR mapping for instruction selection and an accelerator IR-to-assembly code generation driver, and does not model scratchpad reuses. In contrast, ACT focuses on automatically generating a compiler backend just from the accelerator ISA description.

MLIR-based compiler stacks. MLIR [60] provides a multi-level compiler infrastructure with extensible dialects, and many modern ML compilers are built around this ecosystem, including the XLA compiler [20] that uses the StableHLO dialect [23]. MLIR ecosystem includes various lowering passes at a machine-independent IR level such as memref bufferization (note that this is different from the on-chip memory allocation discussed in §6), as well as hand-written target-specific lowering passes [38, 42] for code generation. ACT is complementary to these efforts — it focuses on automatically generating the accelerator-specific compiler backend itself (instruction selection, scheduling, and on-chip tensor memory allocation), and these generated backends can be integrated as a target-specific lowering pass in a larger MLIR-based pipeline.

Kernel programming languages like Exo [45], Pallas [40], Triton [92], and NKI [84] externalize target-specific code generation to user code, simplifying accelerator programming. These languages offer high-level constructs to program hardware, rather than directly coding in assembly, increasing user productivity. Exo, in particular, offers an intuitive scheduling DSL for exploring optimizations. However, these languages do not automatically generate compiler backends; instead, they rely on users to manually write instructions and schedules, and require a device-specific code generator.

Equality Saturation in Compilers. Prior works like Tensat [101], SPORES [99], Cranelift [31] use equality saturation to optimize machine learning computation graphs, linear algebra, and Rust functions, respectively. These works focus only on middle-end optimizations using IR-to-IR rewrites, whereas ACT uses both IR-to-IR rewrites and auto-generated IR-to-ISA rewrites to concurrently explore instruction selection opportunities. MISAAL [72] uses equality saturation to lower Halide IR using synthesized rewrite rules, and LIAR [26] likewise applies synthesized rewrites to recognize latent idioms in a functional array language. However, their rewrites carry no preconditions, and thus can not model parameterized instructions prevalent in tensor accelerator ISAs [84] (§5.2).

E-graph extraction algorithms. Prior equality-saturation workflows typically perform one-shot extraction using either greedy heuristics (the default strategy in egg [100]) or ILP-based optimal extraction [99, 101], with SmoothE [11] recently relaxing the same one-shot problem into a differentiable one. These methods are typically tied to a local objective integrated into the extraction algorithm. In our setting, this is insufficient because a candidate pii graph selected in Phase 1 can still fail in Phase 2 (as discussed in §4.1). Therefore, ACT uses an enumerative extraction algorithm (§5.3) with inter-phase fallback ensuring all equivalent pii graphs are enumerated and remains independent of any built-in cost objectives, allowing integration of external black-box cost models.

Instruction scheduling is the closest problem to the topological ordering discussed in §6.1. A common technique for scheduling instructions for pipelined processors is list scheduling [69]. But unlike our problem, these algorithms minimize the critical path length rather than the interference graphs. The Sethi-Ullman algorithm [87] generates optimal code for arithmetic expressions using minimal registers. Prior works have generalized this to 1-load binary DAGs [15] and non-binary trees [5]. However, any depth-first traversal of the expression tree is not always optimal for more than one register file (counterexample in Appendix B). We extend the Sethi-Ullman algorithm to multiple multi-dimensional tensor buffers with a fallback pruning strategy (§6.4).

On-chip scratchpad and register allocation are the closest problems to the tensor memory allocation discussed in §6.2. A widely adopted solution for register allocation is graph coloring [3] over the interference graph, while the puzzle board model [80] handles registers of different sizes. Unison [63, 64] formulates register allocation and instruction scheduling as a single combinatorial problem, and bit-aware register binding [12] admits a polynomial-time exact solution when re-allocation is permitted. None of these, however, extend to our setting, since we pack multi-dimensional tensor buffers into multi-dimensional scratchpads (§6.2) rather than one-dimensional registers, and not all buffers are spillable [84]. On-chip scratchpad memory allocation is often related to the rectangle packing problem, which can be modeled as a constraint satisfaction problem (CSP), with improved scalability by pruning techniques proposed in meta-CSP [68], TelaMalloc [65], and MiniMalloc [67]. However, these solutions only model interference graphs and, thus, can not represent complex addressing constraints prevalent in tensor accelerator ISAs [84] and the identity instructions (discussed in §5.2), both of which ACT encodes in its generated CSP.

10 Conclusion

In this paper, we propose the first accelerator compiler backend generator, ACT, that automatically generates a sound and complete compiler backend from a tensor accelerator ISA description. We provide a novel ISA-parameterized compilation algorithm and demonstrate the generality of our approach by generating backends for seven tensor accelerators. These backends generate performant code with large compilation coverage and small compilation times.

ACT fills an important gap in software development for accelerator platforms left under-explored due to the lack of compiler backends targeting them. It provides an agile and evolvable methodology to quickly build compiler backends that bridge the hardware and software communities.

11 Data Availability Statement

Our artifact is a repository of code (10+ KLOC) written in C++, Rust, and Python that implements ACT compatible with XLA-HLO IR. The artifact also includes benchmarks and docker environments used for performance evaluation, fuzz testing, stress testing, and scripts to reproduce the plots. Zenodo archive of the artifact is available at <https://doi.org/10.5281/zenodo.21926082>.

ACT is part of a larger open-source ecosystem, built around our ISA description language TAIDL, that automatically generates essential software tools, such as test oracles and compiler backends, from ISA descriptions of tensor accelerators. Our tooling has been adopted by multiple academic and industry teams designing novel tensor accelerators. The ecosystem and its associated tutorials are accessible at <https://github.com/act-compiler/act>; we welcome contributions and feedback.

12 Acknowledgments

We thank the anonymous reviewers for their constructive feedback, as well as Stefanos Baziotis and Damitha Lenadora for their valuable comments on the paper. Special thanks to Zhihao Wang, Advait Tahilyani, Alan Xie, Pedro Couto, Tanmay Garudadri, Nitin Krishna, Mihir Tandon, and Aarav Khanna for their contributions to the ecosystem and its tooling. We are grateful to Niansong Zhang, Jianming Tong, and Minsik Kim for their guidance on using their respective accelerator designs. This work was supported in part by ACE, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA, and by NSF under grant CCF-2338739.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (Savannah, GA, USA) (OSDI'16)*. USENIX Association, USA, 265–283.
- [2] Maaz Bin Safer Ahmad, Alexander J Root, Andrew Adams, Shoaib Kamil, and Alvin Cheung. 2022. Vector Instruction Selection for Digital Signal Processors Using Program Synthesis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 1004–1016. doi:10.1145/3503222.3507714
- [3] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., USA. <https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472>
- [4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarikar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366
- [5] A. W. Appel and K. J. Supowit. 1987. Generalization of the Sethi-Ullman algorithm for register allocation. *Softw. Pract. Exper.* 17, 6 (June 1987), 417–421. doi:10.1002/spe.4380170607
- [6] Jai Arora, Sirui Lu, Devansh Jain, Tianfan Xu, Farzin Houshmand, Phitchaya Mangpo Phothilimthana, Mohsen Lesani, Praveen Narayanan, Karthik Srinivasa Murthy, Rastislav Bodik, Amit Sabne, and Charith Mendis. 2025. TensorRight: Automated Verification of Tensor Graph Rewrites. *Proc. ACM Program. Lang.* 9, POPL, Article 29 (Jan. 2025), 32 pages. doi:10.1145/3704865
- [7] Franz Baader and Tobias Nipkow. 1998. *Term Rewriting and All That*. Cambridge University Press. doi:10.1017/CBO9781139172752
- [8] Florian Brandner, Dietmar Ebner, and Andreas Krall. 2007. Compiler Generation from Structural Architecture Descriptions. In *Proceedings of the 2007 International Conference on Compilers, Architecture, and Synthesis for Embedded*

- Systems (CASES '07)*. Association for Computing Machinery, 13–22. doi:10.1145/1289881.1289886
- [9] Florian Brandner, Viktor Pavlu, and Andreas Krall. 2013. Automatic Generation of Compiler Backends. *Software: Practice and Experience* 43, 2 (2013), 207–240. doi:10.1002/spe.2106
 - [10] Sebastian Buchwald, Andreas Fried, and Sebastian Hack. 2018. Synthesizing an instruction selection rule library from semantic specifications. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization* (Vienna, Austria) (CGO '18). Association for Computing Machinery, New York, NY, USA, 300–313. doi:10.1145/3168821
 - [11] Yaohui Cai, Kaixin Yang, Chenhui Deng, Cunxi Yu, and Zhiru Zhang. 2025. SmoothE: Differentiable E-Graph Extraction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*. Association for Computing Machinery, 1020–1034. doi:10.1145/3669940.3707262
 - [12] Michael Canesche, Ricardo Ferreira, José Augusto Nacif, and Fernando Magno Quintão Pereira. 2022. A Polynomial Time Exact Solution to the Bit-Aware Register Binding Problem. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (CC 2022)*. Association for Computing Machinery, 29–40. doi:10.1145/3497776.3517773
 - [13] Patrick Cegielski and Denis Richard. 2001. Decidability of the theory of the natural integers with the cantor pairing function and the successor. *Theor. Comput. Sci.* 257, 1–2 (April 2001), 51–77. doi:10.1016/S0304-3975(00)00109-2
 - [14] Hongzheng Chen, Niansong Zhang, Shaojie Xiang, Zhichen Zeng, Mengjia Dai, and Zhiru Zhang. 2024. Allo: A Programming Model for Composable Accelerator Design. *Proc. ACM Program. Lang.* 8, PLDI, Article 171 (June 2024), 28 pages. doi:10.1145/3656401
 - [15] Stephen Chen. 1975. On the Sethi-Ullman algorithm. *International Journal of Computer Mathematics* 5 (1975), 37–55. doi:10.1080/00207167508803101
 - [16] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) (ASPLOS '14). Association for Computing Machinery, New York, NY, USA, 269–284. doi:10.1145/2541940.2541967
 - [17] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
 - [18] Yishen Chen, Charith Mendis, Michael Carbin, and Saman Amarasinghe. 2021. VeGen: a vectorizer generator for SIMD and beyond. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 902–914. doi:10.1145/3445814.3446692
 - [19] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 367–379. doi:10.1109/ISCA.2016.40
 - [20] OpenXLA Contributors. 2024. XLA Compiler. <https://openxla.org/xla>
 - [21] OpenXLA Contributors. 2025. Dynamism in StableHLO. <https://openxla.org/stablehlo/dynamism>.
 - [22] OpenXLA Contributors. 2026. Operation Semantics. https://openxla.org/xla/operation_semantics.
 - [23] OpenXLA Contributors. 2026. StableHLO Specification. <https://github.com/openxla/stablehlo/blob/main/docs/spec.md/>.
 - [24] OpenXLA Contributors. 2026. XLA:GPU Emitters. <https://openxla.org/xla/emitters>.
 - [25] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1189, 16 pages. doi:10.52202/068431-1189
 - [26] Jonathan Van der Cruysse and Christophe Dubach. 2024. Latent Idiom Recognition for a Minimalist Functional Array Language Using Equality Saturation. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2024)*. IEEE, 270–282. doi:10.1109/CGO57630.2024.10444879
 - [27] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018). doi:10.48550/arXiv.1810.04805
 - [28] João Dias and Norman Ramsey. 2010. Automatically generating instruction selectors using declarative machine descriptions. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Madrid, Spain) (POPL '10). Association for Computing Machinery, New York, NY, USA, 403–416. doi:10.1145/1706299.1706346
 - [29] Zidong Du, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Xiaobing Feng, Yunji Chen, and Olivier Temam. 2015. ShiDianNao: Shifting vision processing closer to the sensor. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*. 92–104. doi:10.1145/2749469.2750389
 - [30] Will Estes. 2016. Lexical Analysis With Flex, for Flex 2.6.2. <https://westes.github.io/flex/manual/index.html>.

- [31] Chris Fallin. 2023. ægraphs: Acyclic E-graphs for Efficient Optimization in a Production Compiler. <https://pldi23.sigplan.org/details/egraphs-2023-papers/2/-graphs-Acyclic-E-graphs-for-Efficient-Optimization-in-a-Production-Compiler> Invited Talk, EGRAPHS 2023 - E-Graph Research, Applications, Practices, and Human-factors Symposium, PLDI 2023.
- [32] Stefan Farfeleder, Andreas Krall, Edwin Steiner, and Florian Brandner. 2006. Effective Compiler Generation by Architecture Description. In *Proceedings of the 2006 ACM SIGPLAN/SIGBED Conference on Language, Compilers, and Tool Support for Embedded Systems* (Ottawa, Ontario, Canada) (LCTES '06). Association for Computing Machinery, New York, NY, USA, 145–152. doi:10.1145/1134650.1134671
- [33] Florian Freitag, Linus Halder, Simon Himmelbauer, Christoph Hochrainer, Benedikt Huber, Benjamin Kasper, Niklas Mischkulnig, Michael Nestler, Philipp Paulweber, Kevin Per, Matthias Raschhofer, Alexander Ripar, Tobias Schwarzinger, Johannes Zottele, and Andreas Krall. 2024. The Vienna Architecture Description Language. arXiv:2402.09087 [cs.PL] <https://arxiv.org/abs/2402.09087>
- [34] Florian Freitag, Linus Halder, Benedikt Huber, Benjamin Kasper, Michael Nestler, Kevin Per, Matthias Raschhofer, Alexander Ripar, Johannes Zottele, and Andreas Krall. 2025. OpenVADL: An Open Source Implementation of the Vienna Architecture Description Language. In *Architecture of Computing Systems (ARCS 2025) (Lecture Notes in Computer Science)*. Springer, 156–171. doi:10.1007/978-3-032-03281-2_11
- [35] Roy Frostig, Matthew James Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. *Systems for Machine Learning* 4, 9 (2018). <https://mlsys.org/Conferences/doc/2018/146.pdf>
- [36] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2021. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration. In *Proceedings of the 58th Annual Design Automation Conference (DAC)*. doi:10.1109/DAC18074.2021.9586216
- [37] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2024. ucb-bar/gemmini: Berkeley's Spatial Array Generator. <https://github.com/ucb-bar/gemmini>.
- [38] Renato Golin, Lorenzo Chelini, Adam Siemieniuk, Kavitha Madhu, Niranjan Hasabnis, Hans Pabst, Evangelos Georganas, and Alexander Heinecke. 2024. Towards a high-performance AI compiler with upstream MLIR. arXiv:2404.15204 [cs.PL] <https://arxiv.org/abs/2404.15204>
- [39] Google. 2021. Google OR Tools: CP-SAT Solver. https://developers.google.com/optimization/cp/cp_solver.
- [40] Google. 2024. Pallas: a JAX kernel language. <https://jax.readthedocs.io/en/latest/pallas/index.html>.
- [41] Muyan Hu, Ashwin Venkatram, Shreyashri Biswas, Balamurugan Marimuthu, Bohan Hou, Gabriele Oliaro, Haojie Wang, Liyan Zheng, Xupeng Miao, Jidong Zhai, and Zhihao Jia. 2024. Optimal Kernel Orchestration for Tensor Programs with Korch. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 755–769. doi:10.1145/3620666.3651383
- [42] Pengchao Hu, Man Lu, Lei Wang, and Guoyue Jiang. 2023. TPU-MLIR: A Compiler For TPU Using MLIR. arXiv:2210.15016 [cs.PL] <https://arxiv.org/abs/2210.15016>
- [43] Bo-Yuan Huang, Steven Lyubomirsky, Yi Li, Mike He, Gus Henry Smith, Thierry Tambe, Akash Gaonkar, Vishal Canumalla, Andrew Cheung, Gu-Yeon Wei, Aarti Gupta, Zachary Tatlock, and Sharad Malik. 2024. Application-level Validation of Accelerator Designs Using a Formal Software/Hardware Interface. *ACM Trans. Des. Autom. Electron. Syst.* 29, 2, Article 35 (Feb. 2024), 25 pages. doi:10.1145/3639051
- [44] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vizel, Aarti Gupta, and Sharad Malik. 2018. Instruction-Level Abstraction (ILA): A Uniform Specification for System-on-Chip (SoC) Verification. *ACM Trans. Des. Autom. Electron. Syst.* 24, 1, Article 10 (Dec. 2018), 24 pages. doi:10.1145/3282444
- [45] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for productive programming of hardware accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 703–718. doi:10.1145/3519939.3523446
- [46] Intel. 2024. Intel® Advanced Matrix Extensions. <https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/advanced-matrix-extensions/overview.html>.
- [47] Intel. 2024. Intel® Intrinsics Guide. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [48] Intel. 2025. Intel® Advanced Vector Extensions 512 (Intel® AVX-512) Overview. <https://www.intel.com/content/www/us/en/architecture-and-technology/avx-512-overview.html>.
- [49] Intel. 2025. oneDNN Examples. <https://github.com/uxlfoundation/oneDNN/tree/v3.8.1/examples>.
- [50] Intel. 2025. oneDNN Library. <https://github.com/uxlfoundation/oneDNN/tree/v3.8.1>.

- [51] Devansh Jain, Marco Frigo, Jai Arora, Akash Pardeshi, Zhihao Wang, Krut Patel, and Charith Mendis. 2025. TAIDL: Tensor Accelerator ISA Definition Language with Auto-generation of Scalable Test Oracles. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 1316–1333. doi:10.1145/3725843.3756075
- [52] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) (*SOSP '19*). Association for Computing Machinery, New York, NY, USA, 47–62. doi:10.1145/3341301.3359630
- [53] Stephen Johnson. 2001. Yacc: Yet Another Compiler-Compiler. *Unix Programmer's Manual 2* (11 2001). <https://www.cs.utexas.edu/users/novak/yaccpaper.htm>
- [54] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (*ISCA '23*). Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. doi:10.1145/3579371.3589350
- [55] Norman P. Jouppi, Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. A domain-specific supercomputer for training deep neural networks. *Commun. ACM* 63, 7 (June 2020), 67–78. doi:10.1145/3360307
- [56] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmamghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) (*ISCA '17*). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3079856.3080246
- [57] Akash Kothari, Abdul Rafae Noor, Muchen Xu, Hassam Uddin, Dhruv Baronia, Stefanos Baziotis, Vikram Adve, Charith Mendis, and Sudipta Sengupta. 2024. Hydride: A Retargetable and Extensible Synthesis-based Compiler for Modern Hardware Architectures. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (*ASPLOS '24*). Association for Computing Machinery, New York, NY, USA, 514–529. doi:10.1145/3620665.3640385
- [58] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects. *SIGPLAN Not.* 53, 2 (March 2018), 461–475. doi:10.1145/3296957.3173176
- [59] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) (*CGO '04*). IEEE Computer Society, USA, 75. doi:10.1109/CGO.2004.1281665
- [60] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO51591.2021.9370308
- [61] Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2022. Verified tensor-program optimization via high-level scheduling rewrites. *Proc. ACM Program. Lang.* 6, POPL, Article 55 (Jan. 2022), 28 pages. doi:10.1145/3498717
- [62] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (*ASPLOS 2023*). Association for Computing Machinery, New York, NY, USA, 530–543. doi:10.1145/3575693.3575707
- [63] Roberto Castañeda Lozano, Mats Carlsson, Gabriel Hjort Blindell, and Christian Schulte. 2016. Register Allocation and Instruction Scheduling in Unison. In *Proceedings of the 25th International Conference on Compiler Construction (CC 2016)*. Association for Computing Machinery, 263–264. doi:10.1145/2892208.2892237

- [64] Roberto Castañeda Lozano, Mats Carlsson, Gabriel Hjort Blindell, and Christian Schulte. 2019. Combinatorial Register Allocation and Instruction Scheduling. *ACM Transactions on Programming Languages and Systems* 41, 3, Article 17 (2019), 53 pages. doi:10.1145/3332373
- [65] Martin Maas, Ulysse Beaugnon, Arun Chauhan, and Berkin Ilbeyi. 2022. TelaMalloc: Efficient On-Chip Memory Allocation for Production Machine Learning Accelerators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 123–137. doi:10.1145/3567955.3567961
- [66] Charith Mendis, Alex Renda, Dr.Saman Amarasinghe, and Michael Carbin. 2019. Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 4505–4515. <https://proceedings.mlr.press/v97/mendis19a.html>
- [67] Michael D. Moffitt. 2024. MiniMalloc: A Lightweight Memory Allocator for Hardware-Accelerated Machine Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Vancouver, BC, Canada) (ASPLOS '23). Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/3623278.3624752
- [68] Michael D. Moffitt and Martha E. Pollack. 2006. Optimal rectangle packing: a meta-CSP approach. In *Proceedings of the Sixteenth International Conference on International Conference on Automated Planning and Scheduling* (Cumbria, UK) (ICAPS'06). AAAI Press, 93–102. <https://cdn.aaai.org/ICAPS/2006/ICAPS06-010.pdf>
- [69] Steven S. Muchnick and Phillip B. Gibbons. 2004. Efficient instruction scheduling for a pipelined architecture. *SIGPLAN Not.* 39, 4 (April 2004), 167–174. doi:10.1145/989393.989413
- [70] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. DNNFusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 883–898. doi:10.1145/3453483.3454083
- [71] Wei Niu, Md Musfiqur Rahman Sanim, Zhihao Shu, Jiexiong Guan, Xipeng Shen, Miao Yin, Gagan Agrawal, and Bin Ren. 2024. SmartMem: Layout Transformation Elimination and Adaptation for Efficient DNN Execution on Mobile. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 916–931. doi:10.1145/3620666.3651384
- [72] Abdul Rafae Noor, Dhruv Baronia, Akash Kothari, Muchen Xu, Charith Mendis, and Vikram S. Adve. 2025. MISAAL: Synthesis-Based Automatic Generation of Efficient and Retargetable Semantics-Driven Optimizations. *Proc. ACM Program. Lang.* 9, PLDI, Article 198 (June 2025), 24 pages. doi:10.1145/3729301
- [73] Anjali Pal, Brett Saiki, Ryan Tjoa, Cynthia Richey, Amy Zhu, Oliver Flatt, Max Willsey, Zachary Tatlock, and Chandrakana Nandi. 2023. Equality Saturation Theory Exploration à la Carte. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2, Article 258 (2023), 29 pages. doi:10.1145/3622834
- [74] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315. doi:10.1109/ISPASS.2019.00042
- [75] Kevin Per. 2025. *LLVM Compiler Generator in OpenVADL*. Master's thesis. Technische Universität Wien.
- [76] Mangpo Phothilimthana, Sami Abu-El-Haija, Kaidi Cao, Bahare Fatemi, Michael Burrows, Charith Mendis, and Bryan Perozzi. 2023. TpuGraphs: A Performance Prediction Dataset on Large Tensor Computational Graphs. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 70355–70375. doi:10.52202/075280-3083
- [77] Phitchaya Mangpo Phothilimthana, Amit Sabne, Nikhil Sarda, Karthik Srinivasa Murthy, Yanqi Zhou, Christof Angermueller, Mike Burrows, Sudip Roy, Ketan Mandke, Reza Farahani, Yu Emma Wang, Berkin Ilbeyi, Blake Hechtman, Bjarke Roun, Shen Wang, Yuanzhong Xu, and Samuel J. Kaufman. 2021. A Flexible Approach to Autotuning Multi-Pass Machine Learning Compilers. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 1–16. doi:10.1109/PACT52795.2021.00008
- [78] Raghu Prabhakar, Sumti Jairath, and Jinuk Luke Shin. 2022. SambaNova SN10 RDU: A 7nm Dataflow Architecture to Accelerate Software 2.0. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 350–352. doi:10.1109/ISSCC42614.2022.9731612
- [79] GNU Project. 2021. Bison 3.8.1. <https://www.gnu.org/software/bison/manual/bison.html>.
- [80] Fernando Magno Quintão Pereira and Jens Palsberg. 2008. Register allocation by puzzle solving. *SIGPLAN Not.* 43, 6 (June 2008), 216–226. doi:10.1145/1379022.1375609
- [81] Amazon Web Services. 2024. AWS Inferentia. <https://aws.amazon.com/ai/machine-learning/inferentia/>.
- [82] Amazon Web Services. 2024. AWS Trainium. <https://aws.amazon.com/ai/machine-learning/trainium/>.

- [83] Amazon Web Services. 2026. Neuron Compiler CLI Reference Guide (neuronx-cc). <https://awsdocs-neuron.readthedocs-hosted.com/en/v2.27.1/compiler/neuronx-cc/api-reference-guide/neuron-compiler-cli-reference-guide.html>.
- [84] Amazon Web Services. 2026. Neuron Kernel Interface (NKI) Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/v2.27.1/nki/index.html>.
- [85] Amazon Web Services. 2026. NKI Language Guide. <https://awsdocs-neuron.readthedocs-hosted.com/en/v2.27.1/nki/get-started/nki-language-guide.html#control-flow>.
- [86] Amazon Web Services. 2026. The Trainium Memory Hierarchy. <https://awsdocs-neuron.readthedocs-hosted.com/en/v2.27.1/nki/get-started/about/memory-hierarchy-overview.html>.
- [87] Ravi Sethi and J. D. Ullman. 1970. The Generation of Optimal Code for Arithmetic Expressions. *J. ACM* 17, 4 (Oct. 1970), 715–728. doi:10.1145/321607.321620
- [88] Wilson Snyder, Paul Wasson, and Duane et al Galbi. 2026. *Verilator*. <https://github.com/verilator/verilator>
- [89] Amazon Staff. 2024. Amazon invests \$110 million to support AI research at universities using Trainium chips. <https://www.aboutamazon.com/news/aws/amazon-trainium-investment-university-ai-research>.
- [90] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) (POPL '09). Association for Computing Machinery, New York, NY, USA, 264–276. doi:10.1145/1480881.1480915
- [91] Samuel Thomas and James Bornholt. 2024. Automatic Generation of Vectorizing Compilers for Customizable Digital Signal Processors. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 19–34. doi:10.1145/3617232.3624873
- [92] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) (MAPL 2019). Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973
- [93] Jianming Tong, Anirudh Itagi, Prasanth Chatarasi, and Tushar Krishna. 2024. FEATHER: A Reconfigurable Accelerator with Data Reordering Support for Low-Cost On-Chip Dataflow Switching. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 198–214. doi:10.1109/ISCA59077.2024.00024
- [94] Alexa VanHattum, Rachit Nigam, Vincent T. Lee, James Bornholt, and Adrian Sampson. 2020. A Synthesis-Aided Compiler for DSP Architectures (WiP Paper). In *The 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems* (London, United Kingdom) (LCTES '20). Association for Computing Machinery, New York, NY, USA, 131–135. doi:10.1145/3372799.3394358
- [95] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [96] Steven R. Vegdahl. 1982. Phase coupling and constant generation in an optimizing microcode compiler. *SIGMICRO Newsl.* 13, 4 (Oct. 1982), 125–133. doi:10.1145/1014194.800942
- [97] Arya Vohra, Leo Seojun Lee, Jakub Bachurski, Oleksandr Zinenko, Phitchaya Mangpo Phothilimthana, Albert Cohen, and William S. Moses. 2025. Mind the Abstraction Gap: Bringing Equality Saturation to Real-World ML Compilers. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 284 (Oct. 2025), 28 pages. doi:10.1145/3763062
- [98] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. 2021. PET: Optimizing Tensor Programs with Partially Equivalent Transformations and Automated Corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 37–54. <https://www.usenix.org/conference/osdi21/presentation/wang>
- [99] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *Proc. VLDB Endow.* 13, 12 (July 2020), 1919–1932. doi:10.14778/3407790.3407799
- [100] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. doi:10.1145/3434304
- [101] Yichen Yang, Phitchaya Phothilimthana, Yisu Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. 2021. Equality Saturation for Tensor Graph Superoptimization. In *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica (Eds.), Vol. 3. 255–268. https://proceedings.mlsys.org/paper_files/paper/2021/file/cc427d934a7f6c0663e5923f49eba531-Paper.pdf

- [102] Ming Zhong, Fang Lv, Lulin Wang, Lei Qiu, Yingying Wang, Ying Liu, Huimin Cui, Xiaobing Feng, and Jingling Xue. 2025. VEGA: Automatically Generating Compiler Backends using a Pre-trained Transformer Model. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (Las Vegas, NV, USA) (CGO '25)*. Association for Computing Machinery, New York, NY, USA, 90–106. doi:[10.1145/3696443.3708931](https://doi.org/10.1145/3696443.3708931)
- [103] Gaofeng Zhou, Jianyang Zhou, and Haijun Lin. 2018. Research on NVIDIA Deep Learning Accelerator. In *2018 12th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID)*. 192–195. doi:[10.1109/ICASID.2018.8693202](https://doi.org/10.1109/ICASID.2018.8693202)
- [104] Junkang Zhu, Minsik Kim, Cheng-Hsun Lu, Wei Tang, Tianyu Wei, and Zhengya Zhang. 2025. EVA: A 16mm² 1.54TFLOPS Tiled-Based Accelerator for Evolvable Edge Computing. In *2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 1–3. doi:[10.23919/VLSITechnologyandCir65189.2025.11075176](https://doi.org/10.23919/VLSITechnologyandCir65189.2025.11075176)
- [105] Kahfi S. Zulkifli, Wenbo Qian, Shaowei Zhu, Yuan Zhou, Zhen Zhang, and Chang Lou. 2025. Verifying Semantic Equivalence of Large Models with Equality Saturation. In *Proceedings of the 5th Workshop on Machine Learning and Systems (World Trade Center, Rotterdam, Netherlands) (EuroMLSys '25)*. Association for Computing Machinery, New York, NY, USA, 82–89. doi:[10.1145/3721146.3721943](https://doi.org/10.1145/3721146.3721943)

Appendix

This appendix collects the material deferred from the paper: the detailed problem formalization, an expanded account of the six compilation modules, and the soundness and completeness proofs (Appendices A–C); visualizations of the tensor operators (Appendix D); and the per-accelerator data behind the evaluation of §8 (Appendices E–H). Each section restates the definitions and running examples it relies on, and pointers such as §5.1 or Fig. 21 refer to the paper above.

CONTENTS

A	Detailed Problem Formulation	34
A.1	Tensor Accelerator ISA Description ISA^H	34
A.2	Data Model D^H	34
A.3	Instruction Set Θ^H	35
A.4	Semantic Equivalence of a Tiled Kernel and Compiled Assembly Code	37
A.5	Compiler Backend Generator	38
B	Detailed Discussion of the Solution	41
B.1	Analogy to Classical Compiler Component Generators	41
B.2	Module 1: E-Graph Initializer	41
B.3	Module 2: Rewrite Applier	42
B.4	Module 3: Graph Extractor	43
B.5	Module 4: Topological Sort	43
B.6	Module 5: Constraint Satisfaction Problem Generator	45
B.7	Module 6: Code Emitter	45
C	Theoretical Guarantees of ACT’s ISA-parameterized Compilation Algorithm	46
C.1	Notation and Standing Assumptions	46
C.2	The Intermediary π Graph	46
C.3	Theoretical Guarantees of Phase 1	47
C.4	Theoretical Guarantees of Phase 2	48
C.5	Proving Soundness and Completeness	49
C.6	Visualization of the Relevant Rewrite Rules	49
D	Visualization of Tensor Operators	51
D.1	Addressing Operators	51
D.2	Structural Operators	52
D.3	Computational Operators	53
E	NNSmith Kernel Generation Setup	57
E.1	Operator Set	57
E.2	Generation Parameters and Resulting Kernel Sizes	57
F	Loop Fission Factors for the Exo Kernels	59
F.1	Reading the Tables	59
F.2	Candidate Tables	59
G	Detailed Analysis of ACT-Gemmini versus the Gemmini SW Library	63
G.1	“ <i>composite</i> ” Kernels	63
G.2	“ <i>not supported</i> ” Kernels	64
G.3	CISC-type Instructions	65
G.4	Convolution Kernels	65
H	The oneDNN Kernels Compiled by ACT-AMX	66
H.1	Kernel Selection and Statistics	66
H.2	K1: <code>cnn_inf_amx</code>	66
H.3	K2: <code>rnn_inf_amx</code>	68
H.4	K3: <code>mem_format_avx</code>	68
H.5	K4: <code>sgemm_avx</code>	70
H.6	K5: <code>cnn_inf_mix</code>	71

A Detailed Problem Formulation

This section formalizes the problem statement of a compiler backend generator that produces sound and complete compiler backends from an accelerator ISA description alone.

We proceed in three steps. First, we formalize the tensor accelerator ISA description (§A.1–§A.3) written in TAIDL, using existing formal models of tensors and tensor operators [6, 52, 61, 98]. The formalism is driven by TAIDL’s key observation that tensor accelerators support coarse-grained instructions operating on multi-dimensional data, commonly represented as tensors. Second, we define the equivalence condition (§A.4) between a tiled kernel and an accelerator-specific assembly code. Third, we use that equivalence to define soundness and completeness of a compiler backend, completing the problem formulation (§A.5) of a compiler backend generator that addresses the challenges and goals of §1.2.

Throughout, the object a compiler backend consumes is a *tiled kernel*: the single-tile version of a fused sub-graph produced by the XLA middle-end, syntactically an ordinary tensor computation graph except that its tensor shapes are those of one tile rather than of the whole operand.

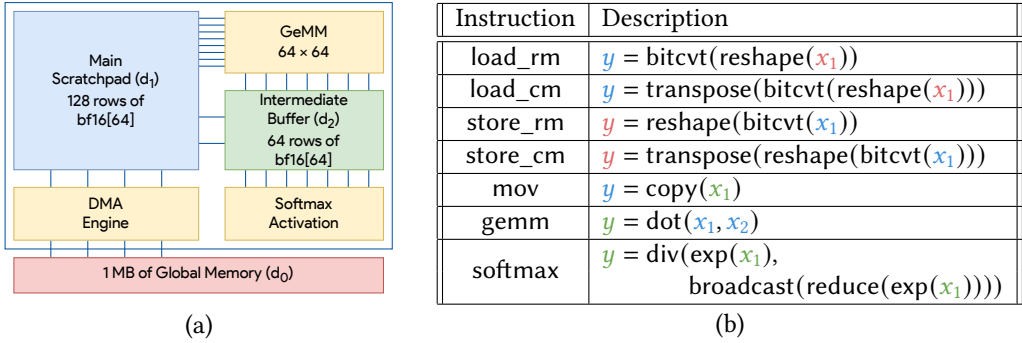


Fig. 24. Running example. (a) High-level accelerator design for a hypothetical accelerator H_{QKV} . (b) Brief description of its instruction set $\Theta^{H_{QKV}}$. Tensor variables are colored based on their storage unit (d_0, d_1, d_2).

A.1 Tensor Accelerator ISA Description ISA^H

An ISA description comprises (1) the software-managed storage units, collectively termed the *data model*, such as scratchpads; and (2) the semantics of instructions that perform computations, such as data movement and compute instructions, on those storage units.

Running example. Fig. 24 reproduces the running example of §3.2 (Fig. 5) alongside the formalization that follows. H_{QKV} is a hypothetical accelerator with two on-chip storage units — a 16 KB scratchpad and an 8 KB intermediate buffer — and access to global memory through a DMA engine. Its two compute units are a general matrix multiply (gemm) unit and a softmax activation unit. It supports five data movement instructions and two compute instructions, described in Fig. 24 (b).

A.2 Data Model D^H

The software-managed storage units, like the scratchpads on an accelerator, are abstractly represented as *tensor buffers*. A tensor buffer is a tensor with a subset of its dimensions representing *access dimensions* and the remaining dimensions representing the granularity of data access. Accelerator instructions modify sub-tensors of these tensor buffers, called *data slices*. We assume that an accelerator has a byte-addressable 1-D global memory (HBM) accessible by the host processor, denoted d_0 . We now define these terms using tensor notation. Definitions are numbered in one

series throughout this section; the boxed ones fix the *objects* an ISA description denotes, and the unboxed ones that follow in §A.4 onwards define functions and relations over those objects.

Def A.1: Tensor Buffer d

A **tensor buffer** d is a tensor of type $\mathbb{E}[S_0, S_1]$ in which the first $\dim(S_0)$ dimensions are *access dimensions* (S_0) and the remaining dimensions are the *granularity of data access* ($\mathbb{E}[S_1]$). Equivalently, d is a multi-dimensional storage unit of shape S_0 whose elements are tensors of uniform type $\mathbb{E}[S_1]$; we write its type as $(S_0, \mathbb{E}[S_1])$ when the access structure is what matters.

Def A.2: Data Slice $d[I_s:I_e]$

A **data slice** $d[I_s:I_e]$ of tensor buffer d is a tensor of type $\mathbb{E}[(I_e - I_s), S_1]$ where $I_s, I_e \in \mathbb{N}_0^{|S_0|}$ and $I_s < I_e \leq S_0$. I_s and I_e are the start (inclusive) and end (exclusive) addresses of the data slice under zero-based multi-dimensional addressing.

Def A.3: Memory State M

A **memory state** $M = (d_0, d_1, \dots)$ is a tuple of tensors, one for each tensor buffer.

Tensor buffer representations for the software-managed storage units of H_{QKV} are annotated in Fig. 24. The 16 KB scratchpad (d_1) consists of 128 rows of 64-length bf16 vectors, and instructions read or modify it at row granularity. It is therefore represented as $d_1 = ([128], \text{bf16}[64])$.

A.3 Instruction Set Θ^H

The instruction set Θ^H is a set of *abstract instructions* that modify the tensor buffers in D^H and are parameterized by instruction attributes. This is analogous to x86 instructions such as mov and add, which modify registers or memory and are parameterized by immediate values, register names, and memory addresses. A *concrete instruction* is an instantiation of an abstract instruction with specific values for those attributes, and is a *deterministic* modification of the tensor buffers. Fig. 24 (b) briefly describes the instruction set $\Theta^{H_{QKV}}$ of H_{QKV} .

Consider the abstract instruction `load_rm` in $\Theta^{H_{QKV}}$, parameterized by three attributes: $n \in [1, 128]$, $\text{addr}_{\text{in}} \in [0, 2^{20})$, and $\text{addr}_{\text{out}} \in [0, 128]$. It is a data movement instruction that loads n rows of data ($n \times 128$ bytes) from HBM (d_0) starting at address addr_{in} , into the main scratchpad (d_1) starting at row addr_{out} . The concrete instruction `load_rm`($n = 4$, $\text{addr}_{\text{in}} = 0$, $\text{addr}_{\text{out}} = 2$) reads 512 bytes from data slice $d_0[0:512]$ of tensor-type `u8[512]` into data slice $d_1[2:6]$ of tensor-type `bf16[4, 64]`. Each bf16 value (2 bytes) is a bitwise cast of 2 contiguous u8 values. This transformation is a concrete tensor computation from `u8[512]` to `bf16[4, 64]` built from the tensor operators of Table 5.

More generally, the semantics of `load_rm` can be written as an abstract tensor computation $y = \text{bitcvt}(\text{reshape}(x_1))$ whose input and output tensors are data slices of d_0 and d_1 respectively. The addresses of these data slices are a function of the attributes addr_{in} and addr_{out} , while the number of rows loaded is a function of the attribute n . This observation is what ACT builds on: we *extend* TAIDL descriptions by classifying the instruction attributes into two sets, $\alpha = \{n\}$ and $\beta = \{\text{addr}_{\text{in}}, \text{addr}_{\text{out}}\}$, termed *computational* and *addressing* attributes respectively. The computational

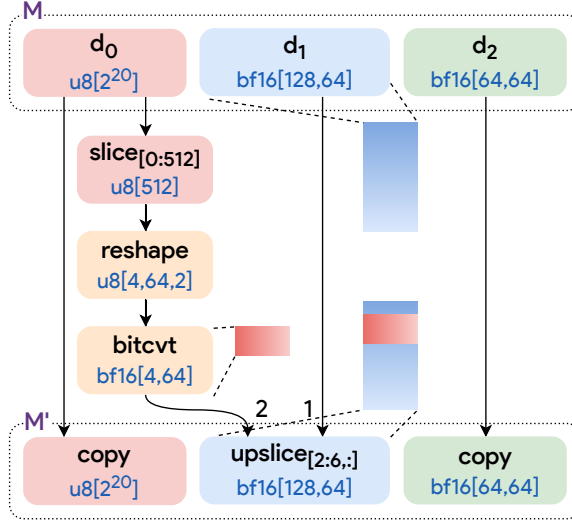


Fig. 25. Execution of the concrete instruction `load_rm(n = 4, addrin = 0, addrout = 2)` as a concrete tensor computation graph.

attributes determine the core tensor computation; every remaining attribute is an addressing attribute. This split is not present in TAIDL itself, and it is what makes the two phases of ACT's algorithm separable — Phase 1 reasons about α and Phase 2 about β .

Generalized representation for instruction set semantics. An abstract instruction is parameterized by two sets of integer attributes α and β . Its semantics are represented as (1) an abstract tensor computation concretized by the set of *computational attributes* α , where (2) the data slices for the input and output tensors of that computation lie on the tensor buffers, and their addresses are parameterized by the set of *addressing attributes* β .

Kernel programming languages like Exo [45] model custom hardware instructions under similar assumptions, albeit defining semantics with scalar operators and Python-like syntax. Intuitively, a scalar pseudo-code description with FOR and IF statements, as used by the Intel Intrinsics Guide [47], can be represented with element-wise tensor operators — scalars being 0-D tensors — together with the `while`³ and `select`⁴ operators.

We now formalize this generalized representation using the notation defined above.

Def A.4: Abstract Instruction θ

An **abstract instruction** $\theta \in \Theta^H$, which reads n_θ tensors located in buffers $d_\theta^{(i)}|_{i=1}^{n_\theta}$ and modifies a tensor located in buffer $d_\theta^{(0)}$ (by convention, superscript (0) denotes the output and (1), ..., (n_θ) the inputs), is represented using an abstract tensor computation g_θ concretized by the set of computational attributes α , together with $(n_\theta + 1)$ *data slice address maps* $h_\theta^{(i)}|_{i=0}^{n_\theta}$ over the set of addressing attributes β , and a *validity constraint* e_θ over the attributes α and β .

³https://openxla.org/xla/operation_semantics#while

⁴https://openxla.org/xla/operation_semantics#select

Def A.5: Concrete Instruction $\theta_{\alpha,\beta}$

A **concrete instruction** $\theta_{\alpha,\beta}$ represents a valid modification if $e_\theta(\alpha, \beta)$ is true. It reads data slices $x_i = d_\theta^{(i)}[I_s^{(i)}:I_e^{(i)}]_{i=1}^{n_\theta}$ and modifies data slice $y = d_\theta^{(0)}[I_s^{(0)}:I_e^{(0)}]$ with the corresponding concrete tensor computation $y = f(x_i)_{i=1}^{n_\theta}$, where $f = g_\theta(\alpha)$ and $\forall i \in [0, n_\theta] (I_s^{(i)}, I_e^{(i)}) = h_\theta^{(i)}(\beta)$.

Def A.6: Execution of a Concrete Instruction $\mathcal{E}(\theta_{\alpha,\beta})$

Execution $\mathcal{E}(\theta_{\alpha,\beta})$ of a concrete instruction $\theta_{\alpha,\beta}$ is a concrete tensor computation over a tuple of tensors that updates the memory state $M = (d_0, d_1, \dots)$ to $M' = (d'_0, d'_1, \dots)$, where

$$d'_i = \begin{cases} \text{upslice}_{[h_\theta^{(0)}(\beta)]} \left(d_\theta^{(0)}, g_\theta(\alpha) \left(\text{slice}_{[h_\theta^{(i)}(\beta)]} (d_\theta^{(i)})_{i=1}^{n_\theta} \right) \right) & \text{for } d_i = d_\theta^{(0)} \\ \text{copy}(d_i) & \text{for } d_i \neq d_\theta^{(0)} \end{cases}$$

Here `upslice`, `slice`, and `copy` are the tensor operators of Table 5, visualized in Appendix D. Intuitively, execution slices each input operand out of its buffer at the addressed range, applies the concretized computation $g_\theta(\alpha)$ to produce the output sub-tensor, and upslices that back into the output buffer $d_\theta^{(0)}$ at its addressed range; every other buffer is copied unchanged.

Revisiting the concrete instruction `load_rm` ($n = 4, \text{addr}_{\text{in}} = 0, \text{addr}_{\text{out}} = 2$), which loads 512 bytes from $d_0[0:512]$ to $d_1[2:6]$: Fig. 25 visualizes its execution $\mathcal{E}$, where the orange nodes represent $g_{\text{load_rm}}(n = 4)$ with input and output tensor-types `u8[512]` and `bf16[4, 64]` respectively. The data slice addresses for the input and output tensors, that is the operator attributes of `slice` and `upslice`, are $h_{\text{load_rm}}^{(1)}(\text{addr}_{\text{in}}=0, \text{addr}_{\text{out}}=2) = (0, 512)$ and $h_{\text{load_rm}}^{(0)}(\text{addr}_{\text{in}}=0, \text{addr}_{\text{out}}=2) = (2, 6)$ respectively.

A.4 Semantic Equivalence of a Tiled Kernel and Compiled Assembly Code

An assembly code ASM^H for a tensor accelerator H consists of a stream of concrete instructions, defined in the ISA description ISA^H , together with a constant tensor. This is analogous to x86 assembly code with its code and data sections.

Semantic equivalence of a compiled assembly code and a tiled kernel G is defined over the output produced by sequentially executing the concrete instructions, compared against the golden output $G(X)$ computed by G for given input tensors X . Since the multi-dimensional input and output tensors of G are stored in HBM, which is a 1-D byte-addressable memory, we first define a byte encoding for tensors.

Def A.7 (bflat). The *byte-flattening* operator `bflat` converts a tensor t of tensor-type $\mathbb{E}[S]$ into a 1-D tensor `bflat`(t) of type `u8[mem( $t$ )]`, where $\text{mem}(t) = \text{mem}(\mathbb{E}) \times \prod S$ and $\text{mem}(\mathbb{E})$ is the memory size in bytes of the scalar basetype $\mathbb{E}$.

`bflat` names what §5.1 calls the 1-D byte representation of a tensor: for a `bf16` tensor it is a `bitcv` of each `bf16` into two `u8` values followed by a `reshape`.

Because equivalence is stated modulo rewrites, we also need the underlying relation on tensor computation graphs.

Def A.8 ($\equiv_{\mathcal{R}}$). Tensor computation graphs G_1 and G_2 are **semantically equivalent modulo** $\mathcal{R}$, written $G_1 \equiv_{\mathcal{R}} G_2$, if G_1 can be transformed into G_2 by applying rewrite rules in $\mathcal{R}$. We take $\mathcal{R}$ to be the XLA compiler's curated set of 175+ tensor rewrites, treated as the foundational axioms of

the XLA-HLO IR. Following prior work [52, 97, 101, 105], we assume the axioms in $\mathcal{R}$ are closed under inversion and composition, so that $\equiv_{\mathcal{R}}$ is reflexive, symmetric and transitive.

The closure assumption in Def. A.8 is what licenses chaining equivalences, which the proofs of Appendix C rely on.

Intuitively, semantic equivalence is a sequence of two events: (1) initialization of HBM with the byte-flattened input tensors and the compile-time constant tensor of the assembly code, and (2) sequential execution of the concrete instructions of the assembly code according to the ISA description. The compiled assembly code is semantically equivalent to the input tiled kernel G if, for all input tensors X , HBM ends up holding the byte-flattened input tensors X followed by the output tensor $G(X)$. We formalize this as a transformation Λ_{LHS} of ASM_G^H and the expected HBM state as a transformation Λ_{RHS} of G .

Fix $\text{mem}_{in} = \text{mem}(X)$ and $\text{mem}_{out} = \text{mem}(X) + \text{mem}(G(X))$. The initial HBM layout places the byte-flattened inputs at offset 0, reserves the region $[\text{mem}_{in}, \text{mem}_{out})$ for the output, and places the constant tensor at offset mem_{out} , so the constant never overlaps the region that equivalence is checked over.

Def A.9 (Λ_{LHS}). Given an assembly code $ASM_G^H = ((\theta_i(\alpha_i, \beta_i))_{i=1}^N, \text{const})$, $\Lambda_{LHS}(ASM_G^H)$ is a concrete tensor computation over a memory state $M = (d_0, d_1, \dots)$ and input tensors X . The memory state after initialization of HBM is

$$M_0 = (\text{upslice}_{[\text{mem}_{out}:\text{mem}_{out}+\text{mem}(\text{const})]}(\text{upslice}_{[0:\text{mem}_{in}]}(d_0, \text{bflat}(X)), \text{bflat}(\text{const})), d_1, \dots).$$

The final memory state is

$$M_N = \mathcal{E}(\theta_N(\alpha_N, \beta_N)) \circ \dots \circ \mathcal{E}(\theta_1(\alpha_1, \beta_1))(M_0),$$

of which only the first mem_{out} bytes of HBM, written $M_N[0]$, are relevant for equivalence. Finally,

$$\Lambda_{LHS}(ASM_G^H)(M, X) = \text{slice}_{[0:\text{mem}_{out}]}(M_N[0]).$$

Def A.10 (Λ_{RHS}). Given an input tiled kernel G ,

$$\Lambda_{RHS}(G)(M, X) = \text{concat}_1(\text{bflat}(X), \text{bflat}(G(X))).$$

Both Λ_{LHS} and Λ_{RHS} map a memory state and input tensors to a single $\text{u8}[\text{mem}_{out}]$ tensor, so they can be compared directly. We now define semantic equivalence using these bisimulation transformations.

Def A.11 ($\equiv_{\mathcal{R}}^H$). Given the foundational axioms $\mathcal{R}$ of the IR and the ISA description ISA^H , the semantic equivalence $\equiv_{\mathcal{R}}^H$ is defined as

$$ASM_G^H \equiv_{\mathcal{R}}^H G \iff \forall M, \forall X, \Lambda_{LHS}(ASM_G^H)(M, X) \equiv_{\mathcal{R}} \Lambda_{RHS}(G)(M, X),$$

where $\equiv_{\mathcal{R}}$ is the relation of Def. A.8. Λ_{LHS} and Λ_{RHS} are visualized in Fig. 26.

Since $\equiv_{\mathcal{R}}$ is an equivalence relation (Def. A.8) and Def. A.11 quantifies over all M and X , $\equiv_{\mathcal{R}}^H$ inherits reflexivity, symmetry and transitivity.

A.5 Compiler Backend Generator

We can now define the soundness and completeness of a compiler backend in terms of the formalism developed above. A compiler backend Compiler^H consumes a tiled kernel G and may (i) emit an assembly code, (ii) return FAIL, denoting compilation failure, or (iii) not terminate, denoted $\perp$.

Def A.12 (Soundness). A compiler backend Compiler^H is **sound** iff

$$\forall G, \text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\} \implies \text{Compiler}^H(G) \equiv_{\mathcal{R}}^H G$$

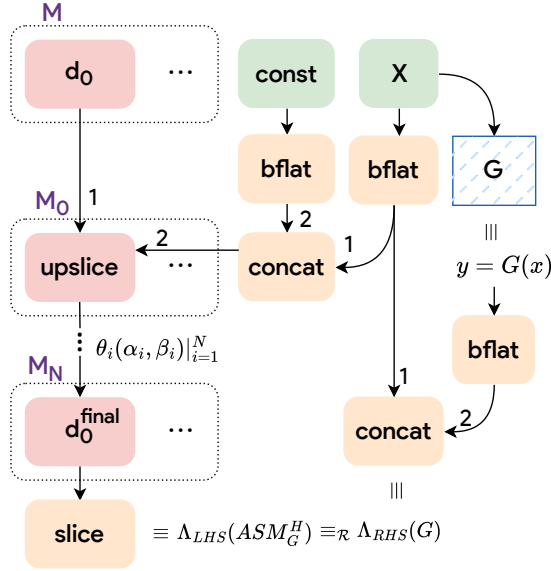


Fig. 26. Visualization of $\equiv_{\mathcal{R}}^H$. M and X are the inputs to the tensor computations. The leaf nodes represent the LHS and RHS of the semantic equivalence.

Def A.13 (Completeness). A compiler backend Compiler^H is **complete** iff

$$\forall G, \exists ASM_G^H \equiv_{\mathcal{R}}^H G \implies \text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\}$$

Discussion: semi-decidability versus totality. Soundness and completeness as defined above do not require the compiler backend to be a total function: it may fail to terminate on tiled kernels that have no semantically equivalent assembly code. This is a deliberate design choice, because requiring all three properties at once is impossible.

THEOREM A.1. *There exists no compiler backend generator CompGen such that for every tensor accelerator ISA description ISA^H in TAIDL, the generated backend $\text{CompGen}(\text{ISA}^H)$ is sound, complete, and total.*

PROOF. Assume for contradiction that such a generator CompGen exists. Then for every ISA description ISA^H , the backend $\text{Compiler}^H = \text{CompGen}(\text{ISA}^H)$ is sound (Def. A.12), complete (Def. A.13), and total, meaning it always terminates and so never returns $\perp$.

Let M be an arbitrary Turing machine and w an arbitrary input. Construct an ISA description $\text{ISA}^{H(M,w)}$ with one abstract instruction θ parameterized by one computational attribute $\alpha = k \in \mathbb{N}$. Define each concrete instantiation $\theta(k)$ by bounded simulation:

- (1) Simulate M on w for exactly k steps.
- (2) If M halts within k steps, $\theta(k)$ computes the identity on a unit tensor.
- (3) Otherwise, $\theta(k)$ computes the constant-zero function.

Each $\theta(k)$ has finite, computable semantics, and this encoding is expressible in TAIDL [51].

Let G be the tiled kernel computing the identity on a unit tensor.

If M halts on w , there exists k^* such that the one-instruction assembly $\text{ASM} = (\theta(k^*), \emptyset)$ satisfies $\text{ASM} \equiv_{\mathcal{R}}^H G$. By completeness, $\text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\}$, so it compiles successfully.

If M does not halt on w , then for all k the instruction $\theta(k)$ computes the constant-zero function, which is not equivalent under $\mathcal{R}$ to the identity computed by G ; hence no assembly built from these

instructions is equivalent to G . By soundness, $\text{Compiler}^H(G)$ cannot return a successful assembly, and by totality it cannot return $\perp$; therefore it returns FAIL.

Hence we can decide whether M halts on w by running $\text{Compiler}^H(G)$ and checking whether the result is a successful compilation or FAIL. This decides the halting problem, a contradiction. Therefore no sound, complete, and total generator exists. $\square$

In practice, semi-decidability is not a concern, since a timeout can always be imposed on the compiler backend and treated as a compilation failure. We therefore focus on soundness and completeness without requiring totality. §8.7 reports that our generated compiler backends have reasonable compilation times and that we encountered no non-termination in practice.

B Detailed Discussion of the Solution

This section expands on the six modules of ACT’s compilation algorithm in the places where the presentation above had to be brief. It is organized to mirror §5 and §6: one subsection per module, in the order the modules run. We begin with an orientation that is easy to lose sight of in the module-by-module presentation, namely what kind of artifact ACT actually is.

B.1 Analogy to Classical Compiler Component Generators

ACT is a *generator*, not a compiler, and the distinction is the source of most of its design. The closest familiar analogues are Flex [30] and Bison [79], which are also generators: each takes a declarative description of an artifact, instantiates a fixed parameterized algorithm against that description, and emits a working component. Table 4 lays the three side by side.

Reading the table row by row makes the correspondence concrete. The *core parameterized algorithm* is the piece of theory that is fixed once and for all and never regenerated: NFA-to-DFA construction for Flex, LALR(1) table generation for Bison, and ACT’s ISA-parameterized compilation algorithm. The *generic skeleton* is the code that is identical across every generated instance — a table-driven scanning loop, a shift-reduce parser loop, and in our case a compiler backend whose accelerator-specific decisions are left as pure virtual functions. The *specialized templates* are what the generator actually writes: transition tables, ACTION/GOTO tables, and for ACT the rewrite rules and the concrete implementations that override those virtual functions. This is why ACT needs no accelerator-specific algorithmic work: as with Flex and Bison, all the accelerator-specific content lives in the specialized templates, and the algorithm above them never changes.

	Flex [30]	Bison [79]	ACT (Our work)
User-provided input	Regular expressions	Context-free grammar	ISA description
Generated output	Lexer/Scanner	LALR(1) Parser	Compiler backend
Core Parameterized Algorithm (Theory)	NFA-to-DFA construction [3]	LALR(1) table generation [3]	ISA-parameterized compilation algorithm
Generic Skeleton	Generic table-driven scanning loop	Generic shift-reduce parser loop	Generic backend with pure virtual functions
Specialized Templates	DFA transition table with accept states	ACTION/GOTO tables with lookahead sets	Rewrites & overridden virtual functions
Specialization Approach	GNU M4 macro processor	GNU M4 macro processor	Python-based regex processor

Table 4. Template-driven compiler component generators based on a parameterized algorithm. The core parameterized algorithm and the generic skeleton are fixed once, when the generator itself is built; the specialized templates are what the generator writes anew for each user-provided input, using the specialization approach of the last row.

B.2 Module 1: E-Graph_INITIALIZER

§5.1 describes Module 1 as lowering the tiled kernel into an initial e-graph, without detailing how XLA-HLO’s tiled memory formats are represented. We expand on that here, since tiled layouts are what make ACT-generated backends drop-in compatible with the XLA compiler.

The XLA compiler often stores tensors in a tiled memory format,⁵ written $\mathbb{E}[S_t]\{T(S_T)\}$, in which a tensor of type $\mathbb{E}[S_t]$ is broken into tiles of shape S_T that are each stored contiguously. Fig. 27 (a)

⁵https://openxla.org/xla/tiled_layout

shows the XLA-HLO IR for a tiled kernel commonly observed in oneDNN examples: a matrix multiplication whose input tensor B and output tensor both carry tiled layouts. ACT models such a layout as a composition of reshape and transpose operators together with the byte-flattening operator `bflat` (Def. A.7), so that no new operator or IR extension is required. Fig. 27 (b) shows this modeling for the layout `i32[32,32]{T(16,16)}`: the `[32, 32]` tensor is split into `[2, 16, 2, 16]`, the tile axes are rearranged into contiguous order by a transpose, and `bflat` then yields the flat byte sequence actually held in memory. Because the layout is expressed in the same operator vocabulary as the rest of the graph, equality saturation can rewrite across it: ACT-AMX applied 22 IR-to-IR rewrites over 6 iterations to compile this kernel into the assembly of Fig. 27 (c).

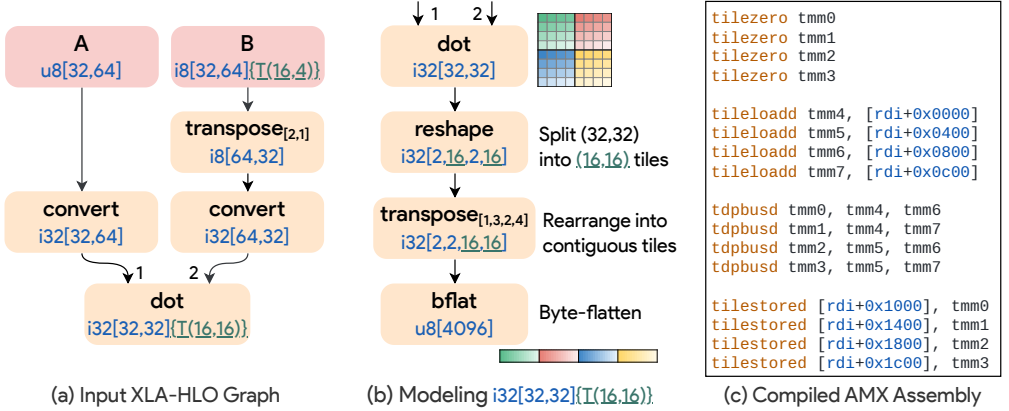


Fig. 27. Modeling a tiled memory layout in Module 1. (a) The input XLA-HLO IR, with tiled layouts on the input tensor B and on the output tensor. (b) The layout `i32[32,32]{T(16,16)}` as modeled by ACT, using reshape and transpose to gather each tile into contiguous order and `bflat` to flatten it to bytes. (c) The final assembly code generated by ACT-AMX.

B.3 Module 2: Rewrite Applier

Module 2 supplies the rewrites used during the exploration stage. Two implementation details are worth recording beyond that description.

Accelerator-specific rewrite filtering. The XLA compiler’s 175+ IR-to-IR rewrites are not specific to any accelerator, and applying all of them wastes exploration budget on operators the target cannot express. We therefore filter the rewrite set against the target ISA before saturation begins. A rewrite is retained only if it can eventually contribute to an IR-to-ISA rewrite: starting from the operators that appear in some abstract computation g_θ , we add any rewrite that produces such an operator, then add the operators that rewrite consumes, and iterate. For example, if operator C appears in some g_θ , then a rewrite producing C is added to the set; if that set now contains a rewrite $B \rightarrow C$, operator B is added in turn, and the process repeats until a fixed point. Iterating rather than taking a single pass is what makes the filter handle transitive rewrite chains. The result is that the operator set potentially supported by an accelerator is pruned at backend-generation time rather than at compile time.

Identity instructions. Beyond their role in completeness, the identity instructions `sliceH` and `concatH` are what let ACT tile an inner loop and pad tensor variables without any dedicated machinery: both are expressible as no-ops on the memory state, so the e-graph can introduce

them freely wherever a shape needs adjusting. Fig. 28 shows a small pii graph generated by ACT-Gemmini in which exactly this happens: two $[32, 16]$ operands are each split by a pair of slice^H nodes into $[16, 16]$ halves, fed through two matmul instructions, and the results reassembled by a concat^H . Appendix C proves that the availability of these two instructions is what makes Phase 2's reconstruction of a pii graph from an assembly code possible, and with it the completeness of the algorithm as a whole.

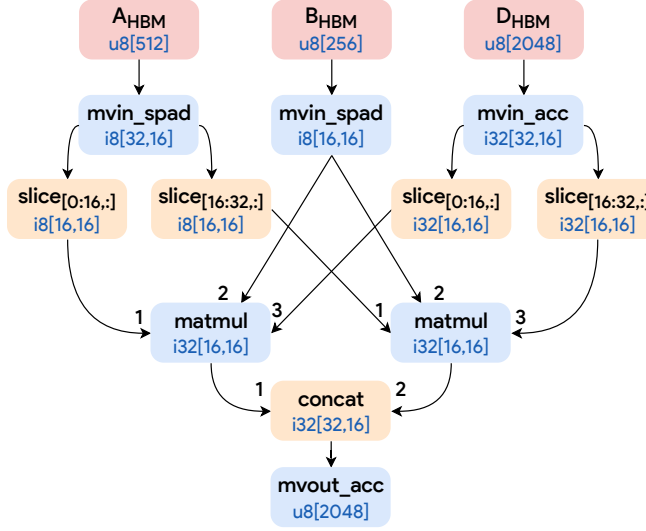


Fig. 28. A pii graph generated by ACT-Gemmini that uses the identity instructions slice^H and concat^H to split an operand across two matmul instructions and reassemble the results.

B.4 Module 3: Graph Extractor

Module 3 extracts pii graphs from the saturated e-graph by a top-down traversal. We expand here on its termination condition, which the pseudocode of §5.3 states compactly.

The traversal terminates at an e-class when it is expecting a tensor in d_0 (HBM) and that e-class either is marked as constant, or contains an e-node representing a flattened input tensor variable. In the second case the flattened input tensor is added directly to the pii graph, since it is already available in HBM and needs no instruction to produce it. In the first case the tensor expression that computes the constant value is added to the pii graph instead, so that the constant is materialized by the accelerator rather than fetched from the host. Constant e-classes are detected by a bottom-up traversal, with the base case appearing in the extraction pseudocode of §5.3. The two cases are mutually exclusive and can never both apply to the same e-class, which is what makes the termination condition well defined.

B.5 Module 4: Topological Sort

Module 4 orders the pii nodes before memory allocation. §6.1 states that we extend Sethi-Ullman numbering to multiple tensor buffers; we give the extension in full here, together with a counterexample showing why the heuristic cannot be relied on alone.

Extending Sethi-Ullman numbering. Classic Sethi-Ullman numbering [3] labels each node of an expression tree with the minimum number of registers needed to evaluate it without spilling:

a leaf needs one register, and a binary node with child labels ℓ_1, ℓ_2 needs $\max(\ell_1, \ell_2)$ registers if $\ell_1 \neq \ell_2$ and $\ell_1 + 1$ otherwise, evaluating the costlier child first. Two things differ in our setting: there are several tensor buffers rather than one register file, and the values occupying them have widely varying sizes. We therefore track occupancy in bytes, per buffer. For a tensor buffer d and a node y with operands x_i , we define

$$\text{esun}_d(y) = \max(\text{mem}_d(y), \min_i \tau_d(x_i)), \quad \tau_d(x_i) = \text{esun}_d(x_i) + \sum_{j \neq i} \text{mem}_d(x_j),$$

where $\text{mem}_d(x_j)$ is the memory size in bytes of the tensor variable represented by node x_j , and is 0 whenever that tensor does not reside on buffer d . For a leaf node y , an input or constant tensor, $\text{esun}_d(y) = \text{mem}_d(y)$. Intuitively $\tau_d(x_i)$ is the peak occupancy of buffer d when operand x_i is evaluated last, after every sibling x_j has already been materialized; the $\min_i$ picks the operand order that minimizes this peak. We compute esun_d for every tensor buffer in D^H except d_0 (HBM), and select operands in decreasing order of $\max_d(\tau_d(x_i)/\text{mem}(d))$, where $\text{mem}(d)$ is the total capacity of buffer d . Normalizing by $\text{mem}(d)$ is what makes pressure on buffers of very different sizes comparable.

Why the heuristic is not sufficient. We chose Sethi-Ullman numbering as the base algorithm because its objective matches ours: to improve the likelihood that the CSP of Module 5 is satisfiable we want the interference graph to be as small as possible, which is analogous to minimizing register pressure. But while Sethi-Ullman numbering is provably optimal for a single register file, it is not optimal for more than one, and Fig. 29 is a counterexample. The graph has two branches that meet at a common sink, and each branch produces values in *both* register files, drawn blue and red — that crossover is what defeats a depth-first order. Any depth-first topological ordering evaluates one subtree fully before starting the other, and is therefore forced to hold three live values in one file, yielding (3, 2) or (2, 3). The ordering $A-I$ instead interleaves the two subtrees and keeps at most two live values in each file, using (2, 2). No depth-first traversal produces this ordering.

This is a genuine limitation rather than an artifact of our extension, and it is precisely why Phase 2 does not rely on the heuristic ordering for completeness but falls back to an exhaustive search over topological orderings (§6.4). In practice the heuristic is nonetheless effective: it guides the search toward low-pressure schedules, and we rarely observe the fallback being needed.

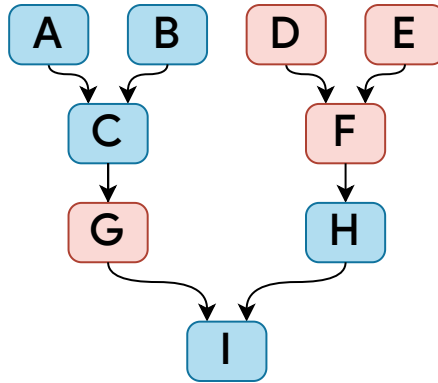


Fig. 29. A counterexample with two register files (blue, red) showing that depth-first topological orderings are not optimal for more than one register file. Every depth-first ordering uses (3, 2) or (2, 3) registers, whereas the interleaved ordering $A-I$ uses only (2, 2).

B.6 Module 5: Constraint Satisfaction Problem Generator

Module 5 encodes memory allocation as a constraint satisfaction problem and discharges it with the Google OR-Tools CP-SAT solver. One practical consequence deserves recording. CP-SAT is multi-threaded, and its search is therefore non-deterministic: the same tiled kernel can compile to different — though equally valid — assembly across runs. Compiler engineers usually expect a backend to be deterministic, so ACT exposes a boolean flag that forces it. When the flag is set, we invoke CP-SAT with `NumSearchWorkers = 1`, which serializes the search and makes the solution reproducible, at some cost in solving time (see the OR-Tools discussion at <https://groups.google.com/g/or-tools-discuss/c/ZGj5vhhZX48>).

B.7 Module 6: Code Emitter

Module 6 is the simplest of the six and needs little expansion: it walks the pii nodes in the order fixed by Module 4 and emits each one with the addressing attributes solved by Module 5. The single point of interest is that identity instructions are *discarded* at this step rather than emitted. This is sound because by the time Module 6 runs, whatever selection or reassembly an identity instruction expressed has already been folded into the data slice addresses that Module 5 solved for its neighbours: the value each consumer reads is fixed by its own addressing attributes, so dropping the intervening `sliceH` or `concatH` changes nothing observable. It is also what keeps their use free: Modules 2 and 3 may introduce as many of them as the search finds convenient without any of them reaching the generated assembly.

C Theoretical Guarantees of ACT's ISA-parameterized Compilation Algorithm

The claims and proofs in this section rest on the formalization of Appendix A and are independent of any implementation detail. Their purpose is to establish a rigorous foundation for the soundness and completeness of ACT's core algorithm for automatically generating compiler backends from tensor accelerator ISA descriptions.

C.1 Notation and Standing Assumptions

The proofs are stated in the vocabulary of the compilation algorithm, which we restate here to fix the notation used throughout.

Phase 1 is built on *equality saturation*. The tiled kernel is loaded into an *e-graph*: a data structure whose nodes (*e-nodes*) are operators and whose *e-classes* are sets of *e-nodes* known to compute the same value. Applying a rewrite does not replace anything — it adds the rewritten form to the same *e-class*, so one *e-graph* represents every graph reachable by the rewrites applied so far. ACT alternates an *exploration* stage, which applies rewrites, with an *extraction* stage, which reads candidate pii graphs back out. Three kinds of rewrite are used (§B.3): the XLA-HLO IR-to-IR axioms, the IR-to-ISA rewrites generated from the ISA description, and the identity-instruction rewrites.

Four further symbols recur throughout.

$\text{Phase}_1, \text{Phase}_2$. The two phases of the compilation algorithm. $\text{Phase}_1(G)$ denotes the sequence of pii graphs enumerated by tensor instruction selection on the tiled kernel G , with $\text{Phase}_1(G)[i]$ its i^{th} element. $\text{Phase}_2(\text{PG}^H)$ denotes the assembly code produced by tensor memory allocation on the pii graph PG^H , or FAIL if the constraint problem is unsatisfiable.

Γ . The pii node budget used by the extraction stage. ACT interleaves exploration and extraction, raising the node limit $N_{\max} = \Gamma(k)$ with the exploration iteration count k , where Γ is strictly monotonically increasing and unbounded (for example $\Gamma(k) = 2^k$).

CSP_4 . The interference constraint of Module 5, $\text{CSP}_4 = \bigwedge_{(i,j) \in \phi} \text{disjoint}(i, j)$, where the interference graph ϕ pairs two nodes exactly when they write the *same* tensor buffer and their live ranges overlap. It forces such a pair to be given non-overlapping addresses within that buffer; nodes on different buffers are never constrained against each other.

We also use freely that $\equiv_{\mathcal{R}}$ is an equivalence relation, which Def. A.8 establishes from the closure of $\mathcal{R}$ under inversion and composition. This is what licenses chaining equivalences in §C.5. Lemma C.2 additionally requires $\equiv_{\mathcal{R}}$ to be a *congruence* — that replacing a sub-graph by an equivalent one leaves the enclosing graph equivalent — which is what makes reasoning *e-class* by *e-class* sound.

We provably guarantee that ACT generates *sound* and *complete* compiler backends:

THEOREM C.1. *For every ISA description ISA^H , the backend $\text{ACT}(\text{ISA}^H)$ is sound (Def. A.12) and complete (Def. A.13).*

We break the proof of Theorem C.1 into five lemmas, combined in §C.5.

C.2 The Intermediary pii Graph

The proofs are organized around an intermediary representation that sits between the tiled kernel and the assembly code: a graph whose nodes are instructions with their computational attributes fixed but their addressing attributes still unknown. Writing the unknown as $?$, we call these *partially instantiated instruction* (pii) nodes. Splitting the problem at exactly this point is what lets Phase 1 reason about α alone and Phase 2 about β alone.

Def C.1. A **pii graph** PG^H is a directed acyclic graph of N pii nodes $\theta_i(\alpha_i, ?)_{i=1}^N$, which can be transformed into a concrete tensor computation by the substitutions $\theta_i(\alpha_i, ?) \rightarrow g_{\theta_i}(\alpha_i)$. A pii node

is either an instruction $\theta_{\alpha,?}$ from the instruction set Θ^H , or one of the identity instructions $\text{slice}_{\alpha,?}^H$ and $\text{concat}_{\alpha,?}^H$.

Def C.2 ($\equiv_{\mathcal{R}}^H$ lifted to pii graphs). Semantic equivalence between a pii graph PG^H and a tiled kernel G is defined as

$$\text{PG}^H \equiv_{\mathcal{R}}^H G \iff \Lambda_{pii}(\text{PG}^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G),$$

where $\Lambda_{pii}(\text{PG}^H)(M, X) = \text{concat}_1(\text{bflat}(X), \text{PG}^H(X))$.

Similarly, $\text{ASM}_G^H \equiv_{\mathcal{R}}^H \text{PG}^H \iff \Lambda_{LHS}(\text{ASM}_G^H) \equiv_{\mathcal{R}} \Lambda_{pii}(\text{PG}^H)$. Here $\text{PG}^H(X)$ denotes the byte-level output the pii graph produces in HBM, so that it has the same tensor-type as $\text{bflat}(G(X))$ in Def. A.10, and Λ_{LHS} , Λ_{RHS} are as defined in §A.4. The memory-state argument M is carried so that Λ_{pii} , Λ_{LHS} and Λ_{RHS} share one input signature. It is inert in Λ_{pii} and Λ_{RHS} , which read only X ; Λ_{LHS} does read it, which is why Def. A.11 quantifies over all M .

Def. C.2 reuses the symbol $\equiv_{\mathcal{R}}^H$ of Def. A.11 on two further pairs of objects. All three readings unfold to $\equiv_{\mathcal{R}}$ on $\text{u8}[\text{mem}_{out}]$ tensors, so they compose, which is what §C.5 relies on.

C.3 Theoretical Guarantees of Phase 1

Phase 1 performs tensor instruction selection. Its two guarantees are that everything it enumerates is correct, and that it enumerates everything.

LEMMA C.2. *Every enumerated pii graph is equivalent to the input tiled kernel G , i.e.,*

$$\forall i, \text{Phase}_1(G)[i] \equiv_{\mathcal{R}}^H G.$$

PROOF. Equality saturation only ever replaces a term by one the rewrite set declares equal, so it suffices that every rewrite used in Module 2 is semantics-preserving. There are three kinds. The IR-to-IR rewrites are a subset of the foundational axioms $\mathcal{R}$, hence valid by definition of $\equiv_{\mathcal{R}}$. The IR-to-ISA rewrites are derived mechanically from the ISA description: each rewrites a sub-graph matching the abstract tensor computation g_{θ} into the corresponding pii node $\theta_{\alpha,?}$, and is therefore valid by Def. A.4. The identity-instruction rewrites are valid because slice^H and concat^H are by construction equal to their tensor-operator counterparts slice and concat . Since each rewrite preserves $\equiv_{\mathcal{R}}$ and $\equiv_{\mathcal{R}}$ is transitive, every e-class of the saturated e-graph contains only terms equivalent to the initial one, and hence every extracted pii graph is equivalent to G . $\square$

LEMMA C.3. *All equivalent pii graphs are enumerated, i.e.,*

$$\forall \text{PG}^H \equiv_{\mathcal{R}}^H G, \exists i \text{ s.t. } \text{Phase}_1(G)[i] = \text{PG}^H.$$

PROOF. Let $\text{PG}^H \equiv_{\mathcal{R}}^H G$ have N_0 nodes. By Def. C.2 this means $\Lambda_{pii}(\text{PG}^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G)$, so the two are joined by a finite sequence of rewrites; since Λ_{pii} and Λ_{RHS} differ from PG^H and G only by the common $\text{concat}_1(\text{bflat}(X), \cdot)$ wrapper, the same sequence joins PG^H to G . Every rewrite in it is drawn from the three kinds Module 2 supplies, and the exploration stage applies every applicable rewrite at each iteration, so there is a finite k_0 after which PG^H is represented in the e-graph. Because Γ is strictly monotonically increasing and unbounded, the set $\{k : N_0 \leq \Gamma(k)\}$ is non-empty; let $k_1 = \min\{k : N_0 \leq \Gamma(k)\}$ be its least element. At iteration $k' = \max(k_0, k_1)$ the graph is both present in the e-graph and within the extraction node budget, so it is among the graphs extracted at k' . $\square$

C.4 Theoretical Guarantees of Phase 2

Phase 2 performs tensor memory allocation. It contributes one soundness lemma, one completeness lemma, and one structural lemma relating assembly codes back to pii graphs.

LEMMA C.4. *Whenever Phase 2 succeeds, the assembly code it emits is equivalent to the input pii graph, i.e.,*

$$\text{Phase}_2(\text{PG}^H) \neq \text{FAIL} \implies \text{Phase}_2(\text{PG}^H) \equiv_{\mathcal{R}}^H \text{PG}^H.$$

PROOF. Write $\text{ASM}_G^H = \text{Phase}_2(\text{PG}^H)$ and transform $\Lambda_{LHS}(\text{ASM}_G^H)$ into $\Lambda_{pii}(\text{PG}^H)$. By Def. A.6 each executed instruction contributes an upsplice of its result into its output buffer, and each later instruction that consumes a value contributes a slice of some buffer. Working backwards from the last instruction, every slice-of-upsplice pair that arises falls into exactly one of two cases.

The read is the consumer of that write. Module 5's def-use constraint assigns the consumer the same data slice addresses the producer was given, so the pair is $\text{slice}_{[s:e]}(\text{upslice}_{[s:e]}(M, X))$, which reduces to X : the consumer receives precisely the tensor the producer computed, and the intermediate buffer write disappears while the value it carried is retained.

The read belongs to some other value. Then the two data slices have overlapping live ranges, so the interference constraint CSP_4 has assigned them non-overlapping addresses within their common buffer, discharging the precondition of

$$\text{slice}_{[s_1:e_1]}(\text{upslice}_{[s_2:e_2]}(M, X)) \rightarrow \text{slice}_{[s_1:e_1]}(M) \quad \text{under} \quad \text{disjoint}([s_1:e_1], [s_2:e_2]),$$

which steps the read back past a write it cannot observe.

Alternating the two cases eliminates every intermediate buffer write while preserving each consumed value, leaving exactly the composition of g_θ sub-graphs that constitutes $\Lambda_{pii}(\text{PG}^H)$. Both rewrites preserve $\equiv_{\mathcal{R}}$, so the whole transformation does. The second has been verified in full generality using TensorRight [6], and Fig. 30 visualizes it. $\square$

LEMMA C.5. *Phase 2 does not fail if an equivalent assembly code exists, subject to the completeness of the CP-SAT solver, i.e.,*

$$\exists \text{ASM}_G^H \equiv_{\mathcal{R}}^H \text{PG}^H \implies \text{Phase}_2(\text{PG}^H) \neq \text{FAIL}.$$

PROOF. Phase 2 exhaustively searches topological orderings, using interference-graph pruning (§6.4) which discards only orderings whose interference graph is a supergraph of one already shown unsatisfiable, and so preserves completeness.⁶ The search therefore covers the ordering in which the instructions of ASM_G^H appear. For that ordering, the data slice addresses already present in ASM_G^H constitute a satisfying assignment to the constraint problem, so the CSP is satisfiable and Phase 2 does not fail. The qualification on CP-SAT is necessary because ACT delegates satisfiability to an external solver; the argument shows a model exists, not that the solver finds it. $\square$

LEMMA C.6. *Every assembly code has an equivalent pii graph, i.e.,*

$$\forall \text{ASM}_G^H, \exists \text{PG}^H, \text{ASM}_G^H \equiv_{\mathcal{R}}^H \text{PG}^H.$$

PROOF. We construct the pii graph from $\Lambda_{LHS}(\text{ASM}_G^H)$. By Def. A.6 and Def. A.9, $\Lambda_{LHS}(\text{ASM}_G^H)$ consists of g_θ sub-graphs together with bflat, slice and upsplice nodes. The bflat nodes coincide with those of Λ_{pii} and can be ignored. Each g_θ sub-graph is replaced by the corresponding pii node

⁶This is the direction implied by the monotonicity of CSP_4 under ϕ stated in §6.4, namely $\phi_1 \subseteq \phi_2 \implies \neg \text{CSP}(\phi_1) \rightarrow \neg \text{CSP}(\phi_2)$: a supergraph of an unsatisfiable interference graph is itself unsatisfiable, so pruning supergraphs never discards a satisfiable ordering. Pruning in the opposite direction would not be sound, since a subgraph carries fewer constraints and may be satisfiable.

$\theta_{\alpha,?}$. Each slice node is replaced by a slice^H node, and each upslice node is decomposed into slice^H and concat^H nodes using

$$\text{upslice}_{[s:e]}(M, X) \rightarrow \text{concat}_1(\text{concat}_1(\text{slice}_{[0:s]}(M), X), \text{slice}_{[e:]}(M)).$$

The result satisfies Def. C.1, so it is a pii graph, and each step preserves $\equiv_{\mathcal{R}}$. Note where the identity instructions earn their place: without slice^H and concat^H as admissible pii nodes there would be no way to express the decomposition above, and this lemma — and with it completeness — would fail. The generalized rewrite has been verified using TensorRight [6], and Fig. 31 visualizes it. $\square$

C.5 Proving Soundness and Completeness

We now assemble Theorem C.1 from the lemmas of §C.3 and §C.4.

PROOF OF THEOREM C.1, SOUNDNESS. Suppose $\text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\}$, so the backend emitted an assembly code $\text{ASM}_G^H = \text{Phase}_2(\text{PG}^H)$ for some enumerated $\text{PG}^H = \text{Phase}_1(G)[i]$. Lemmas C.4 and C.2 give the two halves of the chain:

$$\Lambda_{LHS}(\text{ASM}_G^H) \equiv_{\mathcal{R}} \Lambda_{pii}(\text{PG}^H) \quad \text{and} \quad \Lambda_{pii}(\text{PG}^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G).$$

By transitivity of $\equiv_{\mathcal{R}}$ (Def. A.8) it follows that $\Lambda_{LHS}(\text{ASM}_G^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G)$, which is exactly $\text{Compiler}^H(G) \equiv_{\mathcal{R}}^H G$. Hence Compiler^H is sound in the sense of Def. A.12. $\square$

PROOF OF THEOREM C.1, COMPLETENESS. Suppose there exists $\text{ASM}_G^H \equiv_{\mathcal{R}}^H G$. Def. A.13 requires $\text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\}$, so we must exclude both outcomes.

Not FAIL. By Lemma C.6 there is a pii graph PG^H with $\text{ASM}_G^H \equiv_{\mathcal{R}}^H \text{PG}^H$. Unfolding both hypotheses, $\Lambda_{LHS}(\text{ASM}_G^H) \equiv_{\mathcal{R}} \Lambda_{pii}(\text{PG}^H)$ and $\Lambda_{LHS}(\text{ASM}_G^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G)$; by symmetry and transitivity of $\equiv_{\mathcal{R}}$ we get $\Lambda_{pii}(\text{PG}^H) \equiv_{\mathcal{R}} \Lambda_{RHS}(G)$, that is $\text{PG}^H \equiv_{\mathcal{R}}^H G$. By Lemma C.3, Phase 1 enumerates this PG^H at some finite index. By Lemma C.5, Phase 2 does not fail on it. So the backend emits an assembly code rather than returning FAIL.

Not $\perp$. Let k' be the finite iteration at which Lemma C.3 places PG^H among the extracted graphs. Each exploration iteration applies finitely many rewrites to a finite e-graph and each extraction is bounded by the node budget $\Gamma(k)$, so iterations 1 through k' complete in finite time. For each of the finitely many pii graphs enumerated up to that point, Phase 2 searches a finite set of topological orderings and solves a finite constraint problem. The backend therefore reaches PG^H and succeeds on it after finitely many steps, and so terminates.

Hence $\text{Compiler}^H(G) \notin \{\text{FAIL}, \perp\}$ and Compiler^H is complete in the sense of Def. A.13. $\square$

The termination argument above applies only under the hypothesis of Def. A.13, namely that an equivalent assembly code exists. When none exists, the algorithm may enumerate indefinitely and return $\perp$ — and by Theorem A.1 no generator can do better while remaining sound and complete.

C.6 Visualization of the Relevant Rewrite Rules

The two rewrite rules that carry the Phase 2 proofs are visualized below on concrete tensors, using the colour convention of Appendix D.

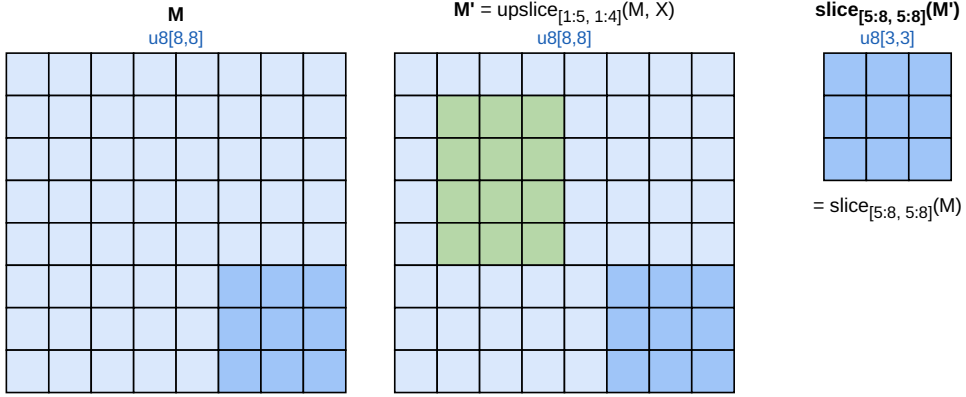


Fig. 30. Visualization of the rewrite rule $\text{slice}_{[s_1:e_1]}(\text{upslice}_{[s_2:e_2]}(M, X)) \equiv_{\mathcal{R}} \text{slice}_{[s_1:e_1]}(M)$, valid under the precondition $\text{disjoint}([s_1:e_1], [s_2:e_2])$. Because the written region (green) and the read region are disjoint, the read returns the same bytes whether or not the write happened, so the upslice can be discarded.

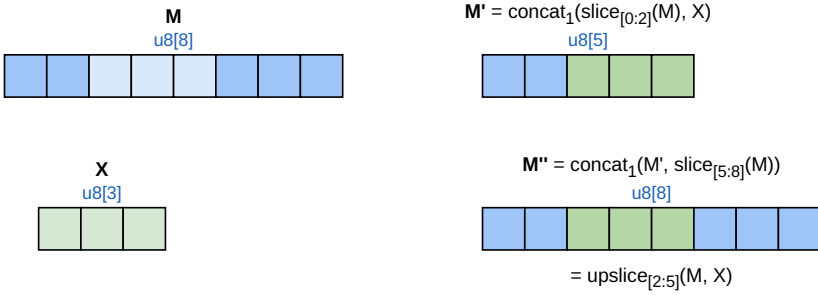


Fig. 31. Visualization of the rewrite rule $\text{upslice}_{[s:e]}(M, X) \equiv_{\mathcal{R}} \text{concat}_1(\text{concat}_1(\text{slice}_{[0:s]}(M), X), \text{slice}_{[e:]}(M))$. Writing X into the middle of M is the same as concatenating the untouched prefix of M , then X , then the untouched suffix — which is what lets an upslice be re-expressed using only the identity instructions slice^H and concat^H .

D Visualization of Tensor Operators

The tensor operators of Table 5 are the vocabulary in which ACT states everything else: tiled kernels are graphs over these operators, the abstract computation g_θ of every instruction is a graph over these operators, and the foundational axioms $\mathcal{R}$ are rewrites between such graphs. Their operational semantics are specified normatively by XLA-HLO [22, 23], and denotational semantics for a subset are given by TensorRight [6]. This section restates the operator list and then visualizes each operator on a small concrete example, since the shape-manipulating operators in particular are far easier to read as pictures than as index arithmetic.

The visualizations share one colour convention throughout:

Blue marks the first operand,

Green marks the second operand, and

Peach marks a tensor whose element values are *computed* rather than moved.

Colouring the operands separately means that when an operator merely rearranges elements, the reader can trace each output cell back to the operand it came from. A darker shade marks a region that is only *part* of a larger tensor — the window a slice reads, or the range an upslice writes — so that partial access is distinguishable at a glance from whole-tensor access. Cells are annotated with element values only where the values themselves carry information; for the purely structural operators the grid geometry is the whole story.

Tensor Operator	Operator Attributes	Description
$y = \text{slice}_{[s:e]}(x)$	$\text{type}(x), s, e$	Read sub-tensor $x[s:e]$
$y = \text{upslice}_{[s:e]}(x_1, x_2)$	$\text{type}(x_1), s, e$	Update sub-tensor $x_1[s:e] = x_2$
$y = \text{concat}_{dim}(x_1, x_2)$	$\text{type}(x_1), \text{type}(x_2), dim$	Concatenate x_1, x_2 across dimension dim
$y = \text{reshape}(x)$	$\text{type}(x), \text{type}(y)$	Reshape from $\text{type}(x)$ to $\text{type}(y)$
$y = \text{bitcvt}(x)$	$\text{type}(x), \text{type}(y)$	Bitcast conversion between basetypes
$y = \text{broadcast}_{dim}(x)$	$\text{type}(x), dim$	Broadcast over dimension dim
$y = \text{reduce}_{dim}(x)$	$\text{type}(x), dim$	Reduce-sum over dimension dim
$y = \text{transpose}_{[dims]}(x)$	$\text{type}(x), dims$	Permute the dimension order to $dims$
$y = \text{dot}_{(c_1, c_2)}(x_1, x_2)$	$\text{type}(x_1), \text{type}(x_2), c_1, c_2$	Dot product of x_1, x_2 over contracting dimensions c_1, c_2
$y = \text{exp}(x)$	$\text{type}(x)$	Element-wise exponential
$y = \text{div}(x_1, x_2)$	$\text{type}(x_1), \text{type}(x_2)$	Element-wise divide
$y = \text{copy}(x)$	$\text{type}(x)$	Identical copy

Table 5. Brief descriptions of the tensor operators discussed in this paper (detailed operational semantics in [22, 23] and denotational semantics for a subset in [6]). $\text{type}(x)$ refers to the tensor-type of tensor variable x . dim represents the dimension number (starting from 1). We use NumPy-like slice notation $x[s_1:e_1, s_2:e_2, \dots]$ and ignore type-based attributes in the visualizations.

D.1 Addressing Operators

slice and upslice are the two operators that name a sub-region of a tensor by address, and they are the pair on which ACT’s memory allocation rests: a data slice written by one instruction and read by another is expressed as an upslice followed by a slice, and Appendix C turns exactly that composition into the rewrite rule that discharges Phase 2’s correctness. Note that upslice returns a tensor of the shape of its *first* operand, not of the sub-tensor being written.

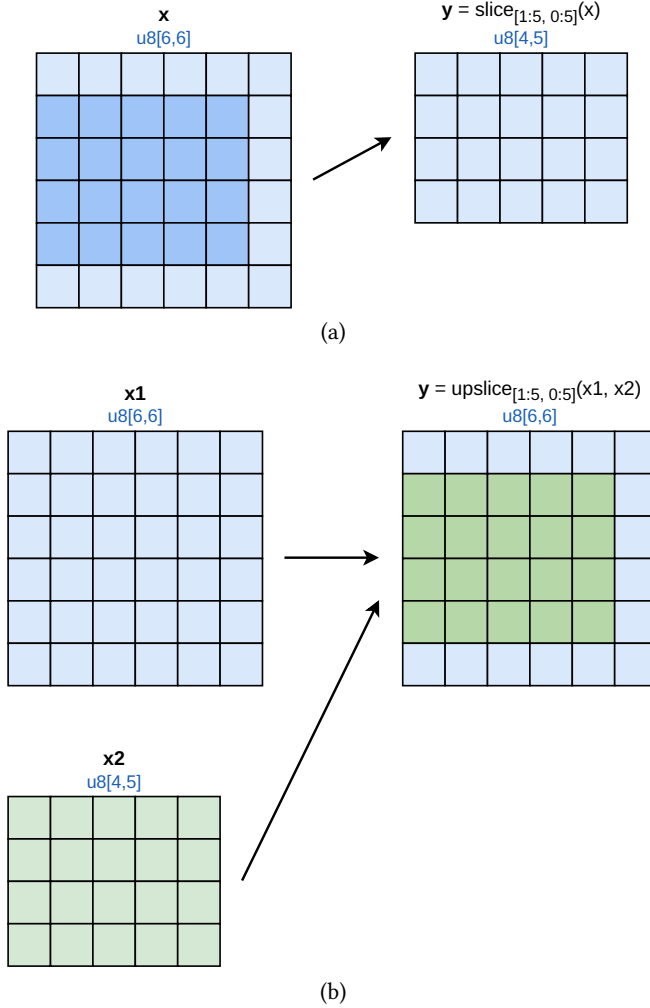


Fig. 32. The addressing operators. (a) slice extracts the 4×5 sub-tensor of x spanning rows 1–4 and columns 0–4; the darker region marks the elements read. (b) upslice returns a copy of x_1 with the 4×5 tensor x_2 written into the range $[1:5, 0:5]$. The output therefore has the shape of x_1 , and the green region records which of its elements came from x_2 .

D.2 Structural Operators

Every output element of these operators is a copy of some input element: they rearrange, replicate or reinterpret the shape of a tensor without doing arithmetic. This is exactly the class whose outputs stay blue or green in the visualizations, since each one can be traced back to the operand it came from. `concat` and `reshape` are of particular interest to ACT: `reshape` together with `transpose` is how ACT models tiled memory layouts (Appendix B), and `concat` is one of the two identity instructions whose availability makes Phase 1 complete.

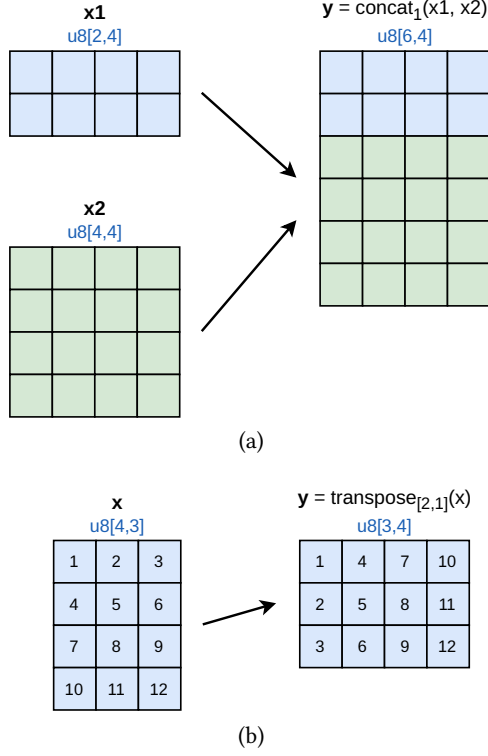


Fig. 33. Concatenation and transposition. (a) `concat` composes a $[6, 4]$ tensor from x_1 of shape $[2, 4]$ and x_2 of shape $[4, 4]$ by stacking them across dimension 1; the output colours record which operand contributed each row. (b) `transpose` permutes the dimension order, taking x from $[4, 3]$ to $[3, 4]$. The element values are shown so that the permutation can be read off directly.

D.3 Computational Operators

The remaining operators produce element values that are not present in any operand, and so are drawn in peach. `bitcvt` belongs here even though it invents no information: reinterpreting the bytes of a `u32` as four `u8` values yields elements that did not exist before, and its output cannot be traced back cell-for-cell to its input.

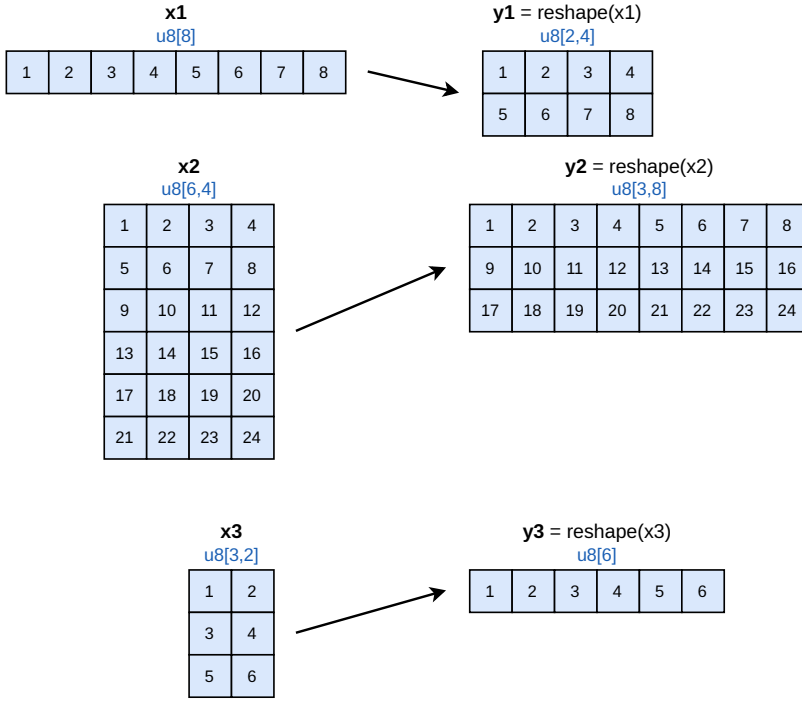


Fig. 34. reshape preserves the row-major element order and changes only the tensor-type, as shown by the element values carrying across each of the three examples: $[8] \rightarrow [2, 4]$, $[6, 4] \rightarrow [3, 8]$, and $[3, 2] \rightarrow [6]$. Because the element order is fixed, a reshape is fully determined by its input and output types, which is what allows ACT to treat it as a layout annotation rather than as data movement.

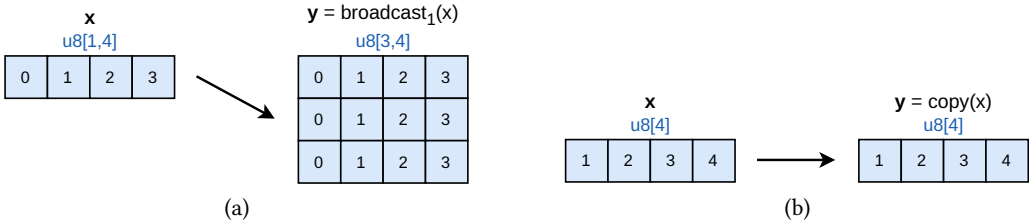


Fig. 35. Replication and copying. (a) broadcast replicates x across dimension 1, turning $[1, 4]$ into $[3, 4]$; every output row is a copy of the single input row. (b) copy copies x into y unchanged. ACT keeps copy as an explicit operator even though it changes nothing, because a copy between two tensor buffers is real data movement that the cost model has to account for.

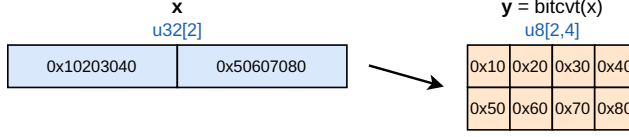


Fig. 36. `bitcvt` performs an element-wise bitcast between basetypes without changing the underlying bytes. Converting from `u32` to `u8` therefore turns each input element into four output elements, and the cell widths in the figure are drawn in proportion to the basetype width to make that four-to-one correspondence visible. `bitcvt` is how ACT reconciles the byte-addressable view of global memory with the typed view used by tensor operators.

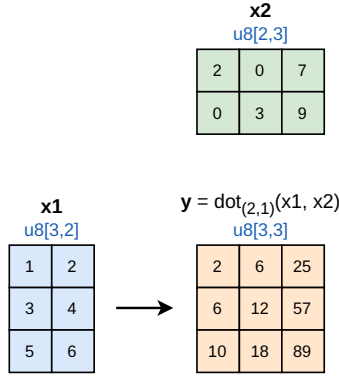


Fig. 37. $y = \text{dot}_{(2,1)}(x_1, x_2)$ matrix-multiplies x_1 of shape $[3, 2]$ with x_2 of shape $[2, 3]$, contracting dimension 2 of x_1 against dimension 1 of x_2 to give y of shape $[3, 3]$. The operands are placed so that each output element sits at the intersection of the row of x_1 and the column of x_2 that produce it.

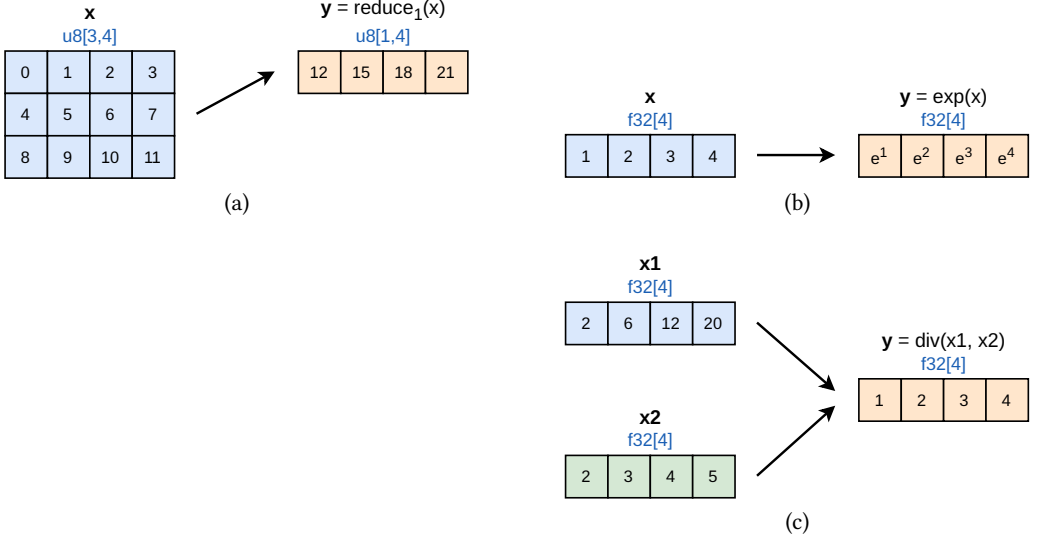


Fig. 38. Reduction and element-wise arithmetic. (a) reduce sum-reduces x across dimension 1, turning $[3, 4]$ into $[1, 4]$; each output element is the sum of the corresponding input column. (b) exp is the element-wise exponential. (c) div divides x_1 by x_2 element-wise. reduce is the counterpart of the broadcast of Fig. 35: both act along one dimension, but only reduce combines values.

E NNSmith Kernel Generation Setup

The compilation-time analysis of §8.7 needs tiled kernels that are representative in *size* rather than in workload. Real Gemmini kernels do not serve: the ones used for the performance comparison were capped at about ten nodes because the baselines were hand-written kernel libraries, which exist only for small, common shapes. Tiled kernels in practice run to 10–50 nodes, as seen in the TPUGraphs dataset [76]. We therefore generated 100 synthetic kernels with NNSmith [62], a random DNN generator, and used them both for that compilation-time analysis and for fuzz testing ACT-Gemmini. This section records the generation setup so the kernel population can be reproduced exactly.

E.1 Operator Set

The generated kernels exercise the 10 XLA-HLO operators listed in Table 6, but only six of them — dot, add, sub, neg, min and max — are requested directly by the include-list of Fig. 39. The other four arrive indirectly. reverse is not in NNSmith’s TensorFlow operator set, so we request core.Abs, which lowers to it; core.Abs is therefore the one include-list entry with no counterpart in Table 6. broadcast and reduce are introduced by XLA’s own lowering and tiling passes rather than being generated, which is the same way they arise in real kernels. clamp likewise appears only after lowering, when XLA fuses an adjacent max and min pair into a single operator. We consider this indirection a feature rather than a limitation of the setup: it means the kernel population contains operators in the same combinations that XLA itself produces, rather than only those a generator was told to emit.

Tensor Operator	Description
$y = \text{dot}(x_1, x_2)$	Matrix multiplication of x_1 and x_2
$y = \text{broadcast}_{dim}(x)$	Broadcast over dimension dim
$y = \text{reduce}_{dim}(x)$	Reduce-sum over dimension dim
$y = \text{reverse}_{dim}(x)$	Reverse elements over dimension dim
$y = \text{add}(x_1, x_2)$	Element-wise addition of x_1 and x_2
$y = \text{sub}(x_1, x_2)$	Element-wise subtraction of x_1 and x_2
$y = \text{neg}(x)$	Element-wise negation of x
$y = \text{min}(x, c)$	Element-wise minimum of x and c
$y = \text{max}(x, c)$	Element-wise maximum of x and c
$y = \text{clamp}(low, x, high)$	Element-wise clamp of x with low and $high$

Table 6. The 10 XLA-HLO operators present in the tiled kernels generated for fuzz testing using NNSmith [62]. Here c , low and $high$ denote constant operands.

E.2 Generation Parameters and Resulting Kernel Sizes

Each invocation fixes a seed, a node budget, an operator include-list, a model type and a backend; Fig. 39 shows one representative command line.

The node budget needs a word of explanation, because the size of the kernel that finally reaches ACT is neither the budget nor the number of operators NNSmith emits. mgen.max_nodes is an upper bound on the operators generated, and the resulting graph has multiple outputs, whereas ACT consumes single-output tiled kernels. We therefore combine all outputs with a tree of add operators, so the node count of the final XLA-HLO graph is

$$(\text{\#operators}) + (\text{\#input parameters}) + (\text{\#output nodes} - 1).$$

The input parameters and the combining add nodes are what push the final graph above the operator count: with `mgen.max_nodes` set to 50, kernels reach up to 89 nodes. Because the budget is only an upper bound, the number of operators NNSmith actually emits — and hence the number of outputs that have to be combined — varies from seed to seed. Varying `mgen.seed` across the 100 invocations therefore yields final sizes spanning 7 to 89 nodes, which is the population reported in §8.7. That range brackets the 10–50 nodes typical of the TPUGraphs kernels, so the compilation-time measurements cover both smaller and substantially larger inputs than are seen in practice.

```

1 nnsmith.model_gen model.type=tensorflow backend.type=xla \
2   mgen.seed=1002 \
3   mgen.max_nodes=50 \
4   mgen.include="[tensorflow.TFMatMul, core.Abs, core.Add, core.Max, core.Min,
5   core.Neg, core.Sub]" \
6   model.path="nnsmith_output/" \
7   debug.viz=true

```

Fig. 39. A representative NNSmith invocation used to generate one of the 100 synthetic kernels. Only `mgen.seed` varies across the population.

F Loop Fission Factors for the Exo Kernels

§8.6 compares ACT-Gemmini against Exo’s expert-written kernel library on six GEMM shapes and reports one speedup per shape. That single number is the outcome of a search: for each shape, ACT-Gemmini enumerates candidate loop fission factors and its cost model selects one. This section gives the full candidate tables behind those six numbers, so that the reported speedup can be seen in context.

F.1 Reading the Tables

We evaluated every candidate on Gemmini’s Verilator-based RTL simulator and recorded its cycle count. Each row is one combination of loop fission factors, together with the accumulator and scratchpad rows it occupies, its measured cycle count, and its speedup over the original hand-optimized Exo kernel.

The five loop variables split into three levels. The outer loops io and jo control outer tiling. The inner loops i and j are the fine-grained tile sizes that load and store into Gemmini’s scratchpad. The innermost loop ko iterates over the partial sums held in the accumulator: within one ko iteration the kernel loads a single 16×64 tile of A and four 16×64 tiles of B , multiplies them as 16×16 matrix multiplications, accumulates the partial sums, and so computes one 16×64 tile of the output C . Candidates whose factors exceed the accumulator or scratchpad row limits are infeasible and are not listed.

In every table the original Exo factors are **bolded** and the best factors, meaning those with the lowest cycle count, are **highlighted in orange**. Where the two coincide, ACT-Gemmini rediscovered Exo’s hand-tuned factors and the speedup is 1.00x. Where the highlighted row has a lower cycle count than the bolded row, ACT-Gemmini found a better loop fission factor than the expert did. This happened for three of the six kernels, which is the result summarized in §8.6.

F.2 Candidate Tables

The six tables follow in the order $512 \times 512 \times 512$, $784 \times 1024 \times 256$, $12544 \times 256 \times 64$, $12544 \times 64 \times 256$, $3136 \times 512 \times 128$, $3136 \times 128 \times 512$. Note that this is *not* the left-to-right order of the bars in Fig. 21, which places $784 \times 1024 \times 256$ last rather than second; match tables to bars by shape rather than by position.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
2	16	2	4	8	256	16384	624170	1.00
2	16	4	2	8	128	12288	624228	1.00
4	8	2	4	8	256	12288	659384	0.95
2	16	8	1	8	64	10240	650725	0.96
8	4	2	4	8	256	10240	855191	0.73
16	2	2	4	8	256	9216	1039928	0.60
32	1	2	4	8	256	8704	1351586	0.46
4	8	4	2	8	128	8192	659357	0.95
4	8	8	1	8	64	6144	687626	0.91
8	4	4	2	8	128	6144	965931	0.65
16	2	4	2	8	128	5120	1223264	0.51
32	1	4	2	8	128	4608	1549222	0.40
8	4	8	1	8	64	4096	1240902	0.50
16	2	8	1	8	64	3072	1385353	0.45
32	1	8	1	8	64	2560	1663782	0.38

Table 7. Loop fission candidates for matmul $512 \times 512 \times 512$. ACT-Gemmini’s best factor coincides with Exo’s hand-tuned factor, giving a 1.00x speedup.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
1	49	8	2	4	128	14592	968206	1.30
1	49	16	1	4	64	13568	1126936	1.12
7	7	2	8	4	512	9984	1258985	1.00
7	7	4	4	4	256	5888	1457055	0.86
7	7	8	2	4	128	3840	1757805	0.72
7	7	16	1	4	64	2816	1857018	0.68
49	1	2	8	4	512	8448	2105258	0.60
49	1	4	4	4	256	4352	2428470	0.52
49	1	8	2	4	128	2304	2442627	0.52
49	1	16	1	4	64	1280	2634956	0.48

Table 8. Loop fission candidates for matmul $784 \times 1024 \times 256$. ACT-Gemmini's best factor outperforms Exo's hand-tuned factor, giving a 1.30x speedup.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
4	196	1	4	1	256	13568	1499481	1.00
4	196	2	2	1	128	13056	1513787	0.99
4	196	4	1	1	64	12800	2027815	0.74
7	112	1	4	1	256	8192	1569955	0.96
7	112	2	2	1	128	7680	1602036	0.94
7	112	4	1	1	64	7424	2059381	0.73
8	98	1	4	1	256	7296	1579550	0.95
8	98	2	2	1	128	6784	1604309	0.93
8	98	4	1	1	64	6528	1842807	0.81
14	56	1	4	1	256	4608	1702098	0.88
16	49	1	4	1	256	4160	1781546	0.84
14	56	2	2	1	128	4096	1540089	0.97
14	56	4	1	1	64	3840	2003559	0.75
16	49	2	2	1	128	3648	1696348	0.88
16	49	4	1	1	64	3392	2103647	0.71
28	28	1	4	1	256	2816	1737137	0.86
28	28	2	2	1	128	2304	1720557	0.87
28	28	4	1	1	64	2048	2119776	0.71
49	16	1	4	1	256	2048	1738821	0.86
56	14	1	4	1	256	1920	1733756	0.86
49	16	2	2	1	128	1536	1742679	0.86
98	8	1	4	1	256	1536	1738183	0.86
112	7	1	4	1	256	1472	1738658	0.86
56	14	2	2	1	128	1408	1743286	0.86
196	4	1	4	1	256	1280	1716865	0.87
49	16	4	1	1	64	1280	2183789	0.69
392	2	1	4	1	256	1152	1720090	0.87
56	14	4	1	1	64	1152	2184509	0.69
784	1	1	4	1	256	1088	1793534	0.84
98	8	2	2	1	128	1024	1775427	0.84
112	7	2	2	1	128	960	1745616	0.86
196	4	2	2	1	128	768	1752644	0.86
98	8	4	1	1	64	768	2186091	0.69
112	7	4	1	1	64	704	2105043	0.71
392	2	2	2	1	128	640	1802506	0.83
784	1	2	2	1	128	576	1883025	0.80
196	4	4	1	1	64	512	2175743	0.69
392	2	4	1	1	64	384	2114435	0.71
784	1	4	1	1	64	320	2146024	0.70

Table 9. Loop fission candidates for matmul $12544 \times 256 \times 64$. ACT-Gemmini's best factor coincides with Exo's hand-tuned factor, giving a 1.00x speedup.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
49	16	1	1	4	1024	5120	1462706	1.29
56	14	1	1	4	896	4608	1800241	1.05
98	8	1	1	4	512	3072	1884887	1.00
112	7	1	1	4	448	2816	2114836	0.89
196	4	1	1	4	256	2048	2110773	0.89
392	2	1	1	4	128	1536	1883721	1.00
784	1	1	1	4	64	1280	2171151	0.87

Table 10. Loop fission candidates for matmul $12544 \times 64 \times 256$. ACT-Gemmini's best factor outperforms Exo's hand-tuned factor, giving a 1.29x speedup.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
2	98	2	4	2	256	14592	1113009	1.06
2	98	4	2	2	128	13568	1205199	0.98
2	98	8	1	2	64	13056	1433162	0.82
4	49	1	8	2	512	10368	1179333	1.00
4	49	2	4	2	256	8320	1239771	0.95
7	28	1	8	2	512	7680	1156398	1.02
4	49	4	2	2	128	7296	1224648	0.96
4	49	8	1	2	64	6784	1328323	0.89
14	14	1	8	2	512	5888	1426823	0.83
7	28	2	4	2	256	5632	1173115	1.01
28	7	1	8	2	512	4992	1562962	0.75
49	4	1	8	2	512	4608	1641041	0.72
7	28	4	2	2	128	4608	1172983	1.01
98	2	1	8	2	512	4352	1667040	0.71
196	1	1	8	2	512	4224	1691607	0.70
7	28	8	1	2	64	4096	1324928	0.89
14	14	2	4	2	256	3840	1393443	0.85
28	7	2	4	2	256	2944	1520771	0.78
14	14	4	2	2	128	2816	1393532	0.85
49	4	2	4	2	256	2560	1611814	0.73
14	14	8	1	2	64	2304	1589328	0.74
98	2	2	4	2	256	2304	1656962	0.71
196	1	2	4	2	256	2176	1692347	0.70
28	7	4	2	2	128	1920	1520284	0.78
49	4	4	2	2	128	1536	1615518	0.73
28	7	8	1	2	64	1408	1689620	0.70
98	2	4	2	2	128	1280	1656610	0.71
196	1	4	2	2	128	1152	1691181	0.70
49	4	8	1	2	64	1024	1782025	0.66
98	2	8	1	2	64	768	1817532	0.65
196	1	8	1	2	64	640	1836277	0.64

Table 11. Loop fission candidates for matmul $3136 \times 512 \times 128$. ACT-Gemmini's best factor outperforms Exo's hand-tuned factor, giving a 1.06x speedup.

io	i	jo	j	ko	Accumulator Rows	Scratchpad Rows	Cycles	Speedup
14	14	1	2	8	128	11264	1145575	1.00
14	14	2	1	8	128	11264	1205532	0.95
28	7	1	2	8	128	7680	1150215	1.00
28	7	2	1	8	128	7680	1215381	0.94
49	4	1	2	8	128	6144	1408841	0.81
49	4	2	1	8	128	6144	1419196	0.81
98	2	1	2	8	128	5120	1599727	0.72
98	2	2	1	8	128	5120	1639071	0.70
196	1	1	2	8	128	4608	1699842	0.67
196	1	2	1	8	128	4608	1739270	0.66

Table 12. Loop fission candidates for matmul $3136 \times 128 \times 512$. ACT-Gemmini's best factor coincides with Exo's hand-tuned factor, giving a 1.00x speedup.

G Detailed Analysis of ACT-Gemmini versus the Gemmini SW Library

§8.6 evaluates ACT-Gemmini against the Gemmini software library over three groups of kernels: “*supported*”, where the library has a hand-written implementation; “*composite*”, which compose several library calls; and “*not supported*”, whose operators have no Gemmini implementation at all. It reports the headline numbers and defers the per-kernel discussion here. This section takes the last two groups in turn, explains where the speedups come from, and then reports two further results: an experiment with Gemmini’s CISC-type instructions, and the convolution kernels.

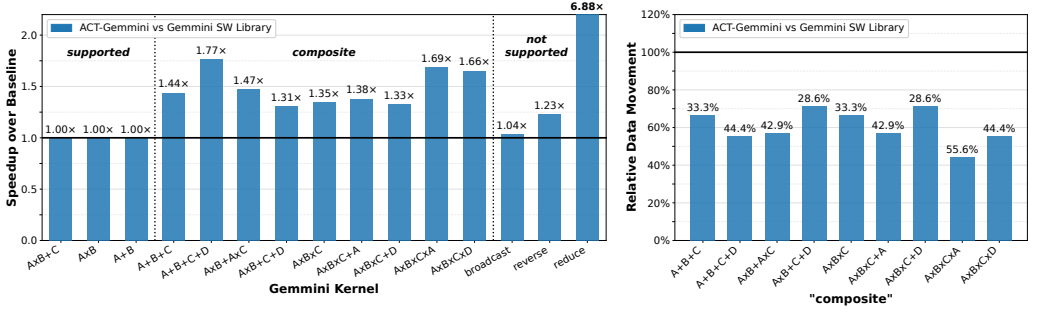


Fig. 40. ACT-Gemmini against the Gemmini SW library. Left: speedup of the compiled code across all three kernel groups, reproduced from Fig. 21 so that the two panels can be read side by side. Right: data movement of ACT-Gemmini relative to the library for the “*composite*” kernels, where lower is better; the number printed on each bar is the *reduction*, that is 100% minus the plotted height. Note that the “*not supported*” bars on the left are measured against host execution, since those operators have no Gemmini implementation in the library.

G.1 “*composite*” Kernels

A “*composite*” kernel is a composition of several GEMM ($\times$) and ADD ($+$) operations that the library can only execute as a sequence of separate calls. Each library call must leave its result in HBM, because the library’s calling convention cannot assume anything about scratchpad residency across calls. The next call then loads that same tensor straight back in. ACT-Gemmini has no such constraint: it compiles the whole kernel at once, so an intermediate that is produced and immediately consumed never leaves the accelerator.

Fig. 41 makes the difference concrete for $A \times B \times C$. The library issues two GEMM kernels, and the store of $A \times B$, the fence, and the reload of $A \times B$ between them (shaded red) are pure overhead. ACT-Gemmini keeps the intermediate in the scratchpad and emits neither. For $A \times B \times C$ the library moves six tiles where ACT-Gemmini moves four, a 33.3% reduction, and the measured speedup is 1.35x. The saving grows with scratchpad utilization, because a larger resident intermediate displaces proportionally more traffic, and it grows with the length of the chain, because each additional library call contributes another store-fence-load round trip that ACT-Gemmini does not emit.

The same pattern holds across the other “*composite*” kernels, since each additional library call in the baseline adds another store-fence-load round trip that ACT-Gemmini removes. The gain is not monotone in kernel size, however: how much is saved also depends on how much of the intermediate stays resident, so two of the three-call kernels in Fig. 40 score slightly below the best two-call one. For chains of GEMM, the speedup goes from 1x for $A \times B$, to 1.35x for $A \times B \times C$, to 1.66x for $A \times B \times C \times D$; for chains of ADD it goes from 1x, to 1.44x, to 1.77x. Taken over the

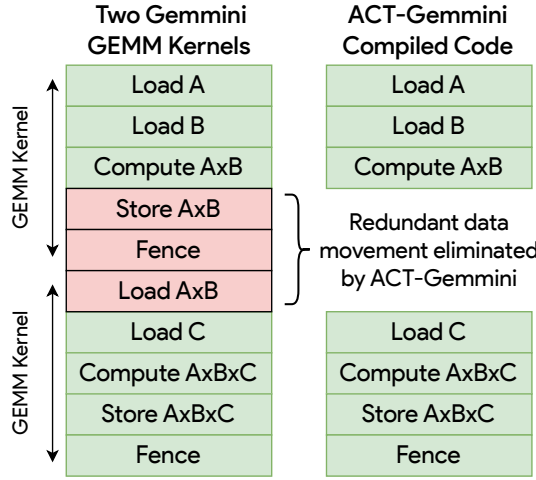


Fig. 41. Breakdown of the Gemini SW library kernels and the ACT-Gemmini compiled code for $A \times B \times C$. ACT-Gemmini eliminates the redundant load-store instructions between library calls (shaded red).

group, Fig. 40 shows ACT-Gemmini outperforming the library by up to **1.77x** (geomean 1.48x) while reducing data movement by up to **55.6%** (average 39.3%).

G.2 “not supported” Kernels

The Gemini SW library has no implementation of broadcast, reverse or reduce, so a conventional toolchain has no choice but to run them on the host Rocket CPU. ACT-Gemmini instead compiles all three onto the systolic array, and it does so without any operator-specific support.

The mechanism is compile-time constant detection (§5.3). Each of these three operators can be written as a matrix multiplication against a fixed matrix: broadcast against a $[1, 16]$ all-ones vector, reduce against a $[16, 1]$ all-ones vector, and reverse against a reversed identity matrix. ACT-Gemmini detects that those operands are compile-time constants, materializes them in constant memory, and emits an ordinary matmul. Fig. 42 shows the resulting pii graphs, with the constant memory region shaded purple and the constant-valued nodes eye (the identity matrix) and ones (the all-ones vector) in orange. Nothing in this is Gemini-specific: the same constant detection would apply to any accelerator whose ISA description exposes a matrix-multiply instruction.

Because the baseline here is host execution rather than an accelerator kernel, the comparison is *cross-architecture*, and it should be read as the practical cost of host fallback rather than as an accelerator-to-accelerator speedup.

The outcome varies sharply across the three operators, and it is worth being precise about why. reduce gains the most, at **6.88x** (§8.6): it is the only one of the three that does real arithmetic proportional to its input, so moving it off the Rocket CPU removes a genuine compute bottleneck. reverse gains **1.23x** and broadcast only **1.04x**. Both are pure data movement, so on the host they are already memory-bound; expressing them as a systolic-array matmul against a constant removes the host round trip but replaces it with an accelerator that is no faster at moving the same bytes. The value of compiling them is therefore not raw speed but *coverage* — the kernel no longer has to leave the accelerator at all, which is what makes the surrounding “*composite*” fusions of §G.1 possible in the first place. For reference on the same axis, $\text{add}(A, B)$ reaches 6.82x over host execution, though it is a “*supported*” kernel that the library does implement.

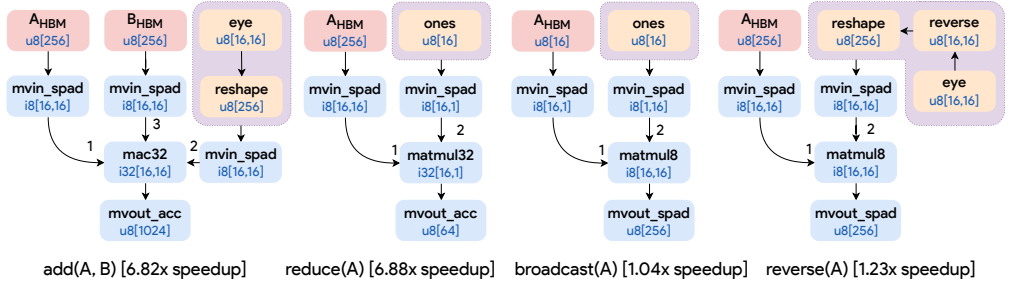


Fig. 42. pii graphs with constant memory (purple region) generated by ACT-Gemmini. The orange nodes hold the values materialized in constant memory: eye is an identity matrix and ones is a vector of all ones. Speedups are with respect to the Gemmini host processor (Rocket chip).

G.3 CISC-type Instructions

The TAIDL description used to generate ACT-Gemmini covers Gemmini’s 11 RISC-type instructions. Gemmini [36] also provides experimental CISC-type instructions, which perform a matrix multiplication over a large tile in a single instruction and stream data directly to and from HBM. These are implemented with a hardware FSM that double-buffers data movement against computation. That gives them an advantage a RISC-type sequence cannot match: a compiler emitting separate load, compute and store instructions is limited by the instruction scheduler’s reordering and pipelining window, whereas the FSM overlaps the two by construction. We therefore expected CISC-type instructions to win even where a RISC-type schedule moves less data.

To test this, we added the CISC-type instructions to the TAIDL description and generated a new backend, ACT-Gemmini+. The outcome was as predicted. ACT-Gemmini+ still enumerated the RISC-only assembly among its candidates — the RISC-type instructions remained in the description, so those candidates were never lost — but its cost model ranked a CISC-type candidate above them, and that is what it emitted.

The result itself is unsurprising; what matters is what it took to obtain. ACT required *no* algorithmic changes to exploit the new instruction class — only the TAIDL description was extended, and the cost-model-driven enumeration of §7 preferred the faster code automatically. A hand-written backend would have needed a new scheduling strategy to use an FSM-driven instruction well. This is evidence that ACT generalizes across RISC- and CISC-style instructions of the same accelerator.

G.4 Convolution Kernels

Convolutions on Gemmini are lowered onto the same systolic array as matrix multiplications and share its compute unit, so they exercise the same instruction selection and memory allocation decisions. We observed correspondingly similar results for convolution kernels, both against the Gemmini SW library and against Exo’s expert-written kernel library. Since they add no separate insight, §8.6 reports no convolution kernels.

H The oneDNN Kernels Compiled by ACT-AMX

§8.5 evaluates ACT-AMX against oneDNN, a library hand-written and heavily optimized for Intel ISA including AMX and AVX512. The comparison is unusual among our evaluations in that the target is not a speedup but a *match*: oneDNN’s assembly is already expert-tuned, and the question is whether an automatically generated backend can reach it. This section presents the five kernels behind that claim so the match can be inspected.

For each kernel we give three artifacts: its tensor computation graph, the XLA-HLO IR that ACT-AMX takes as input, and the assembly ACT-AMX emits. The emitted assembly is the same hand-optimized code the oneDNN library produces — ACT-AMX enumerates it as one candidate among many and its cost model selects it (§8.5) — which is why the two match in performance. For the fp32 and mixed kernels (K3–K5) the assembly listings show a representative excerpt, with elided lines marked

H.1 Kernel Selection and Statistics

The XLA compiler emits tiled kernels which are then compiled to assembly using the oneDNN library, via the `xla_cpu_use_mk1_dnn` flag. We selected five kernels commonly observed in the oneDNN examples [49], listed in Table 13. They are deliberately simple as computations — all five are matrix multiplications with element-wise post-processing — but carry complex memory layouts on their inputs, which is what makes them a demanding test of Module 1’s layout modeling (Appendix B).

Table 13 separates the size of the input from the size of the output. #Nodes counts every instruction of the kernel’s tensor computation graph, parameters and constants included, except the ROOT tuple that packages K5’s two outputs, which is bookkeeping rather than computation. #AMX and #AVX512 count the AMX tile instructions and AVX512 instructions in the compiled assembly. The split across the five kernels is clean and follows their element types: the two int8 kernels (K1, K2) compile purely to AMX tile instructions, the two fp32 kernels (K3, K4) purely to AVX512, and the mixed kernel (K5) to a combination of both. This much is forced rather than chosen — the TAILD description we used covers AMX instructions only for int8 and AVX512 instructions only for fp32, so no other assignment is type-correct. What the cost model decides is the code *within* each class: which tiling, which register blocking, which instruction order.

Kernel	oneDNN example	#Nodes	#AMX	#AVX512
K1	cnn_inf_amx	6	16	0
K2	rnn_inf_amx	6	9	0
K3	mem_format_avx	7	0	100
K4	sgemm_avx	7	0	107
K5	cnn_inf_mix	23	16	37

Table 13. The five selected oneDNN kernels and their optimized assembly code in the oneDNN library. #Nodes is the number of operators in the tensor computation graph; #AMX and #AVX512 count instructions of each class in the compiled assembly.

H.2 K1: `cnn_inf_amx`

K1 is the int8 convolution-inference kernel and the simplest of the five: a single matrix multiplication of a `u8[32,64]` activation against an `i8[32,64]` weight tensor carried in the AMX-native tiled layout `{T(16,4)}`. That layout is the VNNI packing the `tdpbud` instruction requires, with four elements of

the reduction dimension interleaved; §8.2 shows the corresponding transformation on tile register `src1`. The compiled code is 16 AMX instructions with no AVX512 at all: four `tilezero`, four `tileload`, four `tdpbust`, and four `tilestore`, tiling the 32×32 output into four 16×16 tile registers.

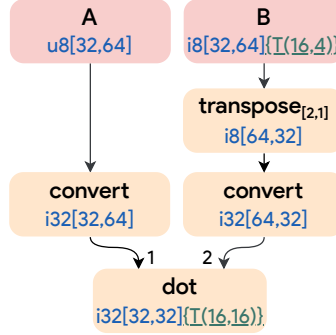


Fig. 43. Tensor computation graph for oneDNN kernel K1.

```

1 ENTRY k1 {
2   A = u8[32,64] parameter(0)
3   B = i8[32,64]{T(16,4)} parameter(1)
4   convert.0 = i32[32,64] convert(A)
5   transpose.1 = i8[64,32] transpose(B), dimensions=[2,1]
6   convert.2 = i32[64,32] convert(transpose.1)
7   ROOT dot.3 = i32[32,32]{T(16,16)} dot(convert.0, convert.2)
8 }

```

Fig. 44. XLA-HLO code for kernel K1.

```

1 tilezero(dst = tmm0)
2 tilezero(dst = tmm1)
3 tilezero(dst = tmm2)
4 tilezero(dst = tmm3)
5 tileload(dst = tmm4, mem_addr = 0x0000)
6 tileload(dst = tmm5, mem_addr = 0x0400)
7 tileload(dst = tmm6, mem_addr = 0x0800)
8 tileload(dst = tmm7, mem_addr = 0x0c00)
9 tdpbust(dst = tmm0, src0 = tmm4, src1 = tmm6)
10 tdpbust(dst = tmm1, src0 = tmm4, src1 = tmm7)
11 tdpbust(dst = tmm2, src0 = tmm5, src1 = tmm6)
12 tdpbust(dst = tmm3, src0 = tmm5, src1 = tmm7)
13 tilestore(src = tmm0, mem_addr = 0x1000)
14 tilestore(src = tmm1, mem_addr = 0x1400)
15 tilestore(src = tmm2, mem_addr = 0x1800)
16 tilestore(src = tmm3, mem_addr = 0x1c00)

```

Fig. 45. Compiled assembly code for kernel K1.

H.3 K2: rnn_inf_amx

K2 is the int8 recurrent-inference kernel, structurally identical to K1 but with a half-height activation, $u8[16,64]$ instead of $u8[32,64]$. The consequence is visible in the assembly: because the output is 16×32 rather than 32×32 , it occupies two tile registers instead of four, and the instruction count falls from 16 to 9. Nothing else changes, which is the point — the same generated backend handles both without any shape-specific path.

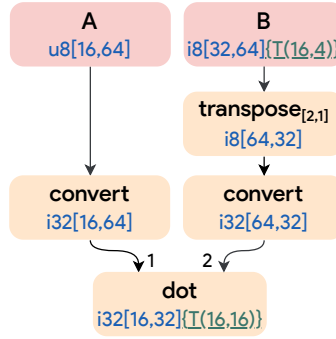


Fig. 46. Tensor computation graph for oneDNN kernel K2.

```

1 ENTRY k2 {
2   A = u8[16,64] parameter(0)
3   B = i8[32,64]{T(16,4)} parameter(1)
4   convert.0 = i32[16,64] convert(A)
5   transpose.1 = i8[64,32] transpose(B), dimensions=[2,1]
6   convert.2 = i32[64,32] convert(transpose.1)
7   ROOT dot.3 = i32[16,32]{T(16,16)} dot(convert.0, convert.2)
8 }

```

Fig. 47. XLA-HLO code for kernel K2.

```

1 tilezero(dst = tmm0)
2 tilezero(dst = tmm1)
3 tileloadadd(dst = tmm2, mem_addr = 0x0000)
4 tileloadadd(dst = tmm3, mem_addr = 0x0400)
5 tileloadadd(dst = tmm4, mem_addr = 0x0800)
6 tdpbusd(dst = tmm0, src0 = tmm2, src1 = tmm3)
7 tdpbusd(dst = tmm1, src0 = tmm2, src1 = tmm4)
8 tilestored(src = tmm0, mem_addr = 0x0c00)
9 tilestored(src = tmm1, mem_addr = 0x1000)

```

Fig. 48. Compiled assembly code for kernel K2.

H.4 K3: mem_format_avx

K3 is an fp32 kernel exercising memory-format propagation. It is an outer product: a $f32[32]$ vector reshaped to a column, multiplied by a single sliced column of $f32[14,4]$ transposed to a row, with the result transposed into the tiled layout $\{1,0:T(14,16)\}$. Since AMX has no fp32 tile instruction, ACT-AMX compiles it entirely to AVX512: 28 accumulator registers are zeroed, the two input rows

are loaded once, and 14 iterations each broadcast one scalar and issue two fused multiply-adds, followed by 28 stores. The excerpt below shows the first two iterations and the last.

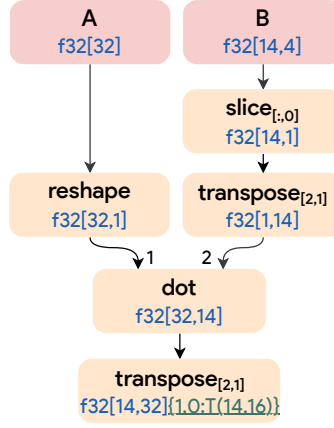


Fig. 49. Tensor computation graph for oneDNN kernel K3.

```

1 ENTRY k3 {
2   A = f32[32] parameter(0)
3   B = f32[14,4] parameter(1)
4   reshape.0 = f32[32,1] reshape(A)
5   slice.1 = f32[14,1] slice(B), dimensions=[:,0]
6   transpose.2 = f32[1,14] transpose(slice.1), dimensions=[2,1]
7   dot.3 = f32[32,14] dot(reshape.0, transpose.2)
8   ROOT transpose.4 = f32[14,32]{1,0:T(14,16)} transpose(dot.3), dimensions=[2,1]
9 }

```

Fig. 50. XLA-HLO code for kernel K3.

```

1 vpxord(dst = zmm31, src0 = zmm31, src1 = zmm31)
2 ...
3 vpxord(dst = zmm4, src0 = zmm4, src1 = zmm4)
4 vmovupsz(dst = zmm2, mem_addr=0x000)
5 vmovupsz(dst = zmm3, mem_addr=0x040)
6 vbroadcastss1(dst = zmm0, mem_addr=0x080)
7 vfmadd231ps(dst = zmm31, src0 = zmm3, src1 = zmm0)
8 vfmadd231ps(dst = zmm30, src0 = zmm2, src1 = zmm0)
9 vbroadcastss1(dst = zmm0, mem_addr=0x090)
10 vfmadd231ps(dst = zmm29, src0 = zmm3, src1 = zmm0)
11 vfmadd231ps(dst = zmm28, src0 = zmm2, src1 = zmm0)
12 ...
13 vbroadcastss1(dst = zmm0, mem_addr=0x150)
14 vfmadd231ps(dst = zmm5, src0 = zmm3, src1 = zmm0)
15 vfmadd231ps(dst = zmm4, src0 = zmm2, src1 = zmm0)
16 vmovupsz(src = zmm31, mem_addr=0x160)
17 vmovupsz(src = zmm30, mem_addr=0x1a0)
18 ...
19 vmovupsz(src = zmm4, mem_addr=0x820)

```

Fig. 51. Compiled assembly code for kernel K3. Iteration n accumulates into the register pair $(zmm_{31-2(n-1)}, zmm_{30-2(n-1)})$, so the 14 iterations consume the 28 accumulators zeroed at the top.

H.5 K4: sgemm_avx

K4 is the fp32 general matrix multiplication. Its graph has the same topology as K3's, but the two dimensions move in opposite directions: the vector grows from $f32[32]$ to $f32[48]$ while the sliced row shrinks from 14 elements to 8. That is what changes the register blocking. K3 holds 28 accumulators and runs 14 iterations of two fused multiply-adds; K4 holds 24 and runs 8 iterations of three, against three loaded input registers rather than two, giving 107 AVX512 instructions. The vaddps block near the end accumulates a further operand read from memory before the results are stored.

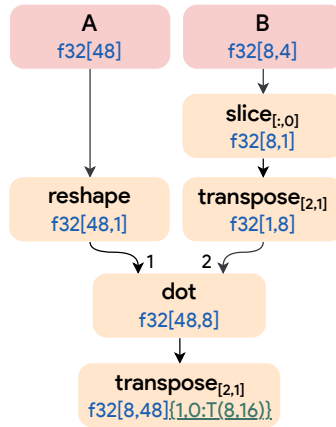


Fig. 52. Tensor computation graph for oneDNN kernel K4.

```

1 ENTRY k4 {
2   A = f32[48] parameter(0)
3   B = f32[8,4] parameter(1)
4   reshape.0 = f32[48,1] reshape(A)
5   slice.1 = f32[8,1] slice(B), dimensions=[:,0]
6   transpose.2 = f32[1,8] transpose(slice.1), dimensions=[2,1]
7   dot.3 = f32[48,8] dot(reshape.0, transpose.2)
8   ROOT transpose.4 = f32[8,48]{1,0:T(8,16)} transpose(dot.3), dimensions=[2,1]
9 }

```

Fig. 53. XLA-HLO code for kernel K4.

```

1 vpxord(dst = zmm8, src0 = zmm8, src1 = zmm8)
2 ...
3 vpxord(dst = zmm31, src0 = zmm31, src1 = zmm31)
4 vmovupsz(dst = zmm0, mem_addr=0x000)
5 vmovupsz(dst = zmm1, mem_addr=0x040)
6 vmovupsz(dst = zmm2, mem_addr=0x080)
7 vbroadcastssl(dst = zmm6, mem_addr=0x0c0)
8 vfmadd231ps(dst = zmm8, src0 = zmm6, src1 = zmm0)
9 vfmadd231ps(dst = zmm16, src0 = zmm6, src1 = zmm1)
10 vfmadd231ps(dst = zmm24, src0 = zmm6, src1 = zmm2)
11 vbroadcastssl(dst = zmm7, mem_addr=0x0d0)
12 vfmadd231ps(dst = zmm9, src0 = zmm7, src1 = zmm0)
13 vfmadd231ps(dst = zmm17, src0 = zmm7, src1 = zmm1)
14 vfmadd231ps(dst = zmm25, src0 = zmm7, src1 = zmm2)
15 ...
16 vbroadcastssl(dst = zmm6, mem_addr=0x120)
17 vfmadd231ps(dst = zmm14, src0 = zmm6, src1 = zmm0)
18 vfmadd231ps(dst = zmm22, src0 = zmm6, src1 = zmm1)
19 vfmadd231ps(dst = zmm30, src0 = zmm6, src1 = zmm2)
20 vbroadcastssl(dst = zmm7, mem_addr=0x130)
21 vfmadd231ps(dst = zmm15, src0 = zmm7, src1 = zmm0)
22 vfmadd231ps(dst = zmm23, src0 = zmm7, src1 = zmm1)
23 vfmadd231ps(dst = zmm31, src0 = zmm7, src1 = zmm2)
24 vaddps(dst = zmm8, src0 = zmm8, mem_addr=0x140)
25 ...
26 vaddps(dst = zmm31, src0 = zmm31, mem_addr=0x700)
27 vmovupsz(src = zmm8, mem_addr=0x740)
28 ...
29 vmovupsz(src = zmm31, mem_addr=0xd00)

```

Fig. 54. Compiled assembly code for kernel K4. The accumulators zmm8–zmm31 are consumed in ascending order by both the vaddps block and the stores that follow it.

H.6 K5: cnn_inf_mix

K5 is the mixed-precision convolution-inference kernel and the most interesting of the five, because it is the only one whose graph spans both element types and therefore both instruction classes. Its graph has two independent halves. The first is exactly K1’s int8 matrix multiplication, which compiles to the same 16 AMX instructions. The second is an fp32 post-processing chain – scale, bias, ReLU, scale again, clamp to [0, 255], and convert down to u8 – which compiles to 37 AVX512 instructions. Neither half is annotated as AMX or AVX512 anywhere in the input; ACT-AMX enumerates candidates for the whole graph and emits the mixture without any per-kernel direction. This is the concrete sense in which ACT does not need per-kernel tuning: the same generated backend produces pure-AMX code for K1, pure-AVX512 code for K3, and the correct mixture here.

Received 2026-03-17; accepted 2026-08-06

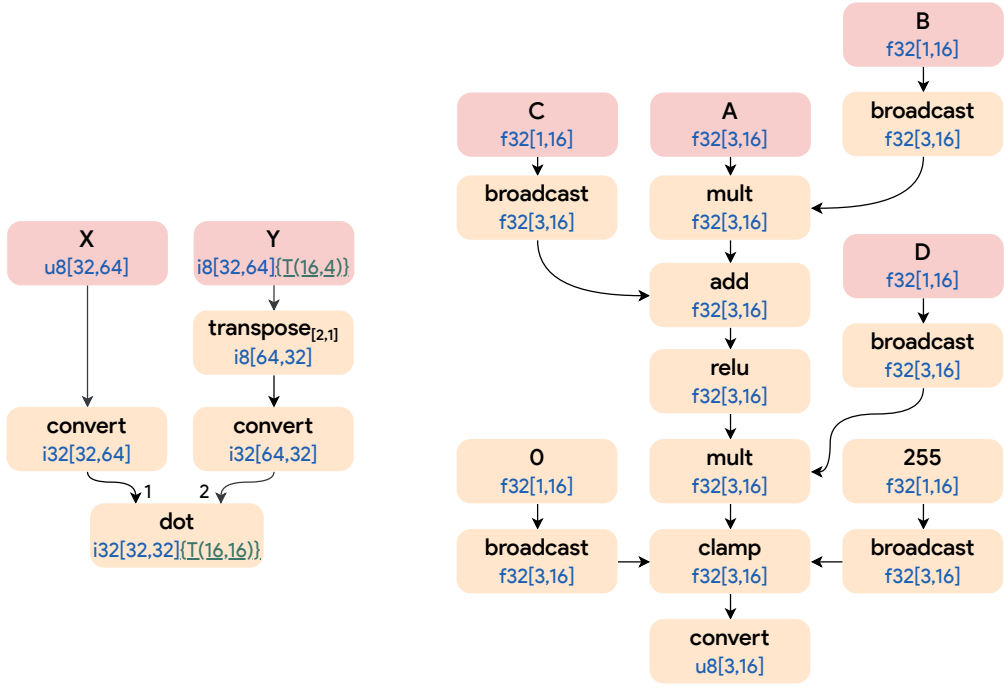


Fig. 55. Tensor computation graph for oneDNN kernel K5. The left half is K1's int8 matrix multiplication; the right half is the fp32 post-processing chain.

```

1 ENTRY k5 {
2   X = u8[32,64] parameter(0)
3   Y = i8[32,64][T(16,4)] parameter(1)
4   convert.0 = i32[32,64] convert(X)
5   transpose.1 = i8[64,32] transpose(Y), dimensions=[2,1]
6   convert.2 = i32[64,32] convert(transpose.1)
7   dot.3 = i32[32,32][T(16,16)] dot(convert.0, convert.2)
8   A = f32[3,16] parameter(2)
9   B = f32[1,16] parameter(3)
10  C = f32[1,16] parameter(4)
11  D = f32[1,16] parameter(5)
12  constant.0 = f32[1,16] constant(0)
13  constant.255 = f32[1,16] constant(255)
14  broadcast.4 = f32[3,16] broadcast(B), dimensions=[1]
15  multiply.5 = f32[3,16] multiply(A, broadcast.4)
16  broadcast.6 = f32[3,16] broadcast(C), dimensions=[1]
17  add.7 = f32[3,16] add(multiply.5, broadcast.6)
18  broadcast.11 = f32[3,16] broadcast(constant.0), dimensions=[1]
19  broadcast.12 = f32[3,16] broadcast(constant.255), dimensions=[1]
20  relu.8 = f32[3,16] maximum(broadcast.11, add.7)
21  broadcast.9 = f32[3,16] broadcast(D), dimensions=[1]
22  multiply.10 = f32[3,16] multiply(relu.8, broadcast.9)
23  clamp.13 = f32[3,16] clamp(broadcast.11, multiply.10, broadcast.12)
24  convert.14 = u8[3,16] convert(clamp.13)
25  ROOT tuple.15 = tuple(dot.3, convert.14)
26 }

```

Fig. 56. XLA-HLO code for kernel K5. The ROOT tuple packages the two outputs and is not counted as an operator in Table 13.

```

1 tilezero(dst = tmm0)
2 tilezero(dst = tmm1)
3 tilezero(dst = tmm2)
4 tilezero(dst = tmm3)
5 tileloadadd(dst = tmm4, mem_addr = 0x0000)
6 tileloadadd(dst = tmm5, mem_addr = 0x0400)
7 tileloadadd(dst = tmm6, mem_addr = 0x0800)
8 tileloadadd(dst = tmm7, mem_addr = 0x0c00)
9 tdpbusd(dst = tmm0, src0 = tmm4, src1 = tmm6)
10 tdpbusd(dst = tmm1, src0 = tmm4, src1 = tmm7)
11 tdpbusd(dst = tmm2, src0 = tmm5, src1 = tmm6)
12 tdpbusd(dst = tmm3, src0 = tmm5, src1 = tmm7)
13 tilestored(src = tmm0, mem_addr = 0x1000)
14 tilestored(src = tmm1, mem_addr = 0x1400)
15 tilestored(src = tmm2, mem_addr = 0x1800)
16 tilestored(src = tmm3, mem_addr = 0x1c00)
17 vmovupsz(dst = zmm10, mem_addr = 0x0000)
18 vmovupsz(dst = zmm15, mem_addr = 0x0400)
19 mov(dst = ebx, imm = 0x437f0000)
20 vmovq(dst = xmm9, src = rbx)
21 vbroadcastss(dst = zmm9, src = xmm9)
22 vmovupsz(dst = zmm31, mem_addr = 0x0800)
23 vcvt dq2ps(dst = zmm31, src = zmm31)
24 vmovupsz(dst = zmm30, mem_addr = 0x0c00)
25 vcvt dq2ps(dst = zmm30, src = zmm30)
26 vmovupsz(dst = zmm29, mem_addr = 0x1000)
27 vcvt dq2ps(dst = zmm29, src = zmm29)
28 vmulps(dst = zmm31, src0 = zmm31, src1 = zmm15)
29 ...
30 vaddps(dst = zmm31, src0 = zmm31, src1 = zmm10)
31 ...
32 vmaxps(dst = zmm31, src0 = zmm31, src1 = zmm8)
33 ...
34 vbroadcastssl(dst = zmm0, mem_addr = 0x144)
35 vmulps(dst = zmm31, src0 = zmm31, src1 = zmm0)
36 ...
37 vmaxps(dst = zmm31, src0 = zmm31, src1 = zmm8)
38 vminps(dst = zmm31, src0 = zmm31, src1 = zmm9)
39 vcvt ps2dq(dst = zmm31, src = zmm31)
40 vpmovusdbx(src = zmm31, mem_addr = 0x144)
41 ...

```

Fig. 57. Compiled assembly code for kernel K5. The AMX half is emitted first and is identical to Fig. 45; the AVX512 half follows. The immediate 0x437f0000 is the fp32 encoding of 255.0, broadcast into zmm9 as the clamp upper bound.